

Machine Learning

Máster en Innovación Tecnológica (MINT)

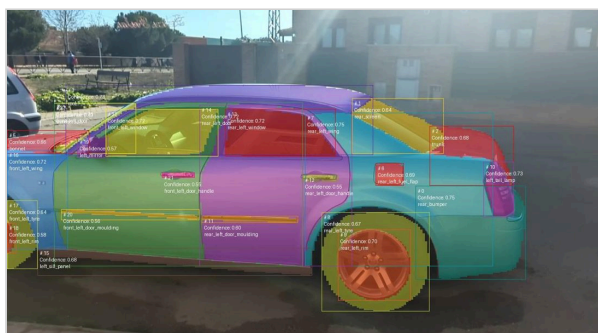
Introduction · Python for machine learning

imanolgranada@gmail.com



Who is your teacher?

- **Imanol Granada** (imanolgranada@gmail.com)
- BSc in Telecommunication Engineering, 2015
- MSc in Telecommunication Engineering, 2017
- PhD in Telecommunication Engineering, 2021
- Data Scientist at **Bdeo**
- My interests:
 - Machine Learning
 - Deep Learning



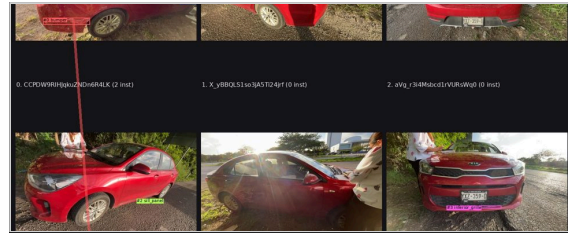
Part segmentation and damage detection on a car photo.



What I have worked on: cars



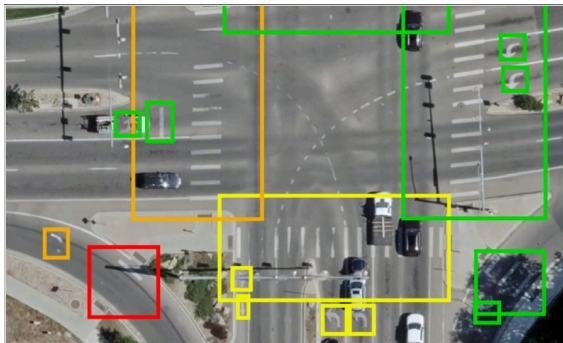
Find the damage in a photo of a vehicle (detection and segmentation).



Decide which photos show the same vehicle and the same damage.

The models you will build in this course are the building blocks: a classifier that says *damaged or not* is logistic regression; one that finds *where* is a CNN (neural-network part, chapter 4).

What I have worked on: roads and sport



Aerial images of roads: each detected element gets a condition from poor to good.



Live football: the probability that a shot becomes a goal.

What about you?

Raise your hand if you have...

- written **Python** before? Used a notebook (Jupyter, Colab)?
- used **NumPy** or **pandas**?
- trained a machine learning model, with scikit-learn or anything else?
- seen the maths of **gradient descent** or a derivative of a loss?

And, in one sentence:

- What is your background?
- What do you want to be able to do after the course?
- What data or problem would you like to apply it to?

There is no wrong answer. The next slides are for everyone: they fix the Python that all the labs use, so that in class we talk about machine learning and not about syntax.

Course overview: [ML-course-labs](#)

Class slides

One deck per topic, with the derivations step by step and interactive figures. → advances every step.

[slides/machine_learning](#)

Course notes

Web pages that go deeper than the slides: full derivations, the assumptions of each method, how to check them and what to do when they fail.

[notes/](#)

Lab notebooks

Jupyter notebooks that open in Google Colab, nothing to install. You implement the algorithm and see it work.

[notebooks/](#)

Part	Topics	Labs
0	Python, NumPy, pandas, Matplotlib (this deck)	Lab 0
I	Supervised learning, linear and logistic regression, softmax, regularization, trees, ensembles, SVM, time series	Labs 1–6 and 16 (scikit-learn, from scratch in NumPy)
II	MLP, backpropagation, training, CNN, RNN and LSTM, transformers, embeddings and RAG	Labs 7–15 (NumPy, then PyTorch)

Python: why?

Readable

Fewer lines of code, high level: the code looks like the idea.

Free and open source

Cross-platform: the same code on your laptop, Colab or a server.

Extensive libraries

NumPy, pandas, scikit-learn, Matplotlib, PyTorch... the whole ML ecosystem is Python.

Object oriented

Models are objects: `model.fit(X, y)`. Built-in data structures: list, dict, set.

Drawback: Python is an interpreted scripting language. That makes it convenient but slower than C or C++. The fix: the libraries do the heavy work in compiled C/C++ (and CUDA) under the hood, and we call them on whole arrays instead of looping in Python. We measure it on slide 22.

This is the part of the language you need, not a complete Python course. Everything below is in **Lab 0**, which you should run before the first class.

Python: fundamentals for ML

Part of Lab 0	What it is	Why it matters in the labs
1 Python essentials	types, containers, control flow, functions, comprehensions	every notebook
2 Object-oriented programming	classes, <code>self</code> , inheritance	models are classes: <code>fit</code> / <code>predict</code> , PyTorch <code>nn.Module</code>
3 NumPy	arrays, indexing, vectorisation, <code>axis</code> , broadcasting, <code>@</code>	vectors, matrices and gradients: the from-scratch labs
4 pandas	Series, DataFrame, <code>loc</code> / <code>iloc</code> , <code>groupby</code> , NaN	loading and inspecting datasets
5 Matplotlib	scatter, line, histogram, subplots	every plot you will produce
6 Putting it together	load → split → scale → fit → evaluate → plot	the skeleton of every lab

Docs you will use all the time: [NumPy](#) · [pandas](#) · [scikit-learn](#) · [Matplotlib](#). In a notebook, `help(np.mean)` or `np.mean?` shows the documentation.

Python: fundamentals for ML

Part 1 of Lab 0

[Python – Notebook](#)

Variables and types

```
n_samples = 150      # int
learning_rate = 0.01  # float
model_name = "logistic" # str
is_fitted = False    # bool
nothing = None        # NoneType

epoch, loss, acc = 7, 0.03421, 0.9812
print(f"epoch {epoch:3d}: loss = {loss:.4f}, "
      f"accuracy = {acc:.2%}")
```

```
# output
epoch    7: loss = 0.0342, accuracy = 98.12%
```

- **Dynamically typed:** no declarations, a name just refers to an object.
- **Strongly typed:** "3" + 4 is an error, Python never converts silently.
- `None` means "no value".

f-strings put expressions inside a string. You will use them to print training metrics:

Format	Meaning
<code>:3d</code>	integer, width 3
<code>:.4f</code>	float, 4 decimals
<code>:.2%</code>	percentage, 2 decimals (0.9812 → 98.12%)

The four containers

Type	Written	Ordered	Mutable	Typical use in ML
list	[1, 2, 3]	yes	yes	a sequence you modify, e.g. the loss history
tuple	(1, 2, 3)	yes	no	a fixed record: array shapes are tuples
dict	{"a": 1}	yes (insertion)	yes	named values: hyperparameters, config
set	{1, 2, 3}	no	yes	unique values: the set of class labels

```
losses = [0.9, 0.6, 0.4]
losses.append(0.3)
losses[0], losses[-1] # 0.9 0.3
losses[1:3]           # [0.6, 0.4]

shape = (150, 4)
n_rows, n_cols = shape # unpacking
```

```
params = {"lr": 0.01, "epochs": 100}
params["batch_size"] = 32
params.get("momentum", 0.9) # 0.9: default

labels = [0, 1, 1, 0, 2, 1, 2]
set(labels) # {0, 1, 2}
```

Careful: slicing is [start:stop] with **stop excluded**. `losses[1:3]` gives elements 1 and 2. The same convention holds in NumPy and in `df.iloc`.

Control flow

Indentation (4 spaces) delimits blocks, there are no braces.

```
accuracy = 0.87
if accuracy > 0.95:
    verdict = "excellent"
elif accuracy > 0.80:
    verdict = "acceptable"
else:
    verdict = "needs work" # → "acceptable"

for loss in [0.9, 0.6, 0.4]: # the elements
    print(f"loss={loss}")

for i, loss in enumerate([0.9, 0.6, 0.4]):
    print(f"epoch {i}: {loss}") # index and value
```

`zip` walks two sequences in parallel: perfect for labels and predictions.

```
y_true = [0, 1, 1, 0]
y_pred = [0, 1, 0, 0]
for true, pred in zip(y_true, y_pred):
    mark = "ok" if true == pred else "WRONG"
    print(f"true={true} pred={pred} {mark}")
```

```
# output
true=0 pred=0 ok
true=1 pred=1 ok
true=1 pred=0 WRONG
true=0 pred=0 ok
```

Functions and docstrings

```
def accuracy_score(y_true, y_pred, as_percentage=False):
    """Fraction of correct predictions."""
    n_correct = sum(t == p
                    for t, p in zip(y_true, y_pred))
    score = n_correct / len(y_true)
    return score * 100 if as_percentage else score

y_true = [0, 1, 1, 0, 1]
y_pred = [0, 1, 0, 0, 1]
accuracy_score(y_true, y_pred)          # 0.8
accuracy_score(y_true, y_pred,
               as_percentage=True)      # 80.0
```

Check by hand: 4 of the 5 predictions match → 4/5 = 0.8.

- `def` defines a function; arguments with a **default value** are optional.
- Pass by name (`as_percentage=True`) to make calls readable: scikit-learn does this everywhere.
- The text in triple quotes is the **docstring**. `help(f)` or `f?` shows it: that is how you read the scikit-learn documentation.

Try it: `help(np.mean)` prints every argument of a NumPy function.

Comprehensions

Read it as: [**expression** | **for each item** | **if condition**]

```
values = [1, 2, 3, 4, 5, 6]

squares = []          # the verbose way
for v in values:
    squares.append(v ** 2)

squares = [v ** 2 for v in values]
evens = [v for v in values if v % 2 == 0]
labels = ["high" if v > 3 else "low"
          for v in values]
squared_map = {v: v ** 2 for v in values}
```

```
# results
squares      [1, 4, 9, 16, 25, 36]
evens        [2, 4, 6]
labels       ['low']*3 + ['high']*3   # 3 + 3
squared_map  {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36}
```

Exercise 1: standardise a list (built-in Python only)

$$z_i = \frac{x_i - \mu}{\sigma}, \quad \mu = \frac{1}{n} \sum_i x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_i (x_i - \mu)^2}$$

Solution: `mu = sum(v)/n, sigma = (sum((x-mu)**2 for x in v)/n) ** 0.5, return [(x-mu)/sigma for x in v]`. For [10, 12, 14, 16, 18]: $\mu = 14$, $\sigma = \sqrt{8} \approx 2.83 \rightarrow z = [-1.414, -0.707, 0, 0.707, 1.414]$, mean 0.

Object-oriented programming

Part 2 of Lab 0
why `model.fit(X, y)` works

Why you need classes

Every model in this course is an **object**:

```
model = LogisticRegression(C=1.0) # create
model.fit(X_train, y_train)      # changes the state
y_pred = model.predict(X_test)   # uses that state
print(model.coef_)               # learnt in fit
```

That `__init__` / `fit` / `predict` pattern **is** the scikit-learn API.
PyTorch models follow the same idea with `nn.Module`.

class	a blueprint
instance	one concrete object built from the class
<code>__init__</code>	the constructor: runs on creation, stores the configuration
<code>self</code>	the instance itself: first argument of every method, used to read and write attributes that survive between calls
attribute	<code>data.name</code> : no parentheses
method	<code>data.n_samples()</code> : with parentheses

A class, step by step

```
class Dataset:
    """A tiny container for features and labels."""

    def __init__(self, X, y, name="unnamed"):
        self.X = X          # attributes of THIS one
        self.y = y
        self.name = name

    def n_samples(self):
        return len(self.y)

    def class_counts(self):
        counts = {}
        for label in self.y:
            counts[label] = counts.get(label, 0) + 1
        return counts
```

```
data = Dataset(X=[[1, 2], [3, 4], [5, 6], [7, 8]],
               y=[0, 1, 1, 1], name="toy")
```

```
data.name          # 'toy'
data.n_samples()   # 4
data.class_counts() # {0: 1, 1: 3}
```

Dunder methods (`__like_this__`) hook into Python syntax.

`__len__` gives `len(data)`; `__repr__` is what a notebook shows for the object: `Dataset(name='toy', n=4, n_features=2)`.

PyTorch's `forward` runs through `__call__`, which is why you write `model(x)` and not `model.forward(x)`.

Inheritance: the scikit-learn contract

```
class BaseEstimator:
    def fit(self, X, y):
        raise NotImplementedError
    def predict(self, X):
        raise NotImplementedError
    def score(self, X, y): # inherited for free
        preds = self.predict(X)
        errors = [(p - t) ** 2
                  for p, t in zip(preds, y)]
        return sum(errors) / len(errors) # MSE

class MeanRegressor(BaseEstimator):
    """Predicts the training mean."""
    def fit(self, X, y):
        self.mean_ = sum(y) / len(y)
        return self # so calls chain
    def predict(self, X):
        return [self.mean_ for _ in X]
```

```
X = [[1], [2], [3], [4], [5]]
y = [2.0, 4.1, 5.9, 8.2, 9.8]
model = MeanRegressor().fit(X, y)
model.mean_ # 6.0
model.predict([[10]]) # [6.0]
model.score(X, y) # 7.78 (MSE)
```

- A subclass **inherits** everything and can override it. `class Net(nn.Module)` in PyTorch works the same way.
- `self.mean_` with a **trailing underscore** = learnt from data in `fit`. Hyperparameters, set in `__init__`, have none.
- `fit` **returns self**, so `Model().fit(X, y).predict(X)` works.
- A model is only good relative to a **baseline** like this one.

NumPy

Part 3 of Lab 0
vectors, matrices, gradients

NumPy arrays: shape and dtype

An array is a **contiguous block of memory of one type**: a vector, a matrix, or a stack of matrices. You operate on it as a whole.

```
a = np.array([1, 2, 3, 4])          # shape (4,)
M = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3)
np.zeros(3)                        # [0. 0. 0.]
np.arange(0, 10, 2)                 # [0 2 4 6 8] stop excluded
np.linspace(0, 1, 5)                # 5 points, both ends
np.eye(3)                           # identity matrix
```

When a lab throws an error it is nearly always a **shape mismatch**: print `.shape` first.

Shape conventions in ML

- **X**: shape **(n_samples, n_features)**, one row per observation (our $m \times n$).
- **y**: shape **(n_samples,)**, a 1-D vector of targets.

```
X = np.array([[5.1, 3.5], [4.9, 3.0],
               [4.7, 3.2], [4.6, 3.1]])
y = np.array([0, 0, 1, 1])
X.shape          # (4, 2): 4 samples, 2 features
y.shape          # (4,)
y_col = y.reshape(-1, 1)
y_col.shape      # (4, 1): a column
y_col.reshape(-1).shape # (4,): flat again
```

A very common bug is **y** with shape **(n, 1)** instead of **(n,)**. **-1** means "work this dimension out for me".

Indexing, slicing and boolean masks

For 2-D arrays: `array[rows, columns]`. Here `M = np.arange(12).reshape(3, 4)`:

	col 0	col 1	col 2	col 3
row 0	0	1	2	3
row 1	4	5	6	7
row 2	8	9	10	11

```
M[0, 0] # 0
M[1]    # [4 5 6 7] row 1
M[:, 2] # [2 6 10] column 2 (':' = all)
M[0:2, 1:3] # [[1 2], [5 6]] the blue block
M[-1]      # [8 9 10 11] last row
```

Boolean masking: the most useful idiom

```
values = np.array([3.2, 7.8, 1.1, 9.4, 5.0])
mask = values > 5 # [F T F T F]
values[mask]      # [7.8 9.4]
mask.sum()        # 2 (True counts as 1)
values[(values > 2) & (values < 6)] # [3.2 5.]
```

Combine conditions with `&` and `|`, **each one in parentheses**.

You will do this in every classification lab: select the rows of one class.

```
X[y == 1] # [[4.7 3.2], [4.6 3.1]]
```

Vectorisation: no loops over elements

Operations apply **elementwise to the whole array**, in compiled C code.

```
a = np.array([1., 2., 3., 4.])
b = np.array([10., 20., 30., 40.])
a + b # [11. 22. 33. 44.]
a * b # [10. 40. 90. 160.] elementwise!
a ** 2 # [1. 4. 9. 16.]
a * 3 # [3. 6. 9. 12.]
np.sqrt(a) # [1. 1.414 1.732 2.]
np.exp(a) # [2.72 7.39 20.09 54.6]
```

`a * b` is **not** a matrix product. That one is `@` (slide 25).

```
big = np.random.rand(2_000_000)
[v ** 2 for v in big] # pure Python loop
big ** 2 # NumPy
```

Square 2 million numbers	Time
Python loop	806 ms
NumPy	25 ms
speed-up	32x

Measured in Lab 0 on one run (your machine will differ, the order of magnitude will not). The same reason is behind GPU training in the neural-network part.

Reductions and the `axis` argument

`axis` is the dimension that gets **collapsed**.

	feature 0	feature 1	axis=1 →
sample 0	1	10	5.5
sample 1	2	20	11.0
sample 2	3	30	16.5
sample 3	4	40	22.0
axis=0 ↓	2.5	25.0	

- `axis=0`: collapse the rows → **one value per column** (per feature).
- `axis=1`: collapse the columns → **one value per row** (per sample).

```
X.shape           # (4, 2)
X.mean()          # 13.75  everything → scalar
X.mean(axis=0)    # [ 2.5 25. ]      per feature
X.mean(axis=1)    # [5.5 11. 16.5 22.] per sample
X.sum(), X.min(), X.max() # 110.0 1.0 40.0
X.std(axis=0)     # [1.118 11.180]
X.argmax(axis=1)  # [1 1 1 1] index of the largest
```

Say it aloud: "*axis=0 gives me a value per feature*". Getting it backwards is a classic silent bug.

`argmax(axis=1)` turns a matrix of class probabilities into predicted labels: the exact line you will use in the multiclass labs.

Broadcasting: standardise in one line

```
mu = X.mean(axis=0) # shape (2,)
sd = X.std(axis=0)  # shape (2,)
X_scaled = (X - mu) / sd # (4,2) - (2,) → (4,2)
```

X (4×2)		mu stretched		X_scaled
1 10	-	2.5 25	→ ÷ sd →	-1.342 -1.342
2 20	-	2.5 25		-0.447 -0.447
3 30	-	2.5 25		0.447 0.447
4 40	-	2.5 25		1.342 1.342

Check row 0, feature 0: $(1 - 2.5) / 1.118 = -1.342$. New mean 0 and std 1 per feature.

The rule: compare the shapes **from the right**. Two dimensions are compatible if they are equal or one of them is 1.

(4, 2) vs (2,): the (2,) is treated as (1, 2) → compatible → stretched to (4, 2). No data is copied.

```
np.ones((4, 2)) + np.ones((3,))
# ValueError: operands could not be broadcast
# together with shapes (4,2) (3,)
```

2 vs 3: neither equal nor 1, so it fails. The error message names the clashing shapes: read it.

Linear algebra: the OLS preview

@ is the matrix product, the operation behind every linear model and every neural-network layer.

```
A = np.array([[1, 2], [3, 4]]); v = np.array([1, 1])
A @ v      # [3 7]
A @ A      # [[ 7 10], [15 22]]
A.T        # [[1 3], [2 4]] transpose
np.linalg.inv(A) # [[-2.  1.], [1.5 -0.5]]
```

Check the first one: row (1, 2) · (1, 1) = 3 and row (3, 4) · (1, 1) = 7.

Lab 1 solves least squares with the normal equation. The maths maps directly onto the code:

$$\hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

```
X = np.column_stack([np.ones(n), x]) # (100, 2)
theta = np.linalg.inv(X.T @ X) @ X.T @ y
```

100 points from $y = 3 + 2x$ plus noise (seed 0): estimated intercept **2.993** (true 3.0), slope **1.987** (true 2.0). The column of ones is the intercept.

Reproducible random numbers

Always seed your generator. Without a seed the results change on every run and nobody can reproduce yours.

```
rng = np.random.default_rng(seed=42)
rng.random(3)      # [0.774 0.439 0.859]
rng.normal(0, 1, 3) # [ 0.941 -1.951 -1.302]
rng.integers(0, 10, 5) # [7 7 7 5]
rng.permutation(10) # shuffled indices 0..9

np.random.default_rng(42).random(3) # same first line
```

- Same seed → same numbers, on every machine.
- Modern API: `np.random.default_rng(seed)`. In old code you will see `np.random.seed(42)`.
- In scikit-learn the equivalent is `random_state=42`, e.g. in `train_test_split`.

Exercise 3

With $X = \begin{bmatrix} 1 & 2 \\ 3 & 6 \\ 5 & 4 \\ 7 & 8 \end{bmatrix}$: (1) mean of each column, (2) rows with first column > 3, (3) standardise per column, (4) shape of $X @ X.T$.

Answers: (1) `X.mean(axis=0)` = [4, 5]; (2) `X[X[:, 0] > 3]` = [[5,4],[7,8]]; (3) `(X - X.mean(axis=0)) / X.std(axis=0)`; (4) (4, 4).

pandas

Part 4 of Lab 0
tables with named columns

pandas: Series and DataFrame

NumPy arrays are homogeneous and anonymous. Real data has **named columns of different types**, text, and missing values.

- **Series**: one column, a 1-D array with an index.
- **DataFrame**: a table, a dict of Series sharing one index.

The workflow in every lab: **load with pandas → explore and clean → hand a NumPy array to the model.**

```
df = pd.DataFrame({
    "sepal_length": [5.1, 4.9, 4.7, 4.6, 6.5, 6.0, 6.7, 5.8],
    "sepal_width": [3.5, 3.0, 3.2, 3.1, 3.0, 2.7, 3.1, 2.7],
    "petal_length": [1.4, 1.4, 1.3, 1.5, 5.2, 5.1, 4.7, 5.1],
    "species": ["setosa"]*4 + ["virginica"]*2
               + ["versicolor", "virginica"]})

df.shape          # (8, 4)
df.head(3)        # first rows
df.info()          # dtypes and non-null counts
df.describe()      # mean, std, quartiles
df["species"].value_counts()
# setosa 4, virginica 3, versicolor 1
```

Those first commands are what you run on any dataset you have never seen. `value_counts()` shows the **class balance**.

Selecting and filtering

Code	Returns
<code>df["col"]</code>	one column, a Series
<code>df[["a", "b"]]</code>	several columns, a DataFrame (double brackets)
<code>df.loc[rows, cols]</code>	selection by label
<code>df.iloc[rows, cols]</code>	selection by integer position

`df.loc[0:2]` includes row 2 (3 rows); `df.iloc[0:2]` excludes position 2 (2 rows). `iloc` follows normal Python slicing, `loc` does not.

Filtering is the NumPy mask again:

```
df[df["petal_length"] > 5.0]      # 3 rows

df[(df["species"] == "virginica")
   & (df["sepal_width"] < 3.0)]  # 2 rows

df[df["species"].isin(["setosa", "versicolor"])].shape
# (5, 4)
```

As in NumPy: `&` and `|`, each condition in parentheses.

New columns, groups and missing values

```
# vectorised: the preferred way
df["petal_ratio"] = (df["petal_length"]
                    / df["sepal_length"])

# elementwise, if no vectorised version exists
df["size"] = df["petal_length"].apply(
    lambda v: "large" if v > 4 else "small")

# models need numbers, not strings
df["target"] = df["species"].map(
    {"setosa": 0, "versicolor": 1, "virginica": 2})
```

groupby = split → apply → combine:

```
df.groupby("species")["petal_length"].mean()
# setosa 1.400 versicolor 4.700 virginica 5.133
```

Real data has holes (NaN). You must decide what to do: dropping and imputing are different modelling choices.

```
dirty = df[["sepal_length", "sepal_width",
            "petal_length"]].copy()
dirty.loc[1, "sepal_width"] = np.nan
dirty.loc[5, "petal_length"] = np.nan

dirty.isna().sum()      # 0, 1, 1 per column
dirty.dropna().shape    # (6, 3): 2 rows lost
dirty.fillna(dirty.mean()) # NaN → column mean
```

Imputing with the mean of the whole dataset before splitting leaks test information: compute it on the training set only (slide 33).

From a DataFrame to a model

```
feature_cols = ["sepal_length", "sepal_width",
                "petal_length"]
X = df[feature_cols].to_numpy() # (8, 3) float64
y = df["target"].to_numpy()    # (8,) int64

# data usually comes from a file:
df = pd.read_csv("data/data.csv")
df = pd.read_csv("https://../data.csv")

read_csv options: sep=";", header=None, names=[...],
na_values=["?", "N/A"], usecols=[...].
```

Exercise 4, using the DataFrame above

1. Samples of each species ?
2. Mean `sepal_width` of virginica only?
3. A column `is_setosa` = 1 / 0?
4. Only the numeric columns, sorted by `petal_length` descending?

```
value_counts() → 4 / 3 / 1.
df.loc[df["species"]=="virginica",
"sepal_width"].mean() = 2.8.
(df["species"]=="setosa").astype(int) .
df.select_dtypes("number").sort_values("petal_length",
ascending=False)
```

Matplotlib in five minutes

```
x = np.linspace(0, 10, 50)
y = 2 * x + 1 + rng.normal(0, 2, size=50)

plt.figure(figsize=(6, 4))
plt.scatter(x, y, alpha=0.7, label="observations")
plt.plot(x, 2 * x + 1, color="red", linewidth=2,
        label="true relationship")
plt.xlabel("x"); plt.ylabel("y")
plt.title("Always label your axes")
plt.legend(); plt.grid(alpha=0.3)
plt.show()
```

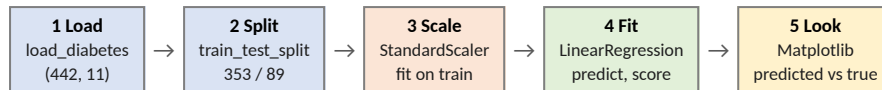
Three things are enough: a **scatter**, a **line**, and the **labels**.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
axes[0].hist(rng.normal(0, 1, 1000), bins=30)
axes[0].set_title("Histogram")
axes[1].plot(losses, marker="o")
axes[1].set_title("Training loss")
plt.tight_layout(); plt.show()
```

`fig` is the canvas, `axes` the individual plots. With `subplots` the methods change name: `plt.xlabel` → `ax.set_xlabel`, `plt.title` → `ax.set_title`.

Training-loss curves like the right panel are how you will diagnose every model in the course.

Putting it together: an ML workflow in 5 steps



```

X = df.drop(columns="target").to_numpy()
y = df["target"].to_numpy()
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler().fit(X_train)      # TRAIN only
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)

model = LinearRegression().fit(X_train_s, y_train)
y_pred = model.predict(X_test_s)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
base = mean_squared_error(
    y_test, np.full_like(y_test, y_train.mean())) ** 0.5
  
```

Test set (89 samples)	Value
Baseline RMSE (predict the mean)	73.22
Linear regression RMSE	53.85
R ²	0.453

The model beats the baseline, so it learnt something. 0.453 means it explains under half the variance: good enough for a first model, not for a final one.

Why fit the scaler on the training set only? Fitting it on all the data lets information about the test set (its mean and std) leak into training, and the test score becomes optimistic. This is the **data leakage** of chapter 01.

Before the first class: can you answer these?

- Difference between a `list` and a `dict`, and when to use each?
- What does `self` refer to inside a method?
- What do `fit` and `predict` do, and why does `fit` return `self`?
- With `X.shape == (150, 4)`, how many samples and features?
- What does `X.mean(axis=0)` return, and with what shape?
- Why is `(X - X.mean(axis=0)) / X.std(axis=0)` valid despite the shape mismatch?
- How do you select the rows of `X` whose label is 1?
- Difference between `df.loc` and `df.iloc`?
- Why must the scaler be fitted on the training set only?

If any is shaky, go back to that part of Lab 0: the labs assume all of them. In Colab: **Shift+Enter** runs a cell, **Tab** autocompletes, **Shift+Tab** shows a function's signature, object? its docstring.

More: [NumPy absolute basics](#) · [10 minutes to pandas](#) · [scikit-learn getting started](#) · Python Data Science Handbook (J. VanderPlas, free online)

LAB 0

Python for Machine Learning

The shared starting point of the course: the subset of Python, NumPy and pandas that the labs use. About 60–90 minutes with the exercises.

- 1 Python essentials: types, containers, control flow, functions, comprehensions (Exercise 1).
- 2 Object-oriented programming: classes, dunder methods, inheritance, the `fit / predict` contract.
- 3 NumPy: shapes, masks, vectorisation, `axis`, broadcasting, `@`, seeds (Exercise 3).
- 4 pandas: inspect, select, filter, group, missing values (Exercise 4).
- 5 Matplotlib, then a complete mini workflow: load, split, scale, fit, evaluate, plot.

[CO Open in Colab](#)

notebook `Lab 0 - Python for Machine Learning.ipynb`
you need nothing installed: a Google account for Colab
runs on CPU
tip run every cell, change things, break them. Each exercise has a solution cell: try first. *File* → *Save a copy in Drive* keeps your changes

Next: chapter 01, supervised learning, then Lab 1 (OLS from scratch).

Machine Learning

Máster en Innovación Tecnológica (MINT)

Supervised learning · Train, validation and test

imanolgranada@gmail.com



1

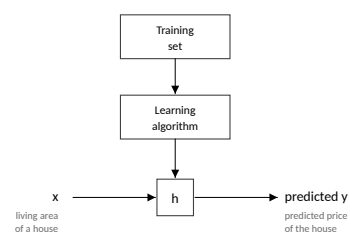
Supervised Learning

Notation:

- m training samples, n features
- $x_j^{(i)}$ feature j of sample i
- $\mathbf{x}^{(i)}$ feature vector i (size $n \times 1$)
- $y^{(i)}$ target i
- $(\mathbf{x}^{(i)}, y^{(i)})$ training pair i
- $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ training set
- $\mathcal{X}$ input space, $\mathcal{Y}$ output space
- $h : \mathcal{X} \rightarrow \mathcal{Y}$ hypothesis (the model)

Objective:

- given a training set
- learn a function h so that $h(\mathbf{x})$ is a "good" predictor of y on **new data**.

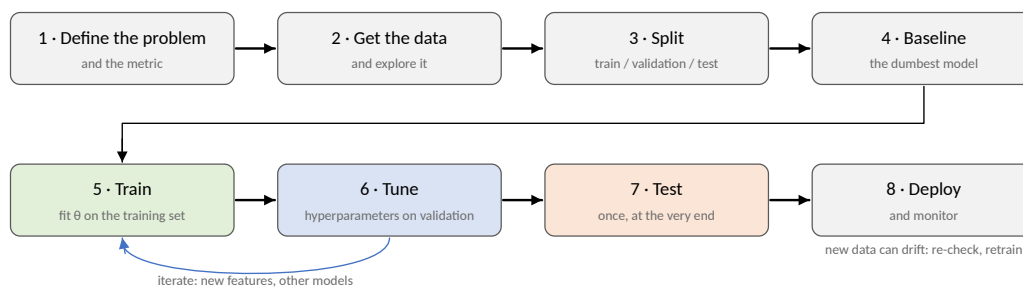


- y continuous $\rightarrow$ **regression** (a price, a cost, a temperature)
- y discrete $\rightarrow$ **classification** (fraud or not, a damage type)



2

The machine learning workflow



Decide the metric first

What counts as "good"? Euros of error, frauds caught, false alarms... Choose it before training (metrics: chapters 02 and 03).

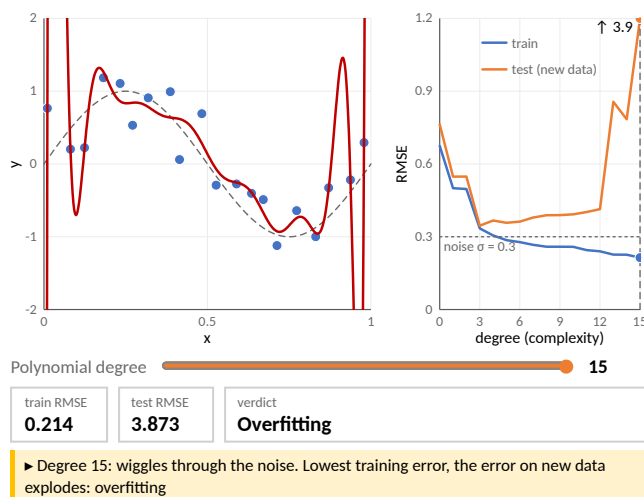
Always a baseline

The dumbest model: predict the mean (regression) or the most frequent class. Yours must beat it. With 5 % fraud, "never fraud" is already 95 % accurate.

Test once, at the end

Every decision is made with the training and validation data. The test set only gives the final number.

Underfitting and overfitting



20 noisy points from the dashed curve. We fit polynomials of growing degree and measure the error on those 20 points (train) and on 300 new points (test).

- **Underfitting:** the model is too simple for the pattern. High error on the training data **and** on new data.
- **Overfitting:** the model is flexible enough to memorize the noise. Low training error, high error on new data.
- The training error only goes **down** as the model gets more complex, so it cannot choose the complexity. We need data the model has not seen.

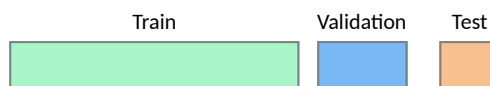
The degree is a **hyperparameter**: a choice made before training. Parameters θ are learned on the training set; hyperparameters are chosen on the validation set.

Dataset: Train – Validation – Test

Set	Used for	Typical size
Training	fitting the parameters θ : m samples	60–80 %
Validation	choosing the hyperparameters (degree, λ , tree depth...) and comparing models	10–20 %
Test	one final, unbiased estimate, on data never used before	10–20 %

Why a separate test set? After choosing the best of many models on the validation set, its validation score is **optimistic**: we picked the winner of a contest on that very data. The test set is an independent referee.

If you change the model after looking at the test score, the test set has become a validation set.



Important:

- Same distribution in the three parts.
- No data leakage (two slides ahead).
- Proportions? They depend on the number of hyperparameters and of samples: (70-15-15), (70-20-10), (80-10-10), (90-7-3). With millions of rows, 1 % is plenty for validation and test.

Dataset: Train – Validation – Test

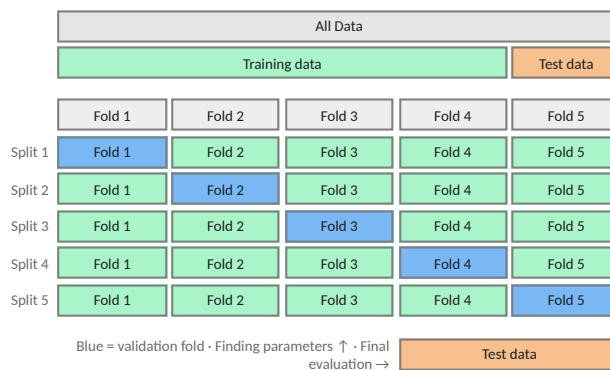
What if we only have a small dataset?

K-fold cross validation:

1. Separate test data.
2. Split the rest into K folds.
3. Calculate K metrics:
 - Train on K-1 folds.
 - Validate on the remaining one.
4. Average the K metrics (and report their spread).

$$CV = \frac{1}{K} \sum_{k=1}^K E_k$$

A frequent choice of K is 5 or 10. Every sample is used for validation exactly once. After choosing the hyperparameters, **refit on all the non-test data**, then evaluate once on the test set.



Splitting correctly: leakage, classes, groups, time

Data leakage: information from the validation or test data (or from the future) sneaks into training. The scores look great, and the model fails in production.

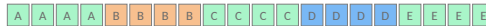
- **Preprocessing fitted on all the data**, e.g. the mean and standard deviation used for scaling. Compute them on the training part only and apply them to the rest. A scikit-learn `Pipeline` does this automatically, fold by fold.
- **Features known only after the outcome**, e.g. "claim paid amount" to predict fraud when the claim is opened.

Classification: **stratify** the split so every part keeps the class proportions (`stratify=y`, `StratifiedKFold`). Essential when a class is rare.

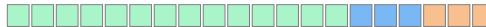
Random, stratified: shuffle the rows (each part keeps the class mix)



Grouped: all the rows of one customer go to the same part



Time: train on the past, validate and test on the future



train validation test

- **Groups:** several rows of the same customer, patient or car must all go to the same part (`GroupKFold`). Otherwise we validate on near-copies of the training data.
- **Time:** never shuffle. Train on the past, validate on the future (`TimeSeriesSplit`). Lab 16, at the end of the ML part, is all about this.

Machine Learning

Máster en Innovación Tecnológica (MINT)

Next: Linear Regression

imanolgranada@gmail.com

Machine Learning

Máster en Innovación Tecnológica (MINT)

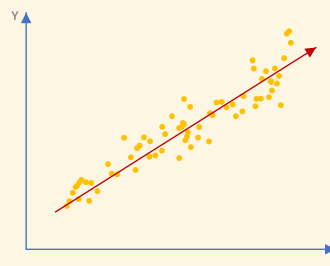
Linear Regression

imanolgranada@gmail.com



1

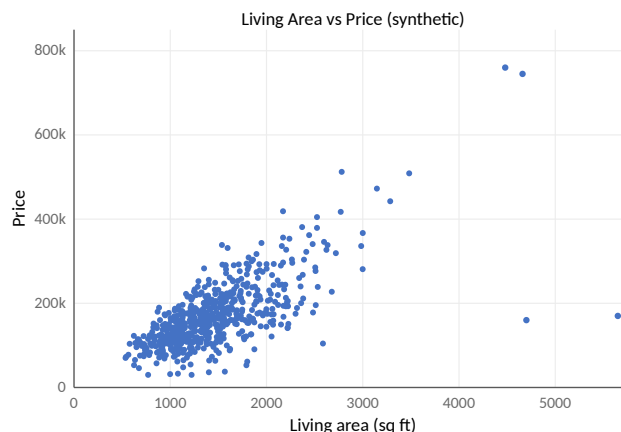
**Linear
Regression**



2

Linear Regression – Real World examples.

- Market sales.
- Medical insurance costs.
- Real Estate prices.
- Red Wine Quality.
- Taxi Fares.
- Life Expectancy.



Linear Regression – Start simple

The simplest possible model: a line through the origin, with a single parameter, the slope θ :

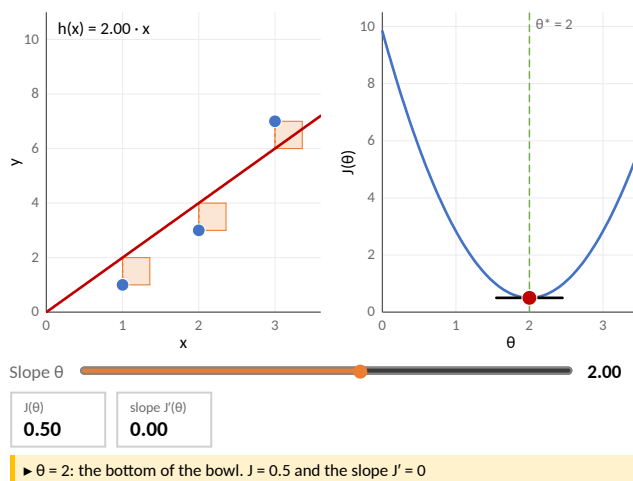
$$h_{\theta}(x) = \theta x$$

Three training points: $(1, 1)$, $(2, 3)$, $(3, 7)$. Which slope fits them best?

Measure the fit with a **cost**: half the average squared error,

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta x^{(i)} - y^{(i)})^2$$

Each orange square has side = the error of one point, so J is their average area, halved (the $\frac{1}{2}$ only makes the derivative cleaner). The black segment on the right is the slope $J'(\theta)$.



Linear Regression – The best θ , step by step

At the bottom of the bowl the slope is zero. Differentiate J with respect to θ :

$$J'(\theta) = \frac{1}{2m} \sum_{i=1}^m 2(\theta x^{(i)} - y^{(i)}) \cdot x^{(i)} = \frac{1}{m} \sum_{i=1}^m (\theta x^{(i)} - y^{(i)}) x^{(i)}$$

chain rule: the derivative of $(\theta x - y)^2$ is $2(\theta x - y) \cdot x$; the 2 cancels the $\frac{1}{2}$

$$= \frac{1}{m} \left(\theta \sum_{i=1}^m (x^{(i)})^2 - \sum_{i=1}^m x^{(i)} y^{(i)} \right) = 0$$

split the sum in two; θ is the same in every term, so it comes out

$$\theta^* = \frac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2}$$

With our three points $(1, 1)$, $(2, 3)$, $(3, 7)$:

$$\sum_i x^{(i)} y^{(i)} = 1 \cdot 1 + 2 \cdot 3 + 3 \cdot 7 = 28$$

$$\sum_i (x^{(i)})^2 = 1 + 4 + 9 = 14$$

$$\theta^* = \frac{28}{14} = 2$$

$$J(2) = \frac{1}{6} \left[(2 - 1)^2 + (4 - 3)^2 + (6 - 7)^2 \right] = \frac{3}{6} = 0.5$$

$m = 3$, so $\frac{1}{2m} = \frac{1}{6}$; the three errors are $+1$, $+1$, -1

No iterations needed: a **closed-form** solution. On the previous slide, $\theta = 2$ is exactly where the slope J' is 0.

Linear Regression – Is it really the minimum? Convexity

A zero slope could also be a maximum. Expand the square inside J :

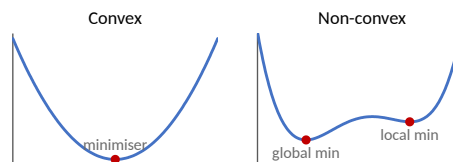
$$\begin{aligned} J(\theta) &= \frac{1}{2m} \sum_i \left(\theta^2 (x^{(i)})^2 - 2\theta x^{(i)} y^{(i)} + (y^{(i)})^2 \right) \\ (a - b)^2 &= a^2 - 2ab + b^2 \text{ with } a = \theta x^{(i)}, b = y^{(i)} \\ &= \underbrace{\frac{\sum_i (x^{(i)})^2}{2m}}_{a \geq 0} \theta^2 - \underbrace{\frac{\sum_i x^{(i)} y^{(i)}}{m}}_b \theta + \underbrace{\frac{\sum_i (y^{(i)})^2}{2m}}_c \end{aligned}$$

a parabola $a\theta^2 - b\theta + c$ with $a \geq 0$: it opens upwards, a bowl

$$J''(\theta) = 2a = \frac{1}{m} \sum_i (x^{(i)})^2 \geq 0$$

the second derivative is an average of squares: it is never negative

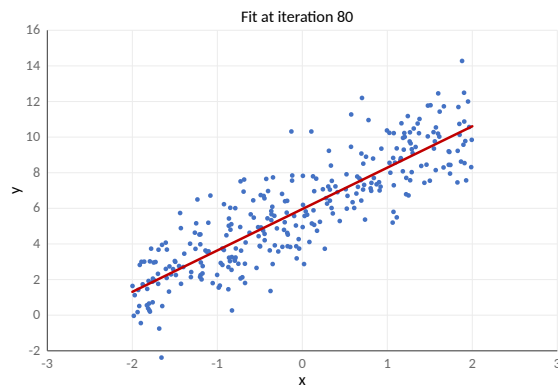
$J'' \geq 0$ everywhere means J is **convex**: one bowl, no other valleys. The point where $J' = 0$ is **the** global minimum, and it is unique as soon as one $x^{(i)} \neq 0$.



Our example: $J(\theta) = \frac{1}{6}(14\theta^2 - 56\theta + 59)$ and $J'' = \frac{14}{3} > 0 \checkmark$

Coming next, in general. Many features and an intercept: $h_{\theta}(\mathbf{x}) = \theta^T \mathbf{x}$. The same three steps (cost $\rightarrow$ derivative $= 0 \rightarrow$ convexity) come back in matrix form: the **normal equation** $\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ and the Hessian $\mathbf{X}^T \mathbf{X}$. With one feature and no intercept they give back $\theta^* = \frac{\sum xy}{\sum x^2}$.

Linear Regression – Many features



Gradient-descent iteration 80

θ_0
5.96

θ_1
2.32

$J(\theta)$
1.19

► Iteration 80: converged, θ stops changing

The general model: n features and an intercept θ_0 .

Approximate y as a linear function of $\mathbf{x}$:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \sum_{j=0}^n \theta_j x_j = \boldsymbol{\theta}^T \mathbf{x}$$

θ 's are called the **parameters** (also called weights). For $n = 1$, θ_0 is the intercept and θ_1 the slope.

Notice that for convenience we made $x_0 = 1$.

How do we select the θ 's?

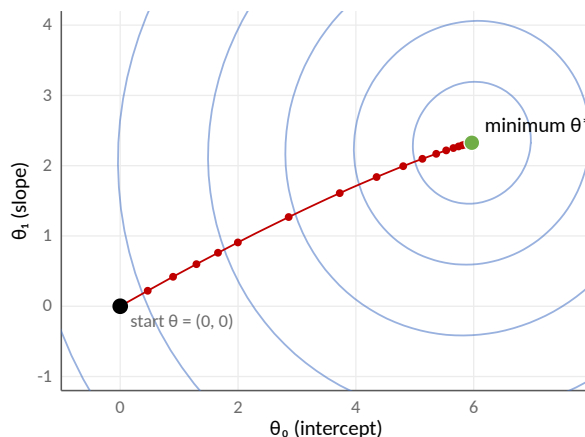
Linear Regression – The cost function

Exactly as for $h = \theta x$: measure, for each θ , how close $h_{\theta}(\mathbf{x}^{(i)})$ is to $y^{(i)}$. The **cost function**:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

The least-squares cost of the **ordinary least squares** (OLS) model. Two ways to minimize it:

1. **Gradient descent**: walk downhill, step by step (next slides).
2. **Set the gradient to zero** and solve: the normal equation, a closed form like $\theta^* = \frac{\sum x_i y_i}{\sum x_i^2}$.



With two parameters $J(\theta_0, \theta_1)$ is still a bowl: its contour lines are rings (ellipses) around the minimum. In red, the 80 gradient-descent steps of the previous slide. Convexity for any n is proved later, with the Hessian.

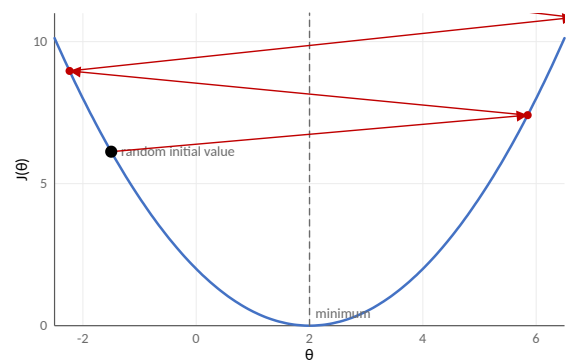
Linear Regression – Gradient Descent

Repeat until convergence (and **simultaneously** among all parameters, i.e. for $j \in \{0, \dots, n\}$):

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

- Initial values of θ_j ? Anything, e.g. random: J is convex, every start leads to the same minimum.
- α is the **learning rate**. What value?
- $\frac{\partial J(\theta)}{\partial \theta_j}$ is the gradient (direction of maximum slope). We step the opposite way: downhill.

Try $\alpha = 0.3, 1.0, 1.7$ and 2.1 on the right (here $J'' = 1$).



Learning rate α **2.10**

error factor $1 - \alpha$
-1.10

behaviour
Diverges

► $\alpha = 2.1$: each step is worse than the last, it diverges

Linear Regression – Gradient Descent

How do we select the correct α ? The same bowl in all three panels; only α changes.



- Too low** ($\alpha = 0.12$): safe but tiny steps, so it needs many updates.
- Just right** ($\alpha = 0.7$): a few big steps land next to the minimum.
- Too high** ($\alpha = 2.08$): every step jumps over the minimum and lands *higher* than before: it diverges.

Where do these numbers come from? On a bowl one step can be computed exactly. The next two slides do it and end with a rule for choosing α .

Linear Regression – Gradient Descent: one step on a bowl

Near its minimum θ^* a cost curve looks like a bowl (with one feature it is exactly a bowl, as for our three points). Let $a = J'' > 0$ be its curvature. One step of gradient descent can then be computed exactly:

$$J(\theta) = J^* + \frac{a}{2}(\theta - \theta^*)^2$$

the bowl: lowest point J^* at θ^* ; a larger a means a narrower bowl

$$J'(\theta) = a(\theta - \theta^*)$$

the 2 from differentiating the square cancels the $\frac{1}{2}$: the slope is proportional to the distance from the minimum

$$\theta_{\text{new}} = \theta - \alpha J'(\theta) = \theta - \alpha a(\theta - \theta^*)$$

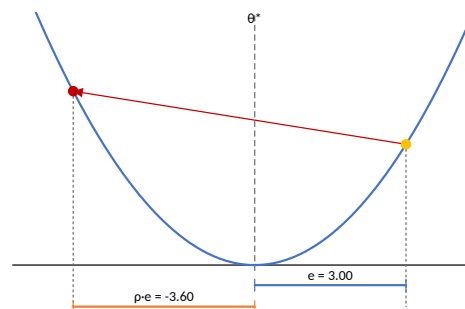
the gradient-descent update

$$\theta_{\text{new}} - \theta^* = (\theta - \theta^*) - \alpha a(\theta - \theta^*) = (1 - \alpha a)(\theta - \theta^*)$$

subtract θ^* from both sides, then take $(\theta - \theta^*)$ as a common factor

$$e_{\text{new}} = \rho e, \quad e = \theta - \theta^*, \quad \rho = 1 - \alpha a$$

One step multiplies the **distance to the minimum** by ρ . Move α on the right (here $a = 1$) and watch it.



Learning rate α

$\rho = 1 - \alpha$

-1.20

new distance ρe

-3.60

behaviour

Farther away: diverges

► $\alpha = 2.2$: $\rho = -1.20$, it jumps to the other side and ends farther away ($|\rho e| = 3.6$)

Linear Regression – Gradient Descent: which α converges?

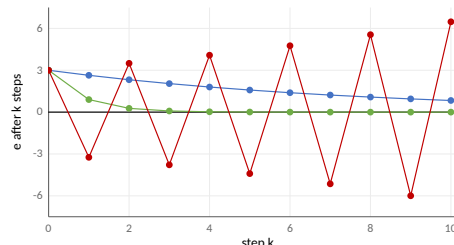
Every step multiplies the distance by the *same* ρ , so after k steps

$$e_k = \rho^k e_0, \quad \rho = 1 - \alpha a$$

$e_k \rightarrow 0$ exactly when $|\rho| < 1$, i.e. $-1 < 1 - \alpha a < 1$, i.e.

$0 < \alpha < 2/a$. The sign of ρ tells how it arrives (bowl of the previous slide, $a = 1$):

ρ	what happens	example
$0 < \rho < 1$	creeps to θ^* from one side; near 1 is slow	$\alpha = 0.12 \rightarrow 0.88$, $\alpha = 0.7 \rightarrow 0.30$
$\rho = 0$	lands on θ^* in one step	$\alpha = 1$
$-1 < \rho < 0$	jumps over θ^* , each time closer	$\alpha = 1.5 \rightarrow -0.50$
$\rho < -1$	jumps over θ^* , each time farther: diverges	$\alpha = 2.08 \rightarrow -1.08$



— $\alpha = 0.12, \rho = 0.88$: slow — $\alpha = 0.7, \rho = 0.30$: fast

— $\alpha = 2.08, \rho = -1.08$: diverges

The three runs of the previous slides, starting at $e_0 = 3$: $e_k = 3\rho^k$.

Our three points: $a = J'' = \frac{14}{3}$, so $\rho = 1 - 4.67\alpha$. $\alpha = 0.1$ gives $\rho = 0.53$ (converges); the fastest is $\alpha = 1/a = 0.21$; the limit is $2/a = 0.43$; $\alpha = 0.5$ gives $\rho = -1.33$ (diverges).

With many parameters: $0 < \alpha < 2/\lambda_{\max}(\frac{1}{m}\mathbf{X}^T\mathbf{X})$, the largest curvature in any direction.

Linear Regression – Gradient Descent

The update $\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$ needs the partial derivative. Let's compute $\frac{\partial J(\theta)}{\partial \theta_j}$ step by step:

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta_j} &= \frac{\partial}{\partial \theta_j} \left(\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 \right) \\ &= \frac{1}{2m} \sum_{i=1}^m \frac{\partial}{\partial \theta_j} (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2\end{aligned}$$

the derivative of a sum is the sum of the derivatives

$$= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot \frac{\partial (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})}{\partial \theta_j}$$

chain rule: $(u^2)' = 2u u'$, and the 2 cancels the $\frac{1}{2}$

$$= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

$h_{\theta} = \sum_k \theta_k x_k$, so $\partial h_{\theta} / \partial \theta_j = x_j$ (and $y^{(i)}$ does not depend on θ)

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

"batch" gradient descent.

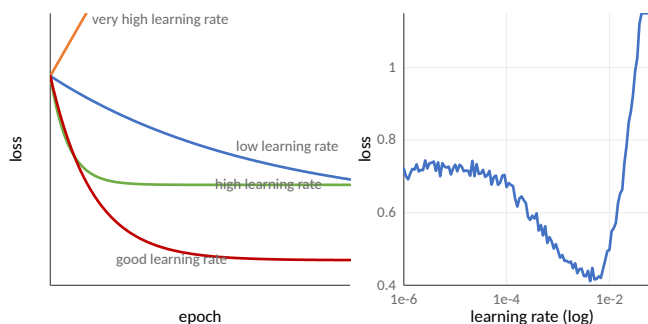
(the m training samples in each step).

Each example pushes θ_j by (its error) $\times$ (its input). Examples already well predicted barely push. With one feature and no intercept it is the $J'(\theta)$ of the simple model.

Linear Regression – Gradient Descent

How do we make sure G.D. is working?

- $J(\theta)$ decreases for every step. If not, α is too large or there is a bug.
- If it decreases too slowly, α is too small.



Try and select a consistent α . Examples: 0.001, 0.003, 0.01, 0.03 (right: a learning-rate sweep. Pick a value where the loss is still falling fast, about 10 $\times$ smaller than where it blows up).

Linear Regression – Normal Equation.

Matrix notation:

$$\mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{pmatrix}_{m \times 1}, \quad \boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{pmatrix}_{(n+1) \times 1}, \quad \mathbf{X} = \begin{pmatrix} -(\mathbf{x}^{(1)})^\top - \\ -(\mathbf{x}^{(2)})^\top - \\ \vdots \\ -(\mathbf{x}^{(m)})^\top - \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_n^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & \cdots & x_n^{(m)} \end{pmatrix}_{m \times (n+1)}$$

Taking this into account:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \boldsymbol{\theta} \\ \vdots \\ (\mathbf{x}^{(m)})^\top \boldsymbol{\theta} \end{pmatrix}_{m \times 1}, \quad \mathbf{E}_{m \times 1} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\boldsymbol{\theta}$$

And finally:

$$J(\boldsymbol{\theta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = \mathbf{y}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\boldsymbol{\theta} - \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\theta}$$

We drop the $\frac{1}{2m}$ here: multiplying J by a positive constant does not move its minimum.

We can optimize by equating $\nabla_{\boldsymbol{\theta}} J = \mathbf{0}$ $((n+1) \times 1)$. **It turns out that there is a known closed-form solution!**



Universidad
de Navarra

15

Linear Regression – Normal Equation.

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} (\mathbf{y}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\boldsymbol{\theta} - \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\theta})$$

$$= 0 - \mathbf{X}^\top \mathbf{y} - \mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta}$$

So we want to solve $\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y}$ (the "normal equations").

$$\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

- Unique only if $\mathbf{X}^\top \mathbf{X}$ is invertible (can be replaced with the pseudo-inverse).
- Slow if $n > 10\,000$ (use gradient descent).
- **Check with the simple model** (one feature, no intercept): $\mathbf{X}$ is the column of x 's, so $\mathbf{X}^\top \mathbf{X} = \sum x^2$ and $\mathbf{X}^\top \mathbf{y} = \sum xy$: $\boldsymbol{\theta} = \frac{\sum xy}{\sum x^2}$ ✓

Geometrically: $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = 0$ says the error is perpendicular to every column of $\mathbf{X}$. $\hat{\mathbf{y}}$ is the projection of $\mathbf{y}$ onto the columns.

HINT 1

$$\nabla_{\boldsymbol{\theta}} (\mathbf{a}^\top \boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} (a_0 \theta_0 + \cdots + a_n \theta_n) = \mathbf{a}$$

$$\nabla_{\boldsymbol{\theta}} (\boldsymbol{\theta}^\top \mathbf{a}) = \mathbf{a} \quad (\text{same number})$$

here $\mathbf{a} = \mathbf{X}^\top \mathbf{y}$, because $\mathbf{y}^\top \mathbf{X}\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{y})^\top \boldsymbol{\theta}$

HINT 2

$$\nabla_{\boldsymbol{\theta}} (\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\theta}) = 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta}$$

To see why, write the product as a double sum:

$$\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} = \sum_{i=0}^n \sum_{j=0}^n [\mathbf{X}^\top \mathbf{X}]_{ij} \theta_i \theta_j$$

Every pair (i, j) adds $[\mathbf{X}^\top \mathbf{X}]_{ij} \theta_i \theta_j$. With 2 parameters: $p\theta_1^2 + 2q\theta_1\theta_2 + r\theta_2^2$ (p, q, r : entries of $\mathbf{X}^\top \mathbf{X}$).

Differentiate with respect to one θ_k : it appears in the terms with $i = k$ and with $j = k$:

$$\frac{\partial}{\partial \theta_k} = \sum_j [\mathbf{X}^\top \mathbf{X}]_{kj} \theta_j + \sum_i [\mathbf{X}^\top \mathbf{X}]_{ik} \theta_i = 2[\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta}]_k$$

using that $\mathbf{X}^\top \mathbf{X}$ is symmetric, the two sums are equal (hence the 2). True for every k : the gradient is $2\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta}$.



Universidad
de Navarra

16

Linear Regression – Normal Equation.

A critical point (where $\nabla_{\theta} J = 0$) is not automatically a minimum: this requires a second-order check. The Hessian (the matrix of second derivatives, the multi-parameter J'') is

$$\nabla_{\theta}^2 J(\theta) = 2\mathbf{X}^T \mathbf{X}.$$

Positive semi-definiteness holds for any $\mathbf{X}$. For an arbitrary vector $\mathbf{v}$:

$$\mathbf{v}^T (\mathbf{X}^T \mathbf{X}) \mathbf{v} = (\mathbf{X} \mathbf{v})^T (\mathbf{X} \mathbf{v}) = \|\mathbf{X} \mathbf{v}\|^2 \geq 0$$

Since the Hessian is positive semi-definite everywhere, $J(\theta)$ is a **convex** function of θ : the matrix version of $J'' = \frac{1}{m} \sum x^2 \geq 0$. For a convex function, every critical point is automatically a **global minimum**: there are no saddle points or local maxima to worry about.

If $\mathbf{X}^T \mathbf{X}$ is invertible, it is positive semi-definite with no zero eigenvalues, hence **positive definite**: $\|\mathbf{X} \mathbf{v}\|^2 > 0$ for all $\mathbf{v} \neq 0$. This makes $J(\theta)$ **strictly convex**, so the minimum is not merely global but **unique**, consistent with the uniqueness argument above.

Linear Regression – Newton method.

Newton's method for optimization updates a guess θ_{old} via:

$$\theta_{\text{new}} = \theta_{\text{old}} - [\nabla_{\theta}^2 J(\theta_{\text{old}})]^{-1} \nabla_{\theta} J(\theta_{\text{old}})$$

Plug in the results from the previous slides:

$$\begin{aligned} \nabla_{\theta} J(\theta_{\text{old}}) &= -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \theta_{\text{old}} \\ \nabla_{\theta}^2 J(\theta_{\text{old}}) &= 2\mathbf{X}^T \mathbf{X} \text{ (constant, does not depend on } \theta) \end{aligned}$$

Substituting, starting from any θ_{old} :

$$\begin{aligned} \theta_{\text{new}} &= \theta_{\text{old}} - (2\mathbf{X}^T \mathbf{X})^{-1} (-2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \theta_{\text{old}}) \\ &= \theta_{\text{old}} - \underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}}_{= \mathbf{I}} \theta_{\text{old}} + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \\ &= \theta_{\text{old}} - \theta_{\text{old}} + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \end{aligned}$$

The θ_{old} terms cancel exactly, regardless of the starting point.

Interpretation. Because $J(\theta)$ is exactly quadratic, its Hessian $2\mathbf{X}^T \mathbf{X}$ is constant. Newton's method, which normally needs many iterations, lands **exactly** on the global minimum in a **single step**, from any starting point.

The normal equation is precisely this one Newton step. Gradient descent takes many small steps to the same point; Newton reaches it at once, at the cost of computing and inverting $\mathbf{X}^T \mathbf{X}$. That is why it is impractical for large n (slow if $n > 10\,000$).

LAB 1

Ordinary least squares from scratch

Linear regression built by hand with NumPy: every formula of this chapter becomes a few lines of code.

- 1 The MSE cost function and its gradient.
- 2 Batch gradient descent, and the cost against the number of iterations.
- 3 The normal equation (closed form), compared with gradient descent.
- 4 Vectorisation: the same computation without Python loops.
- 5 Different starting points reach the same minimum (convexity), and how to choose the learning rate.

[C0 Open in Colab](#)

notebook Lab 1 - Ordinary Least Squares Regression.ipynb

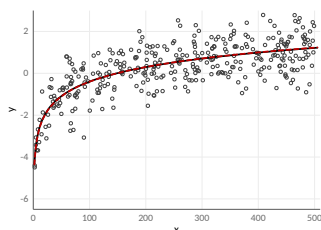
you need this chapter: the cost, gradient descent and α , the normal equation; NumPy from Lab 0

runs on CPU; every cell runs in seconds

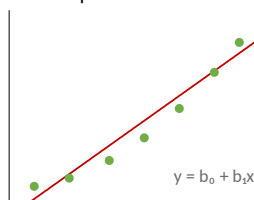
Next: logistic regression (chapter 03), then Labs 3 and 2.

Linear Regression – Feature engineering

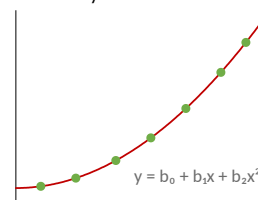
- Make new features that represent important qualities. For example:
 - x_1 is the length of a property.
 - x_2 is the width of a property.
 - Make the new feature $x_3 = x_1 \cdot x_2$ (the area): just a new column of your dataset.
- Polynomial regression. Make new features x_1^p for any p .
- Logarithmic regression: the feature $\log x$.



Simple linear model



Polynomial model

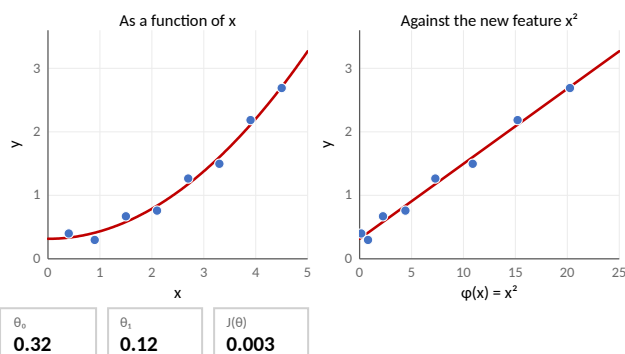


Still **linear regression**: linear in the parameters θ , whatever the features are.

- Feature scaling for gradient descent.
- Regularization.

Linear Regression – A new feature is just a new column

Feature $x_1 =$ x x^2 x^3



► Column $x_1 = x^2$: same least squares, J drops to 0.003 ($\theta_1 = 0.12$) and against x^2 the points lie on a line

Linear regression is linear in the **parameters** θ , not in x . The model never changes, only the column we feed it:

$$h(x) = \theta_0 \cdot 1 + \theta_1 \cdot x_1, \quad x_1 = \text{any function of } x$$

x_0	$x_1 = x^2$	y
1	0.16	0.40
1	0.81	0.30
1	2.25	0.67
1	4.41	0.76
⋮		

With $x_1 = x^2$ a curve in x is a **straight line** in x_1 . Same cost J , same gradient descent, same normal equation

$$\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

Several features at once, $x_1 = x$ and $x_2 = x^2$:

$h = \theta_0 + \theta_1 x_1 + \theta_2 x_2$ is a **plane** in the space of features; seen from x it is a parabola.

Linear Regression – Feature encoding

Data is not always numerical. You must encode the categorical data to numbers before fitting your model.

- Ordinal Variables (the categories have an order).
 - Ordinal Encoding: replace the names by numbers $0, 1, \dots, C-1$ (or $1, \dots, C$) **in that order**.
 - Bad $\rightarrow 0$, Good $\rightarrow 1$, Excellent $\rightarrow 2$
 - In scikit-learn, give the order explicitly: the `categories` argument, a list with the rank order of all expected labels.
- Nominal Variables (no ordered relation).
 - One-Hot Encoding – 1 bit per category (new "features")
 - Cow $\rightarrow [1, 0, 0]$, Dog $\rightarrow [0, 1, 0]$, Cat $\rightarrow [0, 0, 1]$
 - Dummy Encoding – C categories with $C-1$ binary variables.
 - Cow $\rightarrow [1, 0]$, Dog $\rightarrow [0, 1]$, Cat $\rightarrow [0, 0]$
 - Same data is represented.
 - For algorithms with intercept term (such as linear regression).
 - One-Hot Encoding makes $\mathbf{X}^T \mathbf{X}$ non-invertible (linear dependence).

Categorical	
Nominal	Ordinal
pen · pencil · eraser	excellent · good · bad
cow · dog · cat	fantastic · okay · don't like

Why the trap? Every row has exactly one 1, so $d_1 + \dots + d_C = 1 = x_0$. One column is a combination of others $\rightarrow \mathbf{X}^T \mathbf{X}$ is singular. Dropping one level (the reference) fixes it.

Linear Regression – Which encoder when?

The feature is...	Example	Use
ordered , with roughly equal steps	size S < M < L < XL; rating bad < ok < good; damage low < medium < high	ordinal : 0, 1, 2, 3 in that order
ordered, but the steps are very unequal	"0 / 1-2 / 3-5 / 6+ previous claims"	one-hot, or meaningful numbers (0, 1.5, 4, 7)
unordered , a few levels	colour, fuel type, city	one-hot (drop one level if the model has an intercept)
unordered, many levels	postcode, car model, workshop (1 000+)	group the rare levels; frequency or target encoding
any categorical, tree models	(chapter 06)	ordinal codes are fine: a tree only asks "code ≤ 2?"

What ordinal encoding tells a linear model. Size coded 0, 1, 2, 3 and price = $\theta_0 + \theta_1 \cdot \text{size}$: every step up adds the **same** θ_1 , from S to M as from L to XL.

One-hot gives every level its own coefficient: more flexible, but 3 parameters instead of 1. If the steps really are equal, ordinal is simpler and uses the data better.

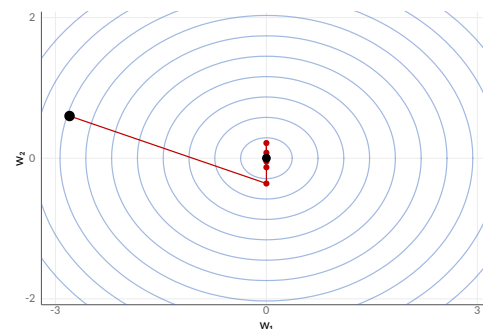
Pitfalls. `OrdinalEncoder` sorts the levels **alphabetically** (L, M, S, XL) unless you pass `categories=` `[["S", "M", "L", "XL"]]`. Never ordinal-encode an unordered feature for a linear model: "city = 3" is not three times "city = 1". Fit the encoders on the training data only, and set `handle_unknown` for levels never seen in training.

Linear Regression – Feature scaling

- Why? Gradient descent converges faster, and penalties (regularization) treat all the features alike.
- Normalization (min-max): $X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \in [0, 1]$
- Standardization: $X' = \frac{X - \mu}{\sigma}$, mean 0 and standard deviation 1, no fixed range.

When to use which?

- Standardization**: the default for linear and logistic regression, SVMs, neural networks and any model with a penalty.
- Min-max**: when a fixed range is needed (e.g. pixel intensities to [0, 1]). One outlier squeezes all the other values into a tiny range.
- Outliers**: standardization suffers less; `RobustScaler` (median and interquartile range) the least. Trees need no scaling at all.
- Compute min, max, μ , σ on the **training data only**.



Without scaling	After scaling
condition number 1.6	steps to converge 6

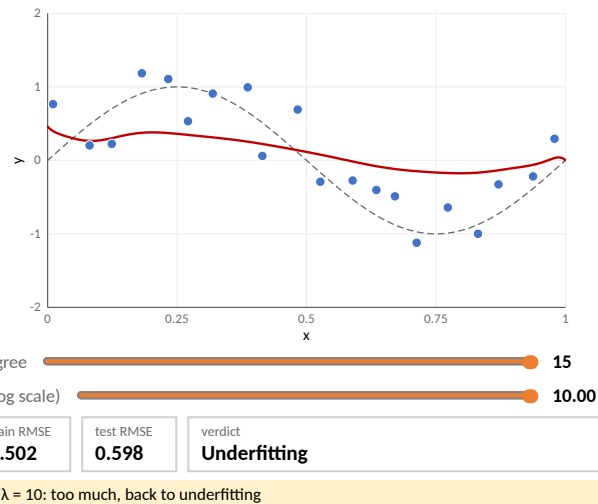
After scaling, gradient-descent steps are updated at the same rate for all the features.

► After scaling: round contours, a straight path, a handful of steps

Linear Regression – Regularization

- Polynomial regression of degree $m - 1$ through m points $\rightarrow J(\theta) = 0$.
- Good model for unseen samples?
- Underfitting**: too simple, high error everywhere.
- Just right**: follows the pattern, not the noise.
- Overfitting**: memorises the noise, fails on new samples.

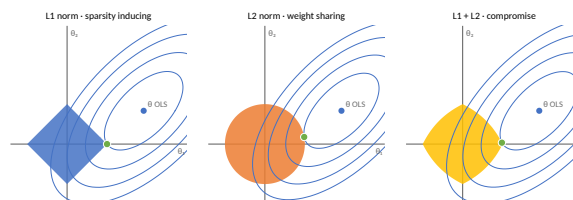
Move the **degree** to see underfitting, "just right" and overfitting. Then, at degree 15, raise λ : the impact of regularization.



Linear Regression – Regularization

Add a penalization in the cost function to the weights!

- L2 (Ridge regression)**: $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$ (not $j = 0$)
 - Shrinks the coefficients.
- L1 (Lasso regression)**: $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j|$
 - Forces some of the coefficients to be zero and results in a simpler model (feature selection).
 - Not differentiable at 0.
- Elastic Net**: the combination of the L1 and L2 regularization.



Chapter 05 goes through regularization slowly: where the formulas come from, why L1 gives exact zeros, and how to pick λ .

Linear Regression – Metrics

Metrics measure the performance of a model on validation and test data. Unlike losses, they don't need to be differentiable.

- Mean Square Error (MSE): $\frac{1}{m} \sum_i (y_i - \hat{y}_i)^2$
 - Squares the errors: big errors count much more, so it is sensitive to outliers.
 - In squared units (€²): hard to read.
- Mean Absolute Error (MAE): $\frac{1}{m} \sum_i |y_i - \hat{y}_i|$
 - Every unit of error counts the same: more robust to outliers.
 - Same units as y : "on average we are 12 k€ off".
 - Not differentiable at 0, but it can also be used as a loss.
- Root Mean Square Error (RMSE): $\sqrt{\frac{1}{m} \sum_i (y_i - \hat{y}_i)^2}$
 - Ranks models exactly like MSE, but in the units of y .
 - Always $\geq$ MAE; much larger when a few errors are big.

Linear Regression – Metrics, a worked example

Five houses, prices in k€:

house	1	2	3	4	5
actual y	200	150	300	250	100
predicted $\hat{y}$	210	140	280	260	110
error $y - \hat{y}$	-10	+10	+20	-10	-10

$$\text{MAE} = \frac{10+10+20+10+10}{5} = 12$$

$$\text{MSE} = \frac{100+100+400+100+100}{5} = 160, \quad \text{RMSE} = \sqrt{160} = 12.6$$

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2} = 1 - \frac{800}{25\,000} = 0.968$$

$\bar{y} = 200$:

$$\sum (y - \bar{y})^2 = 0 + 2\,500 + 10\,000 + 2\,500 + 10\,000 = 25\,000$$

Baseline "always predict 200 (the mean)": $\text{RMSE} = \sqrt{25\,000/5} = 70.7$ and $R^2 = 0$. The model is far better than the baseline.



Five good predictions

One error of 100

MAE (k€)

30.0

RMSE (k€)

46.3

R^2

0.572

MAPE

24.5 %

— actual price — predicted

► House 5 predicted at 200 instead of 100: MAE $\times 2.5$ (30), RMSE $\times 3.7$ (46.3).
One big error dominates RMSE

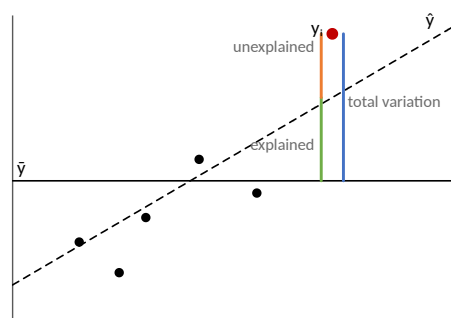
Linear Regression – Metrics

"How much (what %) of the total variation in Y (target) is explained by the variation in X (regression line)"

- Coefficient of determination $R^2 = 1 - \frac{\sum_i^m (y_i - \hat{y}_i)^2}{\sum_i^m (y_i - \bar{y})^2}$
 - 1: perfect predictions.
 - 0: no better than always predicting the mean $\bar{y}$.
 - On new data it can be **negative**: worse than the mean.
 - On the training data it never goes down when a feature is added, even pure noise: it cannot choose the features.

Best predictor before I knew X vs knowing it
(weight of a person vs weight after knowing their height).

- Adjusted $R_a^2 = 1 - \left(\frac{m-1}{m-n-1} \right) (1 - R^2)$, where m is the number of samples and n the number of features.
 - Only increases if adding the feature really helps.



Linear Regression – Which metric when?

Metric	Read it as	Use it when	Careful
MAE	the typical error, in units of y ("12 k€")	every unit of error costs the same; outliers in y	—
RMSE	like MAE, but big errors count more	a big mistake is much worse than several small ones	one outlier can dominate it
MAPE	the typical error in %: $\frac{1}{m} \sum_i \left \frac{y_i - \hat{y}_i}{y_i} \right $	relative errors matter (sales of big and small shops)	explodes when some y are near 0
R^2	the fraction of the variance explained	comparing across problems: it has no units	says nothing about the size of the errors in €

Always compare with a baseline: predict the mean (or the median) for everyone. R^2 does it for you: 0 means "as good as the mean", negative means worse.

Report on validation or test data, in the units the business understands. In the worked example, MAPE went from 6.5 % to 24.5 %: the big error was on the cheapest house.

Linear Regression – Probabilistic interpretation

What we want to show. Why do we minimize the *sum of squared errors* and not, say, the absolute errors? Assume the targets are the line plus random noise. The θ that makes the observed data *most probable* (maximum likelihood) turns out to be the one that minimizes the squared error.

Assume target variables and inputs are related via:

$$y^{(i)} = \theta^\top \mathbf{x}^{(i)} + e^{(i)}, \quad e^{(i)} \sim \mathcal{N}(0, \sigma^2) \text{ i.i.d.}$$

$$p(e^{(i)}) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(e^{(i)})^2}{2\sigma^2}\right)$$

which implies that

$$p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y^{(i)} - \theta^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right)$$

Given $\mathbf{X}$ and θ , the probability of the data is $p(\mathbf{y} | \mathbf{X}; \theta)$. As a function of θ : $L(\theta) = p(\mathbf{y} | \mathbf{X}; \theta)$. Assuming i.i.d. samples:

$$L(\theta) = \prod_{i=1}^m p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = \prod_{i=1}^m \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y^{(i)} - \theta^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right)$$

To make the problem simpler we maximize the log of $L(\theta)$ (monotonically increasing function):

$$\begin{aligned} \ell(\theta) = \log L(\theta) &= \sum_{i=1}^m \log\left(\frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y^{(i)} - \theta^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right)\right) \\ &= m \log \frac{1}{\sigma\sqrt{2\pi}} - \frac{1}{2\sigma^2} \sum_{i=1}^m (y^{(i)} - \theta^\top \mathbf{x}^{(i)})^2 \end{aligned}$$

The first term does not depend on θ . Equivalent to minimizing:

$$\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

Maximum likelihood: choose θ to make the data as highly probable as possible, i.e., maximize $L(\theta)$. Least squares = MLE under Gaussian noise.

Machine Learning

Máster en Innovación Tecnológica (MINT)

Next: Logistic Regression

imanolgranada@gmail.com

Machine Learning

Máster en Innovación Tecnológica (MINT)

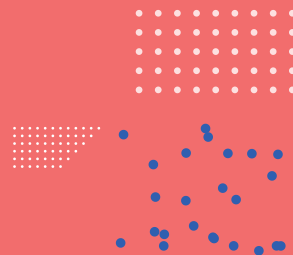
Logistic Regression

imanolgranada@gmail.com



1

**Logistic
Regression**



2

Logistic Regression - Motivation

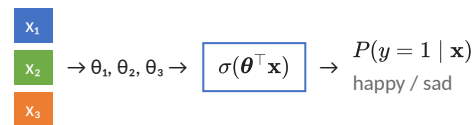
Outputs belong to classes:

- Malign or benign cancer
- Heart attack occurs
- Credit is accepted or not
- Email is spam or not
- Sentiment analysis.
- A military radio signal is from "the enemy"

Also non-binary examples:

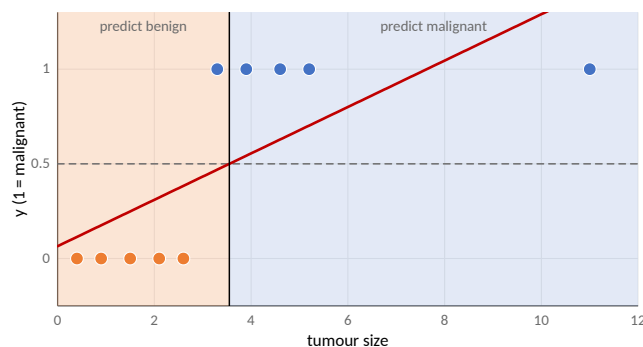
- Hand digit recognition
- (RF) Signal classification

Logistic Regression Model



A linear score, squashed into a probability.

Logistic Regression – Linear assumption



Original data

Add one extra data point

threshold at $x =$
3.55

misclassified
1 of 10

► Add one very large malignant tumour: the line tilts, the cut moves right, and the tumour at 3.3 is now called benign

Approximate y as a linear function of $\mathbf{x}$:

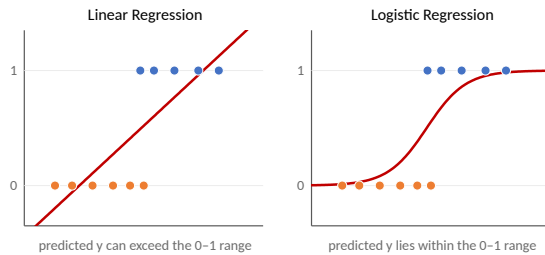
$$h(\mathbf{x}) = \sum_{j=0}^n \theta_j \cdot x_j = \boldsymbol{\theta}^\top \mathbf{x}$$

First perform linear regression and then establish a threshold, e.g. 0.5.

One extra data point leads to bad results. A clearly malignant tumour far to the right drags the line, and the 0.5 crossing moves into the malignant cases.

Also: $\boldsymbol{\theta}^\top \mathbf{x}$ is unbounded, so predictions like -0.3 or 1.4 are not probabilities.

Logistic Regression – Sigmoid function.



Approximate y as a **non-linear** function of $\mathbf{x}$:

$$h_{\theta}(\mathbf{x}) = g(\theta^{\top} \mathbf{x}) = \frac{1}{1 + e^{-\theta^{\top} \mathbf{x}}},$$

where

$$g(z) = \frac{1}{1 + e^{-z}}$$

- $g(z)$ is the **sigmoid** (logistic) function.

The sigmoid function:

- $0 \leq g(z) \leq 1$ is bounded.
- $g(z)$ can be interpreted as the probability that the class is 1.
 - $h_{\theta}(\mathbf{x}) = P(Y = 1 \mid \mathbf{x}; \theta)$
 - $P(Y = 0 \mid \mathbf{x}; \theta) = 1 - h_{\theta}(\mathbf{x})$
- Easy to establish a threshold (or at least it makes sense to establish one).

Logistic Regression – Sigmoid properties

Four facts we use later. Each one is a two-line proof.

1 • Range. $g(z) \in (0, 1)$, $g(0) = \frac{1}{2}$, $g(z) \rightarrow 1$ as $z \rightarrow \infty$, $g(z) \rightarrow 0$ as $z \rightarrow -\infty$.

2 • Symmetry. $1 - g(z) = g(-z)$:

$$1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}} = \frac{1}{e^z + 1} = g(-z)$$

3 • Derivative. $g'(z) = g(z)(1 - g(z))$:

$$g'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = g(z)(1 - g(z))$$

4 • Inverse (the logit). If $p = g(z)$ then $z = \log \frac{p}{1-p}$:

$$p(1 + e^{-z}) = 1 \Rightarrow e^{-z} = \frac{1-p}{p} \Rightarrow z = \log \frac{p}{1-p}$$

Logistic Regression – Odds: another way to say "how likely"

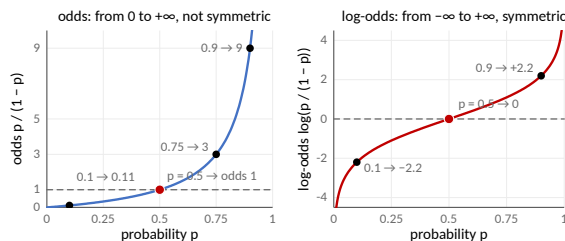
Let p be the probability that a claim is fraud. The **odds** of fraud are

$$\text{odds} = \frac{p}{1-p} = \frac{\text{probability it happens}}{\text{probability it does not}}$$

- $p = 0.75$: odds = $\frac{0.75}{0.25} = 3$. "Three to one": fraud is **3 times more likely than not**.
- $p = 0.5$: odds = 1. "Even": as likely as not.
- $p = 0.1$: odds = $\frac{0.1}{0.9} = 0.11$. Fraud is 9 times **less** likely than not.

Going back: $p = \frac{\text{odds}}{1 + \text{odds}}$. (odds 3 $\rightarrow p = \frac{3}{4}$.)

Odds live in $(0, \infty)$ and are lopsided: $p = 0.9$ gives 9 but $p = 0.1$ gives 0.11. Hard to add or compare. The logarithm fixes that.



Logistic Regression – Log-odds (the logit) and its properties

$$\text{logit}(p) = \log \frac{p}{1-p}$$

The logarithm of the odds: the **log-odds**. In logistic regression it is also called the **score**.

- 1 • **Range.** $p \in (0, 1) \rightarrow \text{odds} \in (0, \infty) \rightarrow \text{log-odds} \in (-\infty, +\infty)$: **any real number**. Exactly what a linear function $\theta^\top \mathbf{x}$ can produce.
- 2 • **Zero in the middle.** $\text{logit}(0.5) = \log 1 = 0$. Positive when $p > 0.5$, negative when $p < 0.5$.
- 3 • **Symmetric.** Swapping the classes only flips the sign: $\log \frac{1-p}{p} = -\log \frac{p}{1-p}$.
- 4 • **Products become sums.** Multiplying the odds by c **adds** $\log c$: $\log(c \cdot \text{odds}) = \log c + \log(\text{odds})$.
- 5 • **Its inverse is the sigmoid.** Solve $z = \log \frac{p}{1-p}$ for p :

$$e^z = \frac{p}{1-p} \Rightarrow e^z - pe^z = p \Rightarrow p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}} = g(z)$$

p	odds	log-odds
0.10	0.11	-2.20
0.25	0.33	-1.10
0.50	1	0
0.75	3	+1.10
0.90	9	+2.20

Three names, one number: **log-odds = logit = the score z** . Sigmoid turns a score into a probability; logit turns a probability back into a score.

Logistic Regression – The linear score is the log-odds

Apply the logit to $p = h_{\theta}(\mathbf{x}) = g(\theta^{\top} \mathbf{x})$ (property 5 undoes the sigmoid): the **log-odds are linear** in $\mathbf{x}$:

$$\log \frac{p}{1-p} = \theta^{\top} \mathbf{x} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

So logistic regression is a **linear regression of the log-odds**. That is where the name comes from.

Reading θ_j . Increasing x_j by one unit (others fixed) adds θ_j to the log-odds, i.e. **multiplies the odds** $\frac{p}{1-p}$ by e^{θ_j} (the odds ratio).

Example: $\theta_j = 0.7$ means the odds roughly double per unit of x_j ($e^{0.7} \approx 2$).

The effect on the probability itself is not constant: it is largest near $p = 0.5$.

Worked example. Fraud model with one feature, claim amount in k€: $\theta_0 = -3$, $\theta_1 = 0.7$.

amount x	score $z = \theta_0 + \theta_1 x$	odds e^z	$p = g(z)$
3 k€	-0.9	0.41	0.29
4 k€	-0.2	0.82	0.45
5 k€	+0.5	1.65	0.62
6 k€	+1.2	3.32	0.77

Each extra k€ adds 0.7 to the score and multiplies the odds by $e^{0.7} \approx 2$. The probability moves by a different amount each time: +0.16, +0.17, +0.15.

Logistic Regression – Cost function.

If we want to use the square error cost, the function $J(\theta)$ would no longer be convex. We define the **Binary Cross-Entropy** loss:

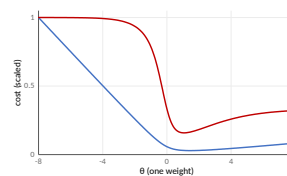
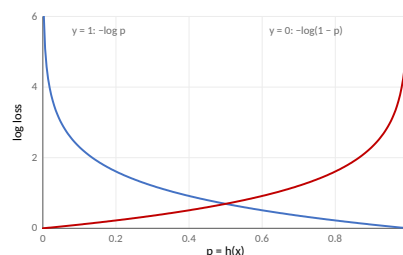
$$L(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(h_{\theta}(\mathbf{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - h_{\theta}(\mathbf{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

or in 1 line,

$$L = -y^{(i)} \log(h_{\theta}(\mathbf{x}^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{x}^{(i)}))$$

- If $y^{(i)} = 1$ we penalize very hard **low** probabilities.
- If $y^{(i)} = 0$ we penalize very hard **high** probabilities.

For a small change in p near the wrong end there is a large change in the loss: confident mistakes cost a fortune.



— cross-entropy: one smooth bowl

— MSE with a sigmoid: a flat plateau (tiny gradients)

Logistic Regression – Cost function.

$$L(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(h_{\theta}(\mathbf{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - h_{\theta}(\mathbf{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

or in 1 line, $L = -y^{(i)} \log(h_{\theta}(\mathbf{x}^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{x}^{(i)}))$.

And the cost function ends up being

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)}) = \frac{1}{m} \sum_{i=1}^m \left(-y^{(i)} \log(h_{\theta}(\mathbf{x}^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{x}^{(i)})) \right)$$

Logistic Regression – Probabilistic interpretation

What we want to show. As in linear regression (there, Gaussian noise gave least squares), maximum likelihood explains our cost. Here y is 0 or 1, so the noise is a coin flip with $P(y = 1) = h_{\theta}(\mathbf{x})$, and the same recipe gives back the cross-entropy cost.

Another fit by maximum likelihood. Assume:

$$\begin{aligned} P(y^{(i)} = 1 \mid \mathbf{x}^{(i)}; \theta) &= h_{\theta}(\mathbf{x}^{(i)}) \\ P(y^{(i)} = 0 \mid \mathbf{x}^{(i)}; \theta) &= 1 - h_{\theta}(\mathbf{x}^{(i)}) \end{aligned}$$

That can be compacted to (plug in $y = 1$ or $y = 0$ to check):

$$p(y^{(i)} \mid \mathbf{x}^{(i)}; \theta) = h_{\theta}(\mathbf{x}^{(i)})^{y^{(i)}} \cdot (1 - h_{\theta}(\mathbf{x}^{(i)}))^{1-y^{(i)}}$$

With m i.i.d. training samples, the likelihood of the θ 's is

$$L(\theta) = p(\mathbf{y} \mid \mathbf{X}; \theta) = \prod_{i=1}^m h_{\theta}(\mathbf{x}^{(i)})^{y^{(i)}} (1 - h_{\theta}(\mathbf{x}^{(i)}))^{1-y^{(i)}}$$

Once again, it is easier to maximize the log likelihood:

$$\begin{aligned} \ell(\theta) &= \log L(\theta) = \sum_{i=1}^m \log p(y^{(i)} \mid \mathbf{x}^{(i)}; \theta) \\ &= \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{x}^{(i)})) \right] \end{aligned}$$

And maximizing ℓ is equivalent to minimizing its negative:

$$J(\theta) = -\frac{1}{m} \ell(\theta)$$

Same recipe as linear regression: minimising the cost is maximum likelihood. Only the noise model changed, Bernoulli instead of Gaussian.

Logistic Regression – Optimization.

We minimize J with gradient descent. Shorthand for this slide: $h^{(i)} = h_{\theta}(\mathbf{x}^{(i)}) = g(z^{(i)})$ with $z^{(i)} = \theta^T \mathbf{x}^{(i)}$.

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta_j} &= \frac{\partial}{\partial \theta_j} \frac{1}{m} \sum_{i=1}^m \left(-y^{(i)} \log h^{(i)} - (1 - y^{(i)}) \log (1 - h^{(i)}) \right) \\ &= \frac{1}{m} \sum_{i=1}^m \left(-\frac{y^{(i)}}{h^{(i)}} + \frac{1 - y^{(i)}}{1 - h^{(i)}} \right) \cdot \frac{\partial h^{(i)}}{\partial \theta_j} \end{aligned}$$

$(\log u)' = u'/u$; the minus sign of $\log(1 - h)$ flips the second sign

$$= \frac{1}{m} \sum_{i=1}^m \frac{h^{(i)} - y^{(i)}}{h^{(i)}(1 - h^{(i)})} \cdot h^{(i)}(1 - h^{(i)}) \cdot \frac{\partial z^{(i)}}{\partial \theta_j}$$

put the two fractions over a common denominator; chain rule

$\partial h / \partial \theta_j = g'(z) \partial z / \partial \theta_j$ with $g' = g(1 - g)$

$$= \frac{1}{m} \sum_{i=1}^m (h^{(i)} - y^{(i)}) \cdot x_j^{(i)}$$

the blue factors cancel; $z = \sum_k \theta_k x_k$ so $\partial z / \partial \theta_j = x_j$

HINT

$$g'(z) = \frac{d}{dz} \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{(1 + e^{-z})^2} = g(z)(1 - g(z))$$

The blue factor $h(1 - h)$ cancels the denominator exactly. That cancellation is why cross-entropy and the sigmoid work so well together.

Same shape as linear regression: **(prediction - target) × input**.

Is J convex, so that gradient descent finds **the** minimum? Yes. Proof in the next two slides.

Logistic Regression – Convexity, step by step (1/2)

What we must show. J is convex if its **Hessian** $\mathbf{H}$ (the matrix of second derivatives, $H_{jk} = \partial^2 J / \partial \theta_j \partial \theta_k$) satisfies $\mathbf{v}^T \mathbf{H} \mathbf{v} \geq 0$ for every vector $\mathbf{v}$. Same test as for linear regression, where $\mathbf{H} = 2\mathbf{X}^T \mathbf{X}$.

Step 1 • the gradient (previous slide), one component:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h^{(i)} - y^{(i)}) x_j^{(i)}$$

Step 2 • differentiate again, now with respect to θ_k . Only $h^{(i)}$ depends on θ ; $y^{(i)}$ and $x_j^{(i)}$ are data:

$$H_{jk} = \frac{\partial^2 J}{\partial \theta_j \partial \theta_k} = \frac{1}{m} \sum_{i=1}^m \frac{\partial h^{(i)}}{\partial \theta_k} x_j^{(i)}$$

Step 3 • chain rule with $h = g(z)$ and $g' = g(1 - g)$:

$$\frac{\partial h^{(i)}}{\partial \theta_k} = g'(z^{(i)}) \frac{\partial z^{(i)}}{\partial \theta_k} = h^{(i)}(1 - h^{(i)}) x_k^{(i)}$$

Step 4 • put it together. Write $s_i = h^{(i)}(1 - h^{(i)})$:

$$H_{jk} = \frac{1}{m} \sum_{i=1}^m s_i x_j^{(i)} x_k^{(i)}$$

All the entries at once, as a matrix (compare with $\frac{1}{m} \sum_i \mathbf{x}^{(i)} \mathbf{x}^{(i)T}$ in linear regression):

$$\mathbf{H} = \frac{1}{m} \sum_{i=1}^m s_i \mathbf{x}^{(i)} \mathbf{x}^{(i)T} = \frac{1}{m} \mathbf{X}^T \mathbf{S} \mathbf{X}$$

with $\mathbf{S} = \text{diag}(s_1, \dots, s_m)$. Unlike linear regression, $\mathbf{H}$ depends on θ (through the s_i), so there is no closed form. But we only need its sign.

Logistic Regression – Convexity, step by step (2/2)

Step 5 • test with an arbitrary vector $\mathbf{v}$. Multiply on both sides:

$$\mathbf{v}^\top \mathbf{H} \mathbf{v} = \frac{1}{m} \sum_{i=1}^m s_i \mathbf{v}^\top \mathbf{x}^{(i)} \mathbf{x}^{(i)\top} \mathbf{v} = \frac{1}{m} \sum_{i=1}^m s_i (\mathbf{v}^\top \mathbf{x}^{(i)})^2$$

$\mathbf{v}^\top \mathbf{x}^{(i)}$ is a number, and $\mathbf{x}^{(i)\top} \mathbf{v}$ is the same number, so their product is a square.

Step 6 • every term is ≥ 0 .

- $s_i = h^{(i)}(1 - h^{(i)}) > 0$, because the sigmoid gives $0 < h^{(i)} < 1$ (sigmoid property 1).
- $(\mathbf{v}^\top \mathbf{x}^{(i)})^2 \geq 0$, because it is a square.
- $\frac{1}{m} > 0$. A sum of non-negative terms is non-negative.

$$\mathbf{v}^\top \mathbf{H} \mathbf{v} \geq 0 \text{ for every } \mathbf{v}$$

$\Rightarrow \mathbf{H}$ positive semi-definite $\Rightarrow J$ is convex ■

What we gain.

- One bowl: every local minimum is the global minimum.
- Gradient descent (small enough α) and Newton's method both reach it, from any starting point.
- **Strictly convex** ($\mathbf{v}^\top \mathbf{H} \mathbf{v} > 0$ for $\mathbf{v} \neq \mathbf{0}$) if the columns of $\mathbf{X}$ are independent: then the minimum is unique.

Why MSE fails here. With $J_{\text{MSE}} = \frac{1}{2m} \sum (h^{(i)} - y^{(i)})^2$ the second derivative contains $(h^{(i)} - y^{(i)}) g''(z^{(i)})$, which changes sign. Plateaus and local minima appear (the flat MSE curve on the cost-function slide).

One caveat: convex does not mean the minimum is reached. With separable data J keeps decreasing forever (next slides).

Logistic Regression – Gradient descent.

Repeat until convergence (and **simultaneously** among all parameters, i.e. for $j \in \{0, \dots, n\}$):

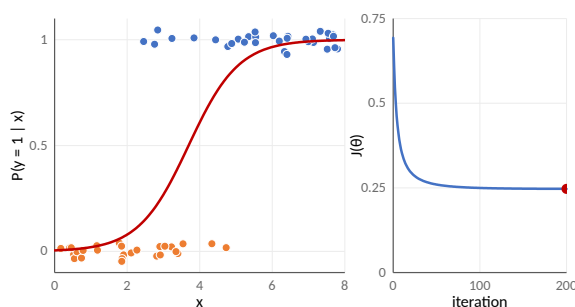
$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

Very similar to linear regression! (with its own non-linear hypothesis)

Vectorized:

$$\theta := \theta - \alpha \frac{1}{m} \mathbf{X}^\top (g(\mathbf{X}\theta) - \mathbf{y})$$

No closed form this time: the unknown is inside the sigmoid. We iterate (GD, or Newton / L-BFGS in scikit-learn).



Iteration 200

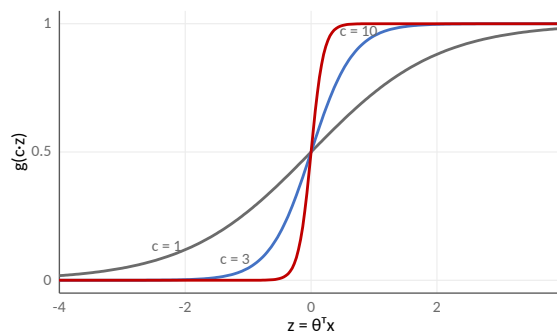
$J(\theta)$
0.247

► Iteration 200: J has flattened out

Logistic Regression – Separable data

If a line separates the classes perfectly, scaling $\theta \rightarrow c\theta$ with $c \rightarrow \infty$ pushes every $h_{\theta}(\mathbf{x}^{(i)})$ to the correct 0 or 1.

- $J \rightarrow 0$ but never reaches it: the maximum likelihood solution **does not exist**.
- Gradient descent keeps increasing $\|\theta\|$ forever: probabilities become over-confident.
- **Fix:** L2 regularization, $J + \frac{\lambda}{2m} \sum_{j \geq 1} \theta_j^2$. scikit-learn does this by default with $C = 1/\lambda = 1$ (chapter 05).



$g(cz)$ for $c = 1, 3, 10$: the same boundary, ever more confident.

Logistic Regression – A nice property of the optimum

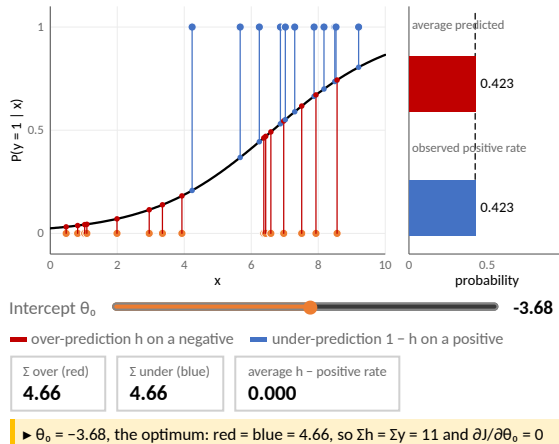
At the minimum every partial derivative of J is zero. Take the first one, for the intercept θ_0 , whose input is $x_0^{(i)} = 1$ (slide on the gradient):

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h^{(i)} - y^{(i)}) \cdot 1 = 0 \Rightarrow \sum_{i=1}^m h^{(i)} = \sum_{i=1}^m y^{(i)}$$

The predicted probabilities add up to the number of positives. Divide by m : the average prediction equals the observed positive rate. If 30 % of the training cases are positive, the model predicts 30 % on average.

In terms of errors, $\sum_{y=0} h = \sum_{y=1} (1 - h)$: the **over-predictions** on the negatives balance the **under-predictions** on the positives. For the plain fit the same holds for every feature: $\sum_i (h^{(i)} - y^{(i)}) x_j^{(i)} = 0$.

True on the **training data**, for the plain fit or a penalty that skips θ_0 (our $j \geq 1$). Class weights or resampling (SMOTE) break it on purpose: see calibration later.



Logistic Regression – Decision

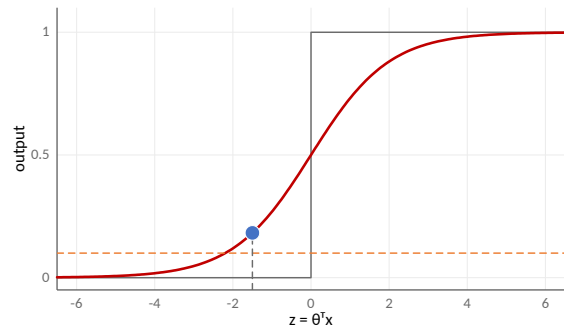
- Need to apply a certain threshold:

$$\hat{y} = \begin{cases} 1 & \text{if } h_{\theta}(\mathbf{x}^{(i)}) \geq \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

- A common threshold is 0.5 but it can vary.

It is application specific:

- Higher thresholds → fewer false positives (only confident positives).
 - Spam detection.
- Lower thresholds → fewer false negatives (more false positives).
 - Medical image screening (before the doctor).



Score z

-1.5

Threshold

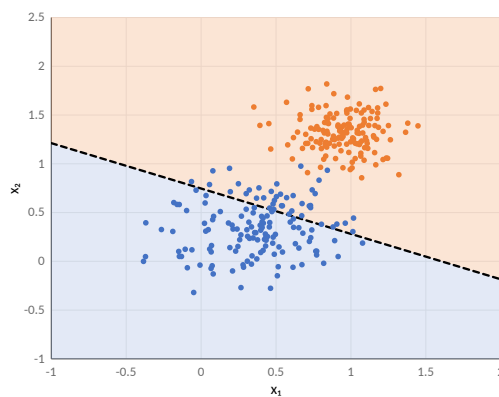
0.10

$h(\mathbf{x}) = g(z)$
0.182

prediction $\hat{y}$
1

► Lower it to 0.1: $g(-1.5) \approx 0.18$ is now class 1 (medical screening)

Logistic Regression – Decision boundary



Threshold

0.10

► Threshold 0.1: the line moves the other way, more points are positive (191 of 300)

- Line given by $h_{\theta}(\mathbf{x}) = \text{threshold}$.
- Separates the feature space in 2.

Since g is increasing and $g(0) = 0.5$:

$$h_{\theta}(\mathbf{x}) \geq 0.5 \iff \theta^T \mathbf{x} \geq 0. \text{ A linear classifier.}$$

Another threshold τ gives the parallel line

$$\theta^T \mathbf{x} = \log \frac{\tau}{1-\tau}. \text{ Move the slider.}$$

fitted
 $\theta = (-6.2, 2.5, 5.4)$

predicted positive
191

Logistic Regression – Metrics.

Confusion matrix:

		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) <i>Type II Error</i>	Sensitivity $\frac{TP}{TP+FN}$
	Negative	False Positive (FP) <i>Type I Error</i>	True Negative (TN)	Specificity $\frac{TN}{TN+FP}$
		Precision $\frac{TP}{TP+FP}$	Negative Predictive Value $\frac{TN}{TN+FN}$	Accuracy $\frac{TP+TN}{TP+TN+FP+FN}$

Columns: predicted class.

Accuracy:

- Correct predictions divided by the total.
- Bad for imbalanced datasets (majority vote).
- Every class is equally important.

Logistic Regression – Metrics.

Confusion matrix:

		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) <i>Type II Error</i>	Sensitivity $\frac{TP}{TP+FN}$
	Negative	False Positive (FP) <i>Type I Error</i>	True Negative (TN)	Specificity $\frac{TN}{TN+FP}$
		Precision $\frac{TP}{TP+FP}$	Negative Predictive Value $\frac{TN}{TN+FN}$	Accuracy $\frac{TP+TN}{TP+TN+FP+FN}$

Columns: predicted class.

Precision:

- Ratio of true positives to all predicted positives.
- Higher precision means that an algorithm returns more relevant results than irrelevant ones.
- Example: potential clients. We do not want to waste money on ads.

Logistic Regression – Metrics.

Confusion matrix:

		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{TP+FN}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{TN+FP}$
		Precision $\frac{TP}{TP+FP}$	Negative Predictive Value $\frac{TN}{TN+FN}$	Accuracy $\frac{TP+TN}{TP+TN+FP+FN}$

Columns: predicted class.

Sensitivity or Recall:

- True positives to all the actual positives.
- High recall: few real positives are missed.
- Example: cancer patients, we do not want false negatives.

Logistic Regression – All the metrics

Metric	Formula	Question it answers
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	What fraction of all predictions is right?
Precision	$\frac{TP}{TP+FP}$	Of the ones I flagged, how many are really positive?
Recall · sensitivity · TPR	$\frac{TP}{TP+FN}$	Of the real positives, how many did I catch?
Specificity · TNR	$\frac{TN}{TN+FP}$	Of the real negatives, how many did I leave alone?
False positive rate	$\frac{FP}{FP+TN} = 1 - \text{specificity}$	Of the real negatives, how many did I flag?
F_1	$\frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}} = \frac{2TP}{2TP+FP+FN}$	Harmonic mean: high only if both are high.
F_β	$\frac{(1+\beta^2) \cdot \text{Prec} \cdot \text{Rec}}{\beta^2 \cdot \text{Prec} + \text{Rec}}$	Recall counts β times as much as precision.

Harmonic mean: precision 1.0 with recall 0.01 gives $F_1 \approx 0.02$, not 0.5.

Metrics – Worked example: 1,000 insurance claims

50 claims are fraudulent (5 %). The detector flags 60 claims, 40 of which are really fraud.

	flagged no	flagged yes
legit · 950	TN = 930	FP = 20
fraud · 50	FN = 10	TP = 40

Accuracy = $\frac{40+930}{1000} = \mathbf{0.97}$. Looks great.

Precision = $\frac{40}{60} = \mathbf{0.67}$: a third of the investigated claims are false alarms.

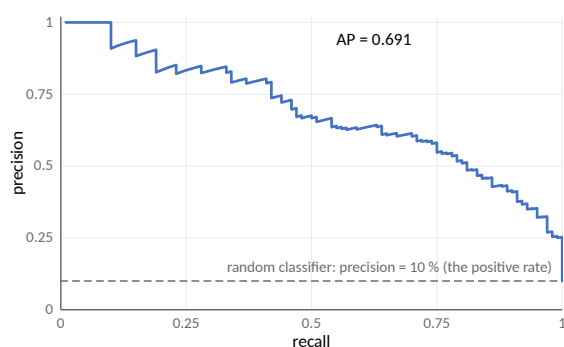
Recall = $\frac{40}{50} = \mathbf{0.80}$: one fraud in five gets through.

$F_1 = \frac{2 \cdot 40}{80+20+10} = \mathbf{0.73}$.

The trivial model "nothing is fraud" has accuracy **0.95**, recall **0** and undefined precision. With imbalanced classes, accuracy mostly reflects the majority class.

Logistic Regression – Prec. vs Recall tradeoff.

By varying the threshold, you can either improve precision or sensitivity/recall – but not both.

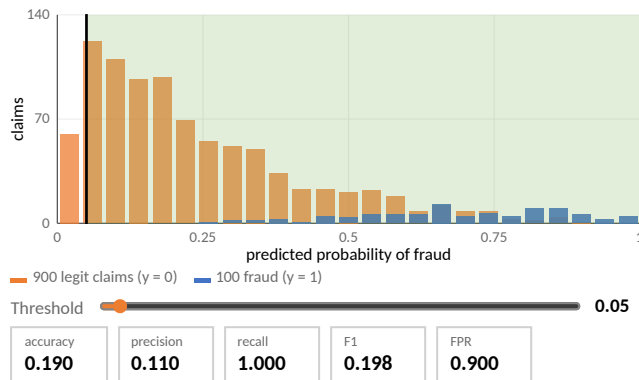


So one of the two alone can be gamed: flag everything and recall is 1; flag only the surest case and precision is 1. Report both, or combine them:

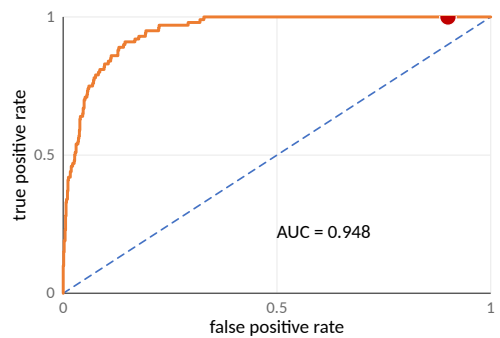
$$F_1 = \frac{2}{\frac{1}{\text{Recall}} + \frac{1}{\text{Precision}}}$$

- Widely used, "good" for comparing.
 - Good balance even on [imbalanced classification problems](#).
 - A low F_1 doesn't say which cases it failed (poor information if alone).
 - The **PR curve** baseline is the positive rate, not a diagonal.
- Summarised by **average precision (AP)**.

Metrics – Move the threshold

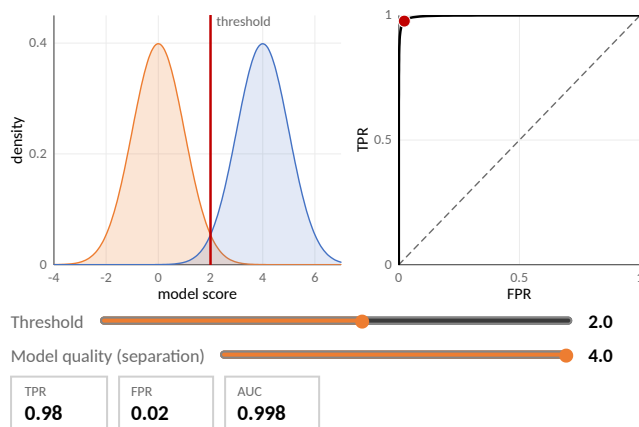


	pred 0	pred 1
$y = 0$	TN 90	FP 810
$y = 1$	FN 0	TP 100



► 0.05: almost every claim is flagged, FPR close to 1

Logistic Regression – ROC.



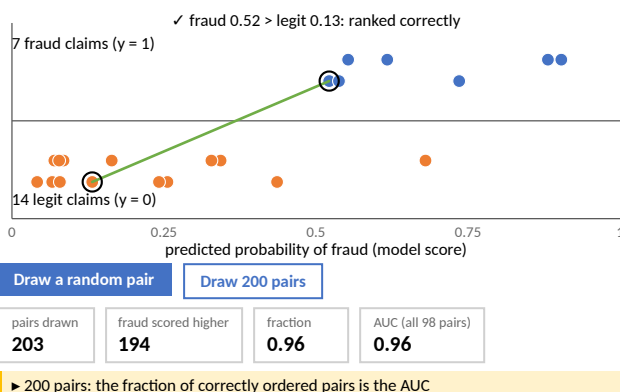
► Strong separation: the curve hugs the top-left corner, AUC $\rightarrow$ 1

Receiver operating characteristic curve: TPR against FPR, computed for many different thresholds on a single graph.

- The diagonal is a model with random scores, whatever the class balance.
- Good for comparing different classifiers: set separation to 0 and the curve collapses to the diagonal (AUC $\approx$ 0.5).
- With very rare positives it can look too optimistic (FPR is divided by a huge number of negatives): look at the PR curve too.
- Good to answer: what TPR can I obtain if I want to fix FPR to 0.02?

1 metric $\rightarrow$ **AUC-ROC**, the area under the curve, between 0.5 (random) and 1 (perfect).

Metrics – What AUC means, in one sentence



Take one fraud claim and one legit claim at random. AUC is the probability that the model gives the fraud the higher score.

- AUC = 1: every fraud scores above every legit claim.
- AUC = 0.5: a coin toss. The scores carry no information.
- AUC = 0.8: "8 times out of 10 the fraud comes out on top".

No threshold is involved: AUC grades the **ordering** of the scores. That is why it is good for comparing models, and why it says nothing about what happens at the one threshold you will actually use.

Metrics – Choosing the threshold, simply

The model gives every claim a score. The threshold is **where you cut**. Three ways to decide, all on the **validation set**:

1 • I have a budget

"We can investigate 60 claims a week." Sort the claims by score and flag the top 60. The threshold is simply the 60th score.

The threshold follows the budget, not the other way round.

2 • I have a requirement

"We must catch 90 % of the fraud." Lower the threshold until recall reaches 0.9, then stop.

"No more than 2 % of legit claims may be bothered." Raise it until FPR is 0.02.

Read the point off the PR or ROC curve.

3 • I know the costs

A missed fraud costs 900 €, a useless investigation 100 €. Flag whenever the expected cost of flagging is lower. Derived on the next slide:

$$\tau^* = \frac{100}{100 + 900} = 0.1$$

0.5 is only the right cut when both mistakes cost the same and the classes are balanced. With rare positives and expensive misses, the right cut is usually **far below 0.5**. Never tune it on the test set.

Metrics – The threshold from costs, slowly

One claim. The model says: probability of fraud = p . Two possible actions.

Flag it. With probability $1 - p$ it was legit and the investigation was wasted:

$$\mathbb{E}[\text{cost of flagging}] = (1 - p) c_{FP}$$

Let it through. With probability p it was fraud and we pay for it:

$$\mathbb{E}[\text{cost of not flagging}] = p c_{FN}$$

Flag when flagging is the cheaper option:

$$(1 - p) c_{FP} < p c_{FN}$$

Expand and collect the p terms on one side:

$$c_{FP} - p c_{FP} < p c_{FN} \Rightarrow c_{FP} < p (c_{FP} + c_{FN})$$

$$p > \frac{c_{FP}}{c_{FP} + c_{FN}} = \tau^*$$

c_{FP}	c_{FN}	τ^*	reading
100	100	0.50	equal costs: the usual 0.5
100	400	0.20	misses hurt 4x: cut lower
100	900	0.10	misses hurt 9x: flag at 10 %
400	100	0.80	false alarms hurt: cut higher

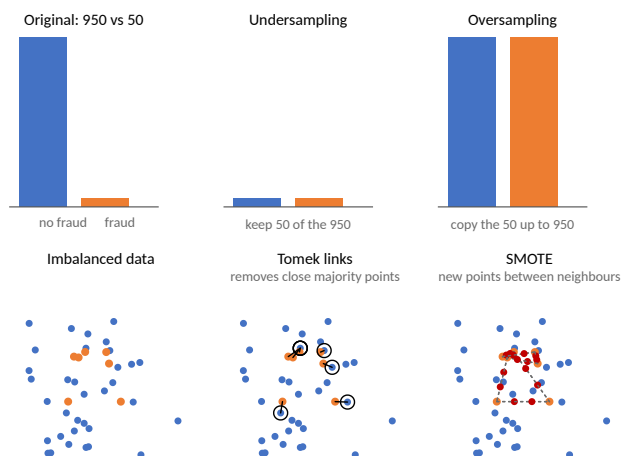
This needs p to be a real probability (calibrated): see the calibration slide. Only the **ratio** of the costs matters.

Class imbalances.

Appear in many domains, such as fraud detection, spam filtering, ... Problem! High accuracy just by predicting the **majority class**.

Solutions:

- Resampling techniques:
 - Undersampling: reduces the amount of data.
 - Oversampling: can cause overfitting (if exact copies).
- Improvements: Tomek links and SMOTE.
- Class weights in the loss (`class_weight="balanced"`).
- Use precision, recall, F1, PR curves instead of accuracy.
- Resample only the training set.** Validation and test keep the real class proportions.

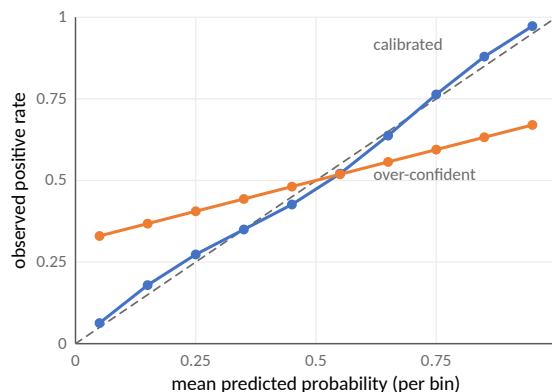


Metrics – Probabilities and calibration

Often we use the probability itself (pricing, risk scores, expected cost).

- **Log-loss** ($= J$ on the test set) and **Brier score** $\frac{1}{m} \sum_i (p^{(i)} - y^{(i)})^2$: lower is better.
- **Calibration**: among claims with predicted $p \approx 0.3$, about 30 % should be fraud. The reliability diagram should follow the diagonal.
- Logistic regression tends to be well calibrated; SVMs and trees often are not. AUC does not see calibration.

Situation	Use
Balanced classes, equal costs	accuracy + confusion matrix
Rare positives	precision, recall, F1, PR curve
Compare rankers	ROC AUC
Probability is used	log-loss, Brier, reliability
Known costs	expected cost, τ^*



Metrics – Scoring probabilities, by hand

Four claims, the model's probability of fraud p , and what really happened:

claim	y	p	log-loss term	Brier term
A	1	0.90	$-\ln 0.90 = 0.11$	$(0.90 - 1)^2 = 0.01$
B	1	0.60	$-\ln 0.60 = 0.51$	$(0.60 - 1)^2 = 0.16$
C	0	0.20	$-\ln 0.80 = 0.22$	$(0.20 - 0)^2 = 0.04$
D	0	0.95	$-\ln 0.05 = 3.00$	$(0.95 - 0)^2 = 0.90$

For $y = 1$ the log-loss term is $-\ln p$; for $y = 0$ it is $-\ln(1 - p)$.
The Brier term is always $(p - y)^2$.

$$\text{log-loss} = \frac{0.11 + 0.51 + 0.22 + 3.00}{4} = 0.96 \quad \text{Brier} = \frac{0.01 + 0.16 + 0.04 + 0.90}{4} = 0.28$$

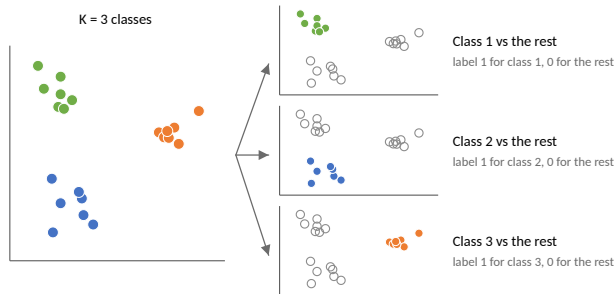
One confident mistake (D: 95 % sure of fraud, but legit) costs 3.00, more than the other three claims together. Log-loss punishes confident mistakes very hard; a Brier term can never exceed 1.

Baseline "always say 0.5": $\text{log-loss} = \ln 2 = 0.69$, $\text{Brier} = 0.25$. Because of claim D, this model is **worse than a coin flip** on both.

Accuracy with a 0.5 threshold: 3 of 4 right (75 %). It cannot see how sure the model was. Use log-loss or Brier when the probability itself is used (prices, reserves, expected costs).

Logistic Regression – More than 2 classes?

One-vs-all (one-vs-rest):



If we have K classes, repeat for each class:

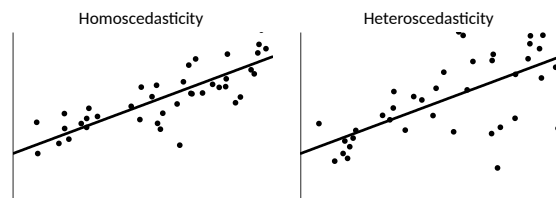
- The class label is set to 1 and the rest to 0.
- Learn a binary classifier.

Run all classifiers and select the one with the highest $h_{\theta}(\mathbf{x})$.

Note: **softmax** regression does it in one model (next chapter).

Linear and logistic regression – Summary

- Pros:
 - Simple model and interpretability.
 - Computationally efficient.
- Cons:
 - Overly-simplistic (linearity assumptions).
 - Affected by outliers.
 - Correlated features (multicollinearity) make the coefficients unstable and hard to interpret; the predictions can still be fine.
 - Solution: drop or combine features, dimensionality reduction (like PCA), or regularization.
 - Linear regression only: its standard errors assume homoscedasticity (constant variance around the line).



LAB 3

Logistic regression from scratch

The model of this chapter coded by hand on two classes of points, then the same with scikit-learn.

- 1 Standardise the features; the sigmoid.
- 2 The binary cross-entropy and its gradient, written by hand.
- 3 Batch gradient descent and the cost curve.
- 4 Predictions, the decision boundary and the probability surface.
- 5 Metrics: accuracy, the confusion matrix, ROC and AUC.
- 6 The same model with scikit-learn, step by step, and the two boundaries compared.

Also ready now: [Lab 2 · polynomial regression and regularisation](#) (overfitting, Ridge and Lasso). It comes back at the end of chapter 05.

[CO Open in Colab](#)

notebook Lab 3 - Logistic Regression.ipynb
you need this chapter: sigmoid, cross-entropy, its gradient, the metrics and ROC; Lab 1
runs on CPU; every cell runs in seconds

Next: softmax regression (chapter 04).

Machine Learning

Máster en Innovación Tecnológica (MINT)

Next: Softmax Regression

imanolgranada@gmail.com

Machine Learning

Máster en Innovación Tecnológica (MINT)

Softmax Regression

imanolgranada@gmail.com



1

**Softmax
Regression**

$$z_k = \boldsymbol{\theta}_k^\top \mathbf{x}$$

↓

$$\frac{e^{z_k}}{\sum_l e^{z_l}}$$



2

Softmax Regression – Motivation

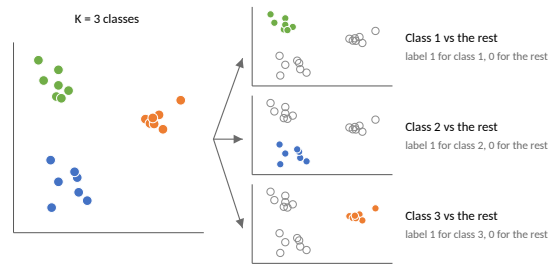
Labels $y^{(i)} \in \{1, \dots, K\}$, e.g. digits, iris species, vehicle types, damage severity.

One-vs-rest (previous chapter) trains K independent binary classifiers:

- K separate models, K separate losses.
- The K outputs **don't sum to 1**: not a probability distribution.

Softmax regression (multinomial logistic regression): one model, one loss, and the output **is** a distribution over the K classes.

Same recipe as always: linear scores $\rightarrow$ probabilities $\rightarrow$ minus the log-likelihood $\rightarrow$ gradient descent.



Softmax Regression – Representing the classes

The label is one of K names: *setosa*, *versicolor*, *virginica*. How do we write it as numbers?

Option A · an integer $y \in \{1, \dots, K\}$: *setosa* = 1, *versicolor* = 2, *virginica* = 3.

Compact, but it invents an order and distances: it says *virginica* is "twice" *versicolor* and "further" from *setosa*. None of that is true. Fine for storing labels, wrong as a regression target.

Option B · one-hot vector $\mathbf{y} \in \{0, 1\}^K$: a 1 in the position of the true class, 0 elsewhere.

$$y = 2 \rightarrow \mathbf{y} = (0, 1, 0) \quad \text{in general } y_k = \mathbb{I}[y = k]$$

No order, all classes at the same distance. It is a **probability distribution** that puts all its mass on the true class. Same trick as one-hot **input** encoding in chapter 02, now used for the **output**.

label	y	one-hot $\mathbf{y}$
setosa	1	(1, 0, 0)
versicolor	2	(0, 1, 0)
virginica	3	(0, 0, 1)

The model will output a vector $\mathbf{p} = (p_1, p_2, p_3)$ of probabilities, e.g. (0.1, 0.7, 0.2). Training pushes $\mathbf{p}$ towards the one-hot target (0, 1, 0): the two vectors are directly comparable.

For the whole dataset: $\mathbf{Y} \in \{0, 1\}^{m \times K}$, one one-hot row per example. Each row sums to 1.

Softmax Regression – The model

Give **each class its own parameter vector** $\theta_k \in \mathbb{R}^{n+1}$ and a linear score (logit):

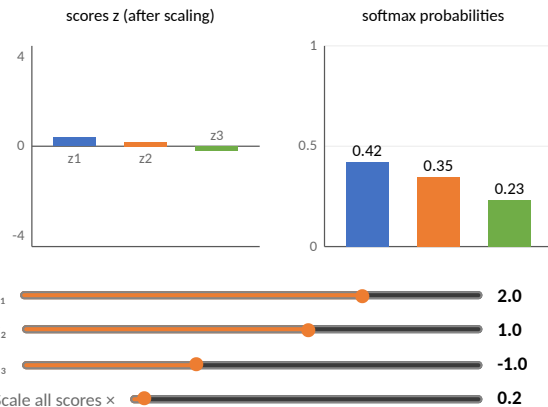
$$z_k = \theta_k^\top \mathbf{x}, \quad k = 1, \dots, K$$

Stack them: $\Theta = [\theta_1, \dots, \theta_K] \in \mathbb{R}^{(n+1) \times K}$, so $\mathbf{z} = \Theta^\top \mathbf{x}$ and, for all the data, $\mathbf{Z} = \mathbf{X}\Theta \in \mathbb{R}^{m \times K}$.

Turn the scores into probabilities with the **softmax**:

$$P(y = k \mid \mathbf{x}; \Theta) = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{l=1}^K e^{z_l}}$$

Prediction: $\hat{y} = \arg \max_k \text{softmax}(\mathbf{z})_k = \arg \max_k \theta_k^\top \mathbf{x}$.



Scale up: it approaches the one-hot arg-max. Scale down: it becomes uniform. A "soft" arg-max.

► Multiply by 0.2: much flatter (0.42, 0.35, 0.23), closer to uniform: the model is unsure

Softmax Regression – Properties

1 • A valid distribution. Each output is > 0 (exponentials) and they sum to 1 (we divide by their sum).

2 • Shift invariance. Adding the same constant c to every score changes nothing:

$$\frac{e^{z_k+c}}{\sum_l e^{z_l+c}} = \frac{e^c e^{z_k}}{e^c \sum_l e^{z_l}} = \text{softmax}(\mathbf{z})_k$$

- The parameters are not identifiable: adding the same vector to every θ_k changes nothing (fix: L2, or $\theta_K = 0$).
- Numerical stability:** compute with $\mathbf{z} - \max_l z_l$ so no exponential overflows.

3 • Log-odds between two classes are linear:

$$\log \frac{P(y = k \mid \mathbf{x})}{P(y = l \mid \mathbf{x})} = z_k - z_l = (\theta_k - \theta_l)^\top \mathbf{x}$$

The boundary between any two classes is a hyperplane: the decision regions are convex polygons.

Softmax Regression – $K = 2$ is logistic regression

With two classes, divide numerator and denominator by e^{z_1} :

$$\begin{aligned} P(y = 1 \mid \mathbf{x}) &= \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{z_2 - z_1}} = \frac{1}{1 + e^{-(z_1 - z_2)}} \\ &= g((\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^\top \mathbf{x}) \end{aligned}$$

This is logistic regression with $\boldsymbol{\theta} = \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2$. Only the **difference** matters: property 2 again.

Everything we learned for logistic regression is the $K = 2$ special case of what follows.

Softmax Regression – Cost function

Labels as **one-hot** vectors (slide 4): $y_k^{(i)} = \mathbb{1}[y^{(i)} = k]$, e.g. class 2 of 3 $\rightarrow (0, 1, 0)$.

Write $p_k^{(i)} = P(y = k \mid \mathbf{x}^{(i)}; \boldsymbol{\Theta})$.

The model says $y^{(i)}$ is **categorical**. One formula for every class (only the true class's factor is not 1):

$$P(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\Theta}) = \prod_{k=1}^K (p_k^{(i)})^{y_k^{(i)}}$$

Take minus the mean log-likelihood: the **categorical cross-entropy**,

$$J(\boldsymbol{\Theta}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log p_k^{(i)} = -\frac{1}{m} \sum_{i=1}^m \log p_{y^{(i)}}^{(i)}$$

Minus the log of the probability given to the **correct** class. For $K = 2$ it is exactly the binary cross-entropy.

Useful form for the derivation: $-\log p_y = -z_y + \log \sum_l e^{z_l}$.

Softmax Regression – Gradient

Step 1 • derivative by the scores. One example with true class y :

$$\ell = -z_y + \log \sum_l e^{z_l}.$$

$$\frac{\partial \ell}{\partial z_k} = -\mathbb{1}[y = k] + \frac{e^{z_k}}{\sum_l e^{z_l}} = p_k - y_k \quad \Rightarrow \quad \nabla_{\mathbf{z}} \ell = \mathbf{p} - \mathbf{y}$$

Predicted distribution minus the one-hot target.

Step 2 • chain rule. $z_k = \theta_k^\top \mathbf{x}$ depends only on θ_k , and $\nabla_{\theta_k} z_k = \mathbf{x}$:

$$\nabla_{\theta_k} J = \frac{1}{m} \sum_{i=1}^m (p_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}$$

$$\nabla_{\Theta} J = \frac{1}{m} \mathbf{X}^\top (\mathbf{P} - \mathbf{Y})$$

$\mathbf{P}, \mathbf{Y} \in \mathbb{R}^{m \times K}$: predicted probabilities and one-hot labels. Batch GD:

$$\Theta := \Theta - \alpha \nabla_{\Theta} J.$$

model	gradient
linear	$\frac{1}{m} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y})$
logistic	$\frac{1}{m} \mathbf{X}^\top (g(\mathbf{X}\theta) - \mathbf{y})$
softmax	$\frac{1}{m} \mathbf{X}^\top (\mathbf{P} - \mathbf{Y})$
Always (prediction - target) $\times$ input.	

Softmax Regression – Convexity

$\ell = -z_y + \text{LSE}(\mathbf{z})$ with $\text{LSE}(\mathbf{z}) = \log \sum_l e^{z_l}$, and $\mathbf{z}$ is linear in Θ . A linear term is convex, so it is enough to show that **LSE is convex**.

Its gradient is $\mathbf{p} = \text{softmax}(\mathbf{z})$. Differentiating $p_k = e^{z_k} / \sum_l e^{z_l}$ gives $\partial p_k / \partial z_j = p_k (\mathbb{1}[k = j] - p_j)$, so the Hessian is $\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$.

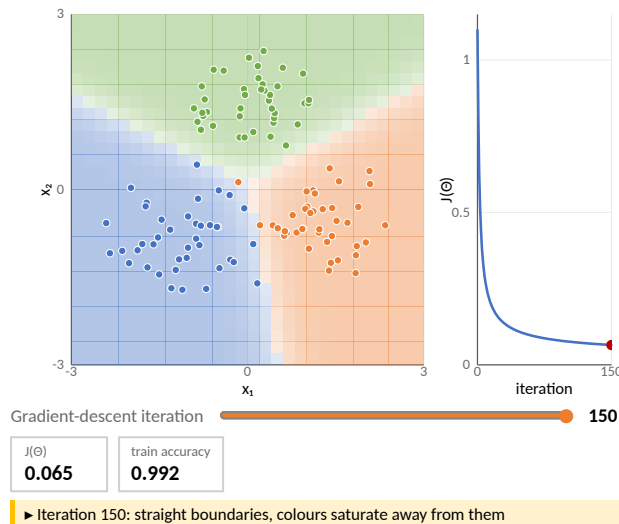
For any vector $\mathbf{v}$:

$$\mathbf{v}^\top (\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top) \mathbf{v} = \sum_k p_k v_k^2 - \left(\sum_k p_k v_k \right)^2 = \text{Var}_{k \sim \mathbf{p}}[v_k] \geq 0$$

It is a **variance**, so it is never negative: LSE is convex, hence J is convex. ■

Not *strictly* convex, because of shift invariance. L2 regularization makes it strictly convex.

Softmax Regression – Training it



Three classes, two features, softmax from scratch with the gradient $\frac{1}{m} \mathbf{X}^\top (\mathbf{P} - \mathbf{Y})$.

- Iteration 0: $\Theta = \mathbf{0}$, every class has probability $\frac{1}{3}$.
- The colour is the predicted class; its strength is the predicted probability.
- Boundaries between regions are straight lines (property 3).

Implementation detail: subtract $\max_k z_k$ before exponentiating.

Softmax Regression – Multiclass metrics

	One-vs-rest	Softmax
training	K binary models	one joint model
probabilities	don't sum to 1	a proper distribution
use when	any binary classifier (SVMs); multi-label	mutually exclusive classes

The confusion matrix becomes $K \times K$: rows = actual class, columns = predicted class. The diagonal holds the hits.

Precision, recall and F_1 are computed **per class** (that class vs the rest), then averaged:

- **macro**: the plain mean over the classes. Every class counts the same: a rare class that is badly predicted pulls it down.
- **weighted**: the mean weighted by the class frequencies. Dominated by the big classes.
- **micro**: pool all the TP, FP and FN first. For single-label problems it equals the accuracy.

Also useful: **balanced accuracy** = the macro average of the recalls.

Softmax Regression – Multiclass metrics, a worked example

	predicted			
	minor	moderate	severe	
minor	65	4	1	total 70
moderate	5	12	3	total 20
severe	1	3	6	total 10
	$\Sigma 71$	$\Sigma 19$	$\Sigma 10$	

rows: actual class

100 damaged cars: 70 minor, 20 moderate, 10 severe. Rows: actual, columns: predicted.

Per class. Precision = diagonal ÷ column total; recall = diagonal ÷ row total:

class	precision	recall	F_1	share
minor	$65/71 = 0.92$	$65/70 = 0.93$	0.92	70 %
moderate	$12/19 = 0.63$	$12/20 = 0.60$	0.62	20 %
severe	$6/10 = 0.60$	$6/10 = 0.60$	0.60	10 %

$$\text{accuracy} = \text{micro-}F_1 = \frac{65+12+6}{100} = 0.83$$

$$\text{macro-}F_1 = \frac{0.92+0.62+0.60}{3} = 0.71$$

every class counts the same

$$\text{weighted-}F_1 = 0.7 \cdot 0.92 + 0.2 \cdot 0.62 + 0.1 \cdot 0.60 = 0.83$$

weights = the class shares

Accuracy looks fine, but the costly **severe** class is caught only 6 times out of 10. Macro- F_1 exposes it; always show the per-class table too.

Softmax Regression – Summary

Piece	Formula
scores	one parameter vector per class, $z_k = \theta_k^\top \mathbf{x}$
probabilities	$\text{softmax}(\mathbf{z})_k = e^{z_k} / \sum_l e^{z_l}$
loss	categorical cross-entropy = $-\frac{1}{m} \sum_i \log p_{y^{(i)}}^{(i)}$ = negative log-likelihood
gradient	$\frac{1}{m} \mathbf{X}^\top (\mathbf{P} - \mathbf{Y})$
convexity	the Hessian of log-sum-exp is a variance
special case	$K = 2$ is logistic regression
in practice	subtract $\max_k z_k$; <code>LogisticRegression()</code> in scikit-learn is multinomial by default

Machine Learning

Máster en Innovación Tecnológica (MINT)

Next: Regularization

imanolgranada@gmail.com



Machine Learning

Máster en Innovación Tecnológica (MINT)

Regularization

imanolgranada@gmail.com



1

Regularization

$$J(\theta) + \lambda R(\theta)$$



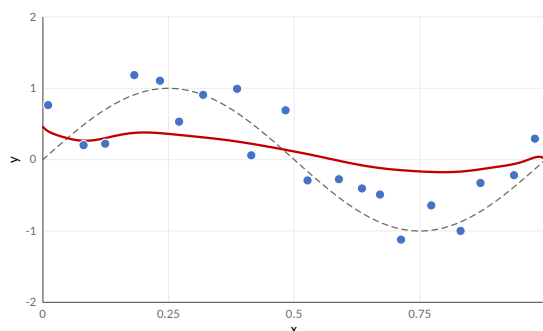
2

Regularization – Why?

Three problems that all show up as **large, unstable parameters**:

1. **Overfitting**: many features (or a high polynomial degree) relative to m . The model fits the noise; the θ_j become huge with alternating signs.
2. **Collinearity**: correlated features make $\mathbf{X}^\top \mathbf{X}$ nearly singular. Small changes in the data change θ a lot.
3. **Separable classes** in logistic regression: $\|\theta\| \rightarrow \infty$.

Regularization accepts a little more training error (**bias**) in exchange for a more stable model (less **variance**).



Degree **15**

λ (log scale) **10.00**

train RMSE
0.502

test RMSE
0.598

verdict
Underfitting

► $\lambda = 10$: over-penalized, the curve flattens: underfitting

Regularization – The idea in one picture

The degree-15 curve on the previous slide ($\lambda = 0$) wiggles to reach the noisy points. Written as a plain polynomial $\theta_0 + \theta_1 t + \dots + \theta_{15} t^{15}$ (with $t = 2x - 1$), its coefficients are

$\theta \approx (0, -4, 3, 90, -70, -1267, 373, 8302, \dots)$ up to $\approx 49\,000$

Wiggles need huge coefficients with alternating signs. A smooth curve has small ones. So: make large coefficients **expensive**.

$$\text{minimize} \quad \underbrace{J(\theta)}_{\text{fit the data}} + \lambda \cdot \underbrace{\text{size}(\theta)}_{\text{a tax on big } \theta\text{'s}}$$

- λ is the **tax rate**. $\lambda = 0$: no tax, plain least squares. $\lambda \rightarrow \infty$: any non-zero θ_j is ruinous, all go to 0, a flat line.
- In between, the model gives up a little training error to keep θ small. Less wiggle, better on new data.

Two ways to measure "size":

Name	size(θ)
L2 · Ridge	$\sum_{j=1}^n \theta_j^2$
L1 · Lasso	$\sum_{j=1}^n \theta_j $

Squares punish big coefficients hard and small ones hardly at all. Absolute values punish every coefficient at the same rate, even tiny ones. That difference is the whole story of Ridge vs Lasso.

Bias–variance: a bit more **bias** (worse fit on train) for a lot less **variance** (stability on new data).

Regularization – The regularized cost

$$J_{\text{reg}}(\boldsymbol{\theta}) = \underbrace{J(\boldsymbol{\theta})}_{\text{fit the data}} + \underbrace{\lambda R(\boldsymbol{\theta})}_{\text{keep } \boldsymbol{\theta} \text{ small}}, \quad \lambda \geq 0$$

Name	Penalty $R(\boldsymbol{\theta})$	Effect
L2 · Ridge · weight decay	$\sum_{j=1}^n \theta_j^2 = \ \boldsymbol{\theta}\ _2^2$	shrinks all coefficients smoothly
L1 · Lasso	$\sum_{j=1}^n \theta_j = \ \boldsymbol{\theta}\ _1$	sets some exactly to 0 (feature selection)
Elastic Net	a mix of both	sparse and stable with correlated features

Two rules that always apply:

- **Do not penalize the intercept** θ_0 (the sums start at $j = 1$). It only sets the average level of y ; shrinking it would bias every prediction.
- **Standardize the features first.** The size of θ_j depends on the units of x_j : income in € vs k€ changes it 1000×. Without scaling, the tax is arbitrary.

λ is a hyperparameter, chosen by cross-validation (later slide). scikit-learn often writes $C = 1/\lambda$: large C = weak regularization.

Regularization – L2 (Ridge): the gradient, slowly

We write the L2 penalty with a $\frac{1}{2m}$ in front, only so that the derivative comes out clean:

$$J_{\text{reg}}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Step 1 · derivative of the penalty with respect to one θ_j . Only the j -th term of the sum contains θ_j ; the others are constants:

$$\frac{\partial}{\partial \theta_j} \left(\frac{\lambda}{2m} \sum_{k=1}^n \theta_k^2 \right) = \frac{\lambda}{2m} \cdot 2\theta_j = \frac{\lambda}{m} \theta_j$$

Step 2 · add it to the data gradient (whatever J is: least squares or cross-entropy):

$$\frac{\partial J_{\text{reg}}}{\partial \theta_j} = \frac{\partial J}{\partial \theta_j} + \frac{\lambda}{m} \theta_j \quad (j \geq 1; \text{ for } \theta_0 \text{ nothing changes})$$

Step 3 · plug into gradient descent and regroup the θ_j terms:

$$\begin{aligned} \theta_j &:= \theta_j - \alpha \left(\frac{\partial J}{\partial \theta_j} + \frac{\lambda}{m} \theta_j \right) \\ &= \theta_j \left(1 - \frac{\alpha \lambda}{m} \right) - \alpha \frac{\partial J}{\partial \theta_j} \end{aligned}$$

Weight decay. Every step first multiplies θ_j by a number slightly below 1 (e.g. $\alpha = 0.1$, $\lambda = 1$, $m = 100$: by 0.999), then takes the usual gradient step. Coefficients that the data does not keep pushing up slowly decay towards 0.

This is exactly what `weight_decay` does in neural-network optimizers.

Regularization – L2 (Ridge): the closed form, slowly

For linear regression, $J = \frac{1}{2m} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2$ and its gradient is (chapter 02) $\nabla J = \frac{1}{m} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$.

Step 1 • the penalty gradient as a matrix. Step 1 of the previous slide for all j at once, with a 0 for the intercept:

$$\nabla \left(\frac{\lambda}{2m} \sum_{j \geq 1} \theta_j^2 \right) = \frac{\lambda}{m} \mathbf{L} \boldsymbol{\theta}, \quad \mathbf{L} = \text{diag}(0, 1, \dots, 1)$$

Step 2 • set the total gradient to zero and drop the common $\frac{1}{m}$:

$$\mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) + \lambda \mathbf{L} \boldsymbol{\theta} = \mathbf{0}$$

Step 3 • collect the $\boldsymbol{\theta}$ terms on the left, the rest on the right:

$$\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} + \lambda \mathbf{L} \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y} \Rightarrow (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{L}) \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y}$$

$$\boldsymbol{\theta}_{\text{ridge}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{L})^{-1} \mathbf{X}^\top \mathbf{y}$$

Compare with the normal equation $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$: Ridge just adds λ to the diagonal of $\mathbf{X}^\top \mathbf{X}$ (except the intercept entry).

One standardized feature ($\mathbf{X}^\top \mathbf{X} = m$, no intercept).

Let $z = \mathbf{X}^\top \mathbf{y} / m$ be the OLS coefficient:

$$(m + \lambda) \theta = \mathbf{X}^\top \mathbf{y} = m z \Rightarrow \theta_{\text{ridge}} = \frac{z}{1 + \lambda/m}$$

Ridge divides. Smaller than OLS, but never exactly zero.

The inverse always exists for $\lambda > 0$, even with duplicated features or $n > m$: for $\mathbf{v} \neq \mathbf{0}$,

$\mathbf{v}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{L}) \mathbf{v} = \|\mathbf{X} \mathbf{v}\|^2 + \lambda \sum_{j \geq 1} v_j^2 > 0$ (if all $v_{j \geq 1} = 0$ then $\mathbf{X} \mathbf{v} = v_0 \mathbf{1} \neq \mathbf{0}$). Positive definite $\rightarrow$ a unique minimum.

Regularization – L1 (Lasso): see why exact zeros happen

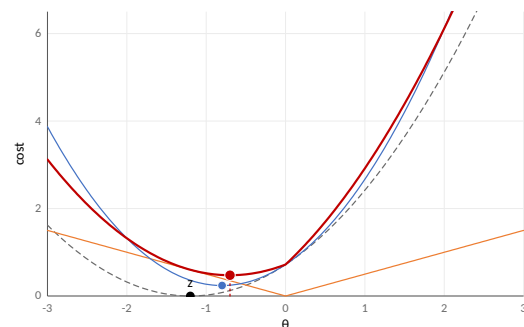
One feature again. Without a penalty the best coefficient is the OLS value z . With the penalty we look for the lowest point of two costs added together:

- Dashed parabola $\frac{1}{2}(\theta - z)^2$: the data, how far θ is from z .
- Orange V $\lambda|\theta|$: the L1 tax, λ for every unit of $|\theta|$.
- Red curve: their sum. Its lowest point is the Lasso coefficient. (Blue: the same with Ridge.)

Look at $\theta = 0$. The parabola is smooth there (slope $-z$), but the V has a **corner**: slope $+\lambda$ on the right, $-\lambda$ on the left.

Slope of the red curve just right of 0: $\lambda - z$. Just left of 0: $-\lambda - z$. If $|z| \leq \lambda$ the red curve **rises on both sides** of 0 ($\lambda - z \geq 0$ and $-\lambda - z \leq 0$), so its lowest point is exactly $\theta = 0$.

Ridge has no corner (the slope of its penalty at 0 is $\lambda \cdot 0 = 0$), so its minimum reaches 0 only when $z = 0$.



z (OLS)
-1.2

λ
0.50

Lasso θ (red)

-0.70

Ridge θ (blue)

-0.80

what happens

Pulled λ towards 0

► $z = -1.2$: the mirror case, Lasso -0.70 (Ridge -0.80)

Regularization – L1 (Lasso): the formula

From the picture to the formula. The same cost, a parabola centred at z plus the L1 tax:

$$g(\theta) = \frac{1}{2}(\theta - z)^2 + \lambda|\theta|$$

$|\theta|$ has no derivative at its corner, so we treat the two sides and the corner separately. Examples use $\lambda = 0.5$.

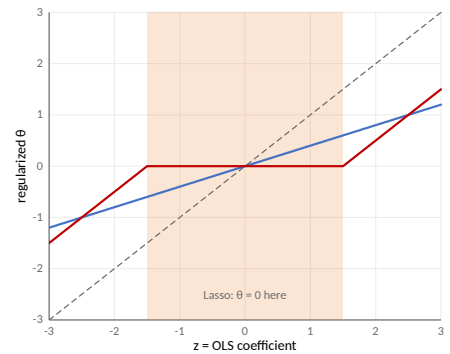
If $\theta > 0$: $g' = \theta - z + \lambda = 0 \Rightarrow \theta = z - \lambda$, valid only if $z > \lambda$. E.g. $z = 2 \rightarrow 1.5$.

If $\theta < 0$: $g' = \theta - z - \lambda = 0 \Rightarrow \theta = z + \lambda$, valid only if $z < -\lambda$. E.g. $z = -1.2 \rightarrow -0.7$.

Otherwise ($|z| \leq \lambda$) the minimum sits at the corner: $\theta = 0$. E.g. $z = 0.3 \rightarrow 0$.

$$\theta_{\text{lasso}} = \text{sign}(z) \max(|z| - \lambda, 0)$$

Lasso subtracts, Ridge divides. The L2 penalty's slope 2θ vanishes near 0, so it never pushes all the way. The L1 penalty pushes with constant force λ , however small θ is: anything with $|z| \leq \lambda$ is pushed **exactly** to 0.



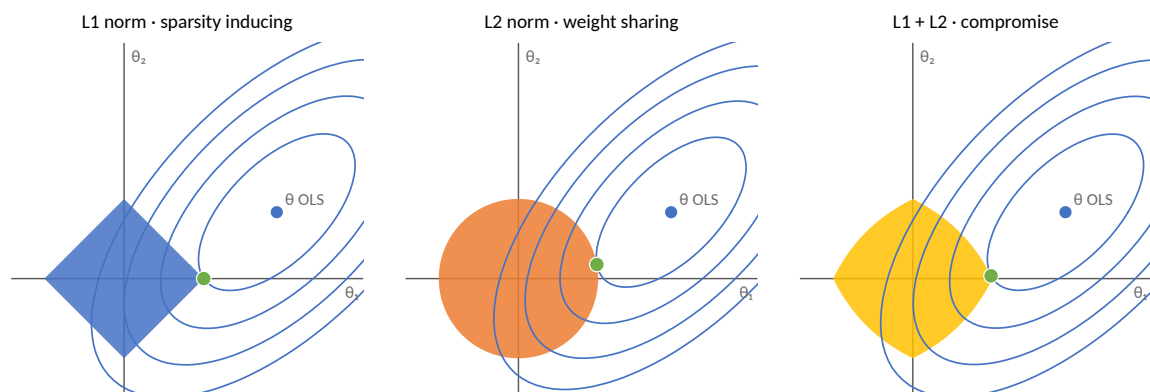
λ 1.50

— OLS — Ridge — Lasso

Both penalties here are per example: Ridge $\frac{1}{2}(\theta - z)^2 + \frac{\lambda}{2}\theta^2$ gives $\theta = \frac{z}{1+\lambda}$ (the λ/m of the Ridge slide is written λ).

► $\lambda = 1.5$: a wide dead zone, only large coefficients survive Lasso

Regularization – The geometric picture



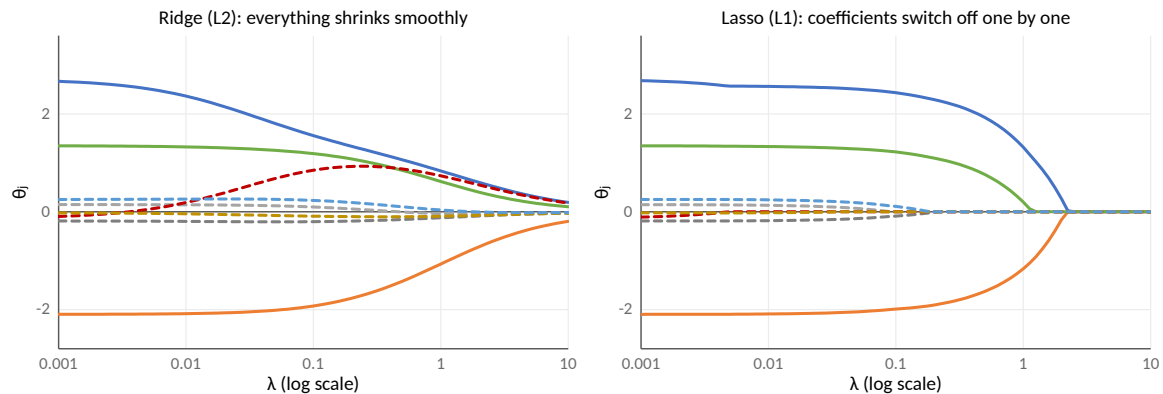
Minimizing $J + \lambda R$ is the same as minimizing J subject to $R(\theta) \leq t$. The solution (green) is where the contours of J **first touch** the region. The L1 diamond has **corners on the axes**, so it usually touches at a corner, where $\theta_2 = 0$.

Elastic Net: $\lambda \left[\rho \|\theta\|_1 + \frac{1-\rho}{2} \|\theta\|_2^2 \right]$, $\rho \in [0, 1]$.

Lasso is unstable on correlated features: it keeps one of near-duplicates arbitrarily, or splits the weight erratically. The L2 part makes them share the weight; the L1 part still gives sparsity.

Regularization – Coefficient paths

8 standardized features, only 3 truly matter (solid lines; dashed = irrelevant). Feature 5 (dark red, dashed) is a near-copy of feature 0 (blue).



Ridge shrinks everything gradually. Lasso switches coefficients off one by one: at a moderate λ only the useful features survive. Which one of a correlated pair Lasso keeps is somewhat arbitrary.

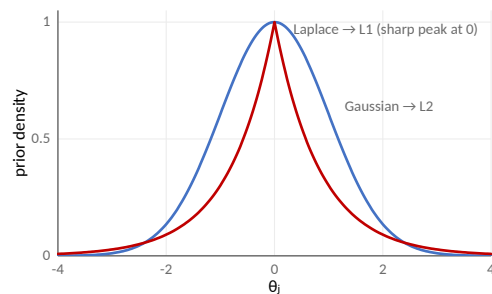
Regularization – A prior in disguise (MAP)

Bayes: $p(\theta | \mathcal{D}) \propto p(\mathcal{D} | \theta) p(\theta)$. Maximize the posterior:

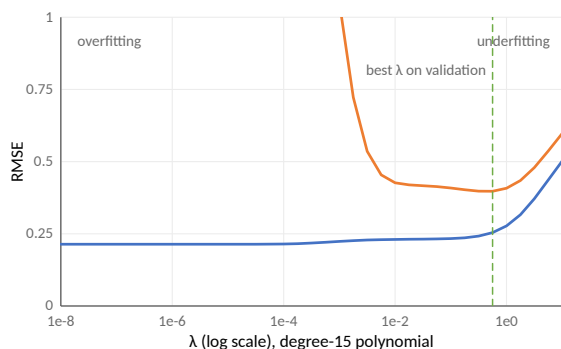
$$\hat{\theta}_{\text{MAP}} = \arg \min_{\theta} \underbrace{[-\log p(\mathcal{D} | \theta)]}_{\text{loss}} + \underbrace{[-\log p(\theta)]}_{\text{penalty}}$$

- **Gaussian prior** $\theta_j \sim \mathcal{N}(0, \tau^2)$: $-\log p(\theta) = \frac{1}{2\tau^2} \sum_j \theta_j^2 + c \rightarrow$ **L2**.
- **Laplace prior** $p(\theta_j) \propto e^{-|\theta_j|/b}$: $-\log p(\theta) = \frac{1}{b} \sum_j |\theta_j| + c \rightarrow$ **L1**.

For linear regression with noise σ^2 : **MAP = Ridge** with $\lambda = \sigma^2/\tau^2$. A strong prior (small τ) or noisy data (large σ) means more regularization.



Regularization – Choosing λ



Degree-15 polynomial on 20 noisy points. RMSE is the typical error, in the units of y . Blue: error on the data used to fit. Orange: error on data not used to fit.

λ is a **hyperparameter**: the fit cannot learn it, we choose it. How to read the picture:

- **Left, λ tiny: overfitting.** Training error is as low as it gets, validation error is high: the curve memorizes the noise.
- **Right, λ huge: underfitting.** Both errors go up: the penalty squeezes the curve towards a constant.
- **In between** the validation error has a minimum (dashed green): that is the λ we want.
- **Training error cannot choose λ :** it only grows with λ , so it always says $\lambda = 0$. We need data the fit has not seen.

But we cannot keep peeking at the test set to choose λ , and a single validation split of 20 points is noisy. The next slide shows the standard answer: cross-validation.

Regularization – Choosing λ with cross-validation

1. **Grid.** Candidate values on a **log scale**, e.g. $10^{-6}, 10^{-4}, \dots, 10^1$: λ acts multiplicatively, so 0.1 vs 0.2 matters as much as 10 vs 20.
2. **Folds.** For each λ split the training data into $k = 5$ folds. Fit on 4 folds, measure the error on the 5th; repeat so each fold validates once: 5 errors.
3. **Average** the 5 errors: one number per λ (that is the orange curve of the previous slide).
4. **Choose** the λ with the lowest mean (or the largest λ within one standard error of it: a simpler model for free).
5. **Refit** on all the training data with that λ , and look at the test set **once**, at the end.

Put the scaling **inside** the loop (scikit-learn `Pipeline`), or each validation fold leaks into μ, σ . `RidgeCV`, `LassoCV`, `LogisticRegressionCV` do all of this for you.

The **real numbers** for the 20 points of the previous slide: validation RMSE of each fold.

λ	fold 1	fold 2	fold 3	fold 4	fold 5	mean
10^{-6}	46	53	15	14	9.31	27
10^{-4}	2.41	1.60	2.20	4.98	3.13	2.86
10^{-3}	1.36	0.98	1.57	3.20	1.08	1.64
10^{-2}	0.60	0.67	1.12	1.65	0.77	0.96
10^{-1}	0.52	0.46	0.51	0.58	0.70	0.56
10^0	0.37	0.42	0.37	0.41	0.62	0.44
10^1	0.45	0.60	0.45	0.52	0.76	0.55

$\lambda = 10^{-6}$ explodes (folds from 9 to 53: overfitting). The mean is lowest at $\lambda = 1$ (0.44), highlighted, and rises again at 10: the same U shape as the orange curve.

Regularization – Logistic regression

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(\mathbf{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

$$\nabla J = \frac{1}{m} \left[\mathbf{X}^{\top} (g(\mathbf{X}\theta) - \mathbf{y}) + \lambda \mathbf{L}\theta \right]$$

- The Hessian gains $+\frac{\lambda}{m} \mathbf{L}$: strictly convex, and it **always has a finite solution**, even for separable data.
- L1 works the same way and gives sparse logistic models.
- scikit-learn's `LogisticRegression` is **L2-regularized by default** with $C = 1/\lambda = 1$. Large C = weak regularization.

Regularization – Input variables

Scale numerics

Standardize before GD and before any penalty. Compute μ_j, σ_j on the **training** data only and reuse them everywhere. After standardizing, θ_j = effect of +1 standard deviation: comparable across features.

Encode categoricals

Ordinal codes for ordered levels, one-hot for unordered ones (chapter 02: "which encoder when?"). One-hot: keep $K - 1$ dummies with an intercept (dummy trap). With L2, keeping all K is also fine: the penalty makes the solution unique. High-cardinality categories: group rare levels, or target-encode inside the CV folds.

Add transforms

pattern	feature
curved	x^2 , splines
skewed	$\log(1 + x)$
interaction	$x_1 x_2$
periodic	$\sin \frac{2\pi t}{T}, \cos \frac{2\pi t}{T}$

More features means more variance: pair them with regularization and CV. Watch for **leakage features**, known only after the outcome (e.g. "claim paid amount" when predicting fraud at claim opening).

Practical guide – what to do when... (the data)

Situation	What you see	What to do
features on very different scales	GD zig-zags; the penalty hits some features harder	standardize , with μ and σ from the training data
a skewed feature (income, claim amount)	a long right tail, a few huge values	use $\log(1 + x)$
outliers	a few points far from the rest	first check they are not errors; <code>RobustScaler</code> , the MAE or Huber loss; trees are robust
missing values	empty cells (NaN)	impute (median or most frequent) + a 0/1 column "was missing"; fit on train only
a categorical with many levels	postcode, car model	group the rare levels; frequency or target encoding
a level never seen in training	an error at prediction time	<code>OneHotEncoder(handle_unknown="ignore")</code>
a rare positive class	great accuracy, poor recall	class weights, move the threshold, PR metrics (chapter 03)

Every "fit" in the last column (scaler, imputer, encoder) is learned on the training data only and then applied to validation and test: put them in a `Pipeline`.

Practical guide – what to do when... (the model)

Symptom	Diagnosis	What to do
high error on train and validation	underfitting (bias)	better features (x^2 , interactions, logs), a smaller λ , a more flexible model
low train error, high validation error	overfitting (variance)	more data, a larger λ , fewer features, a simpler model
validation score suspiciously good	leakage?	preprocessing inside the CV folds; drop features known only after the outcome; no duplicates across splits
coefficients jump between runs, odd signs	collinearity	check the VIF (collinearity slides); drop or combine features; Ridge
residuals show a curve	non-linearity	add transforms (x^2 , $\log x$, splines)
residuals fan out	heteroscedasticity	model $\log y$; weighted least squares
a skewed target (prices, claim costs)	large errors on the expensive cases	fit $\log y$; $e^{\hat{y}}$ is the median, so multiply by $e^{\hat{\sigma}^2/2}$ for the mean

First compare train and validation errors: that single check tells you whether to add complexity or to remove it.

Regularization – Collinearity: what goes wrong

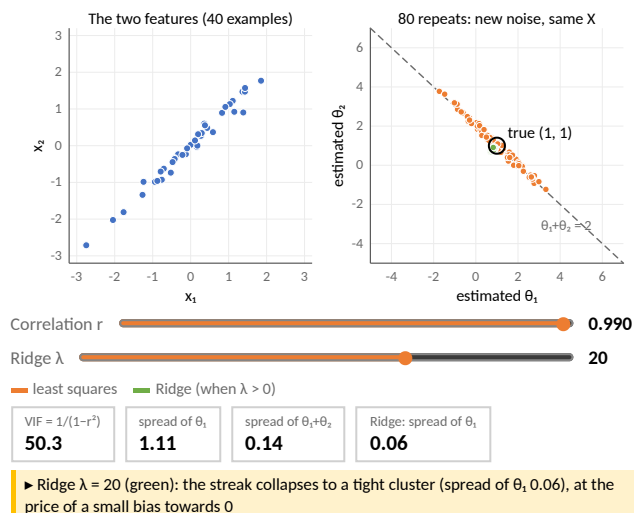
Collinear features carry (almost) the same information: $x_2 \approx x_1$ (house area in m^2 and in sq ft; height and arm length). Then many parameter pairs predict equally well:

$$\hat{y} = 1 \cdot x_1 + 1 \cdot x_2 \approx 2 \cdot x_1 + 0 \cdot x_2 \approx 0 \cdot x_1 + 2 \cdot x_2.$$

The data cannot tell them apart; they only pin down the **sum** $\theta_1 + \theta_2$.

Right: repeat the experiment 80 times (same $\mathbf{X}$, fresh noise) and fit each time. With independent features the estimates form a round cloud around the truth. As the correlation r grows the cloud stretches into a streak along $\theta_1 + \theta_2 = 2$: **each estimate is unreliable, their sum is not**. Predictions stay fine; interpretations do not.

Ridge adds λ to the diagonal of $\mathbf{X}^\top \mathbf{X}$, which removes the weak direction: the streak shrinks, at the price of a small bias towards 0.



Regularization – Collinearity: the formulas

Under the model assumptions, with $\hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ and $\mathbf{y} = \mathbf{X}\theta + \epsilon$:

$$\hat{\theta} = \theta + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \epsilon \Rightarrow \mathbb{E}[\hat{\theta}] = \theta \text{ (unbiased)}$$

$$\text{Cov}(\hat{\theta}) = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$$

Nearly dependent columns $\rightarrow \mathbf{X}^\top \mathbf{X}$ has a tiny eigenvalue $\rightarrow$ its inverse has a huge one $\rightarrow$ **the variance of the coefficients explodes**. Predictions barely suffer; interpretations do.

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

Two standardized features with correlation r (so $\frac{1}{m} \mathbf{X}^\top \mathbf{X}$ has 1 on the diagonal and r off it):

$$\left(\frac{1}{m} \mathbf{X}^\top \mathbf{X} \right)^{-1} = \frac{1}{1 - r^2} \begin{pmatrix} 1 & -r \\ -r & 1 \end{pmatrix}$$

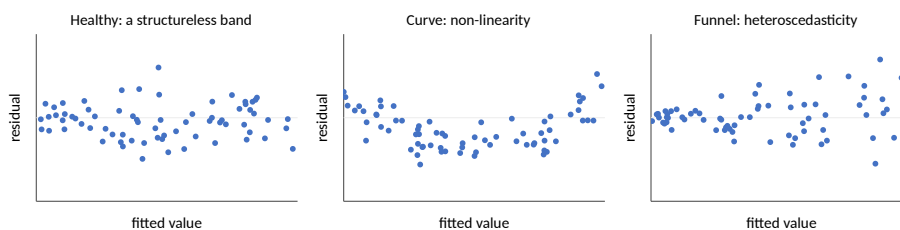
R_j^2 is the R^2 of regressing x_j on all the other features. VIF above 5–10 signals a problem. Remedies: drop or combine features, or use Ridge.

$$\text{Var}(\hat{\theta}_1) = \frac{\sigma^2}{m} \cdot \frac{1}{1 - r^2} = \frac{\sigma^2}{m} \text{VIF}_1$$

Here $R_1^2 = r^2$. For $r = 0.99$ the variance is $\frac{1}{1 - 0.9801} \approx 50$ times that of independent features; the covariance $-r\sigma^2/(m(1 - r^2))$ is negative: the streak runs along $\theta_1 + \theta_2 = \text{const}$.

Assumptions – Linear regression

Assumption	Formal	If violated	How to check
Linearity	$\mathbb{E}[y \mathbf{x}] = \boldsymbol{\theta}^\top \mathbf{x}$	biased predictions	residuals vs fitted show a curve
Exogeneity	$\mathbb{E}[\varepsilon \mathbf{x}] = 0$	biased, not causal	domain knowledge
Independence	$\varepsilon^{(i)}$ independent	over-confident standard errors	residuals vs time
Homoscedasticity	$\text{Var}(\varepsilon \mathbf{x}) = \sigma^2$	wrong standard errors	residuals fan out
Normal errors	$\varepsilon \sim \mathcal{N}(0, \sigma^2)$	only exact tests	Q-Q plot
No perfect collinearity	independent columns	not unique	rank, VIF



Gauss–Markov: under 1–4 and 6, OLS is the Best Linear Unbiased Estimator (BLUE).

Assumptions – Logistic regression

Assumption	Notes and check
Bernoulli outcome	multiclass $\rightarrow$ softmax
Independent observations	repeated measures break it: grouped CV
Linear log-odds	bin a feature, plot $\log \frac{\hat{y}}{1-\hat{y}}$ per bin: roughly straight?
No perfect collinearity	VIF
No perfect separation	else $\ \boldsymbol{\theta}\ \rightarrow \infty$: use L2
Enough data	~ 10 – 20 events of the rarer class per feature

Not assumed:

- normally distributed features,
- a linear relation between $\mathbf{x}$ and p itself (it is S-shaped by design),
- equal variance: $p(1-p)$ is part of the model.

There is no error term, so no normality or homoscedasticity is needed.

Regularization – Summary

- Regularization = loss + $\lambda \times$ penalty; don't penalize θ_0 ; standardize first; choose λ by CV.
- **L2 (Ridge)**: closed form $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$, always unique; shrinks, never zeroes; Gaussian prior; weight decay in GD.
- **L1 (Lasso)**: soft-thresholding, exact zeros, feature selection; Laplace prior. **Elastic Net** for correlated features.
- Logistic regression: L2 guarantees a finite solution; scikit-learn uses $C = 1/\lambda$.
- Inputs: scale, encode (ordinal or one-hot with $K - 1$ dummies), transforms, watch collinearity (VIF) and leakage.
Practical guide: "what to do when..."

LAB 2

Polynomial regression, overfitting and regularisation

One noisy curve, polynomials of growing degree: watch a model overfit, then tame it with a penalty.

- 1 Underfit, good fit, overfit: polynomials of growing degree.
- 2 Train vs test error as the degree grows: the gap is what matters.
- 3 Ridge: train vs test error as λ grows.
- 4 Choose λ properly, with cross-validation inside a pipeline.
- 5 Ridge vs Lasso: where the coefficients go, and Lasso's exact zeros.
- 6 Exercises, with answers.

 [Open in Colab](#)

notebook lab 2 - Polynomial Regression and Regularisation.ipynb

you need polynomial features (chapter 02), cross-validation (chapter 01), Ridge, Lasso and choosing λ (this chapter)

runs on CPU; every cell runs in seconds

Next: decision trees (chapter 06), then Lab 4.

Machine Learning

Máster en Innovación Tecnológica (MINT)

Next: Decision Trees

imanolgranada@gmail.com



Machine Learning

Máster en Innovación Tecnológica (MINT)

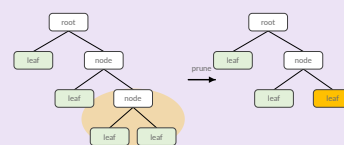
Decision Trees and CART

imanolgranada@gmail.com



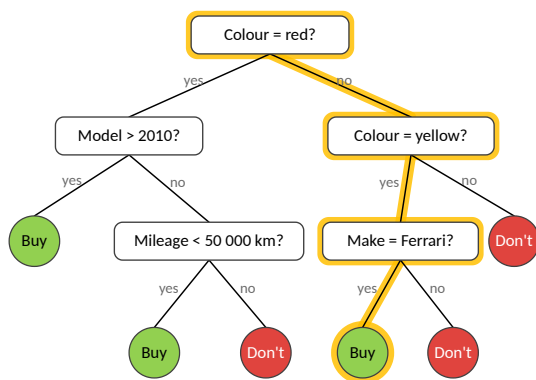
1

Decision Trees and CART



2

Decision Trees: an example



A yellow Ferrari: red? no → yellow? yes → Ferrari? yes → Buy

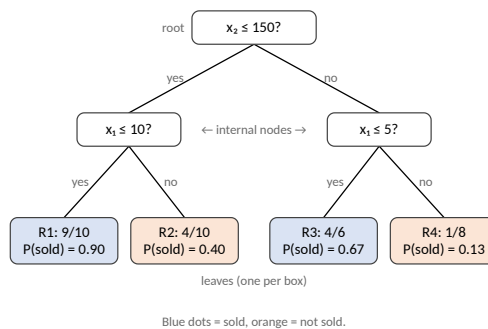
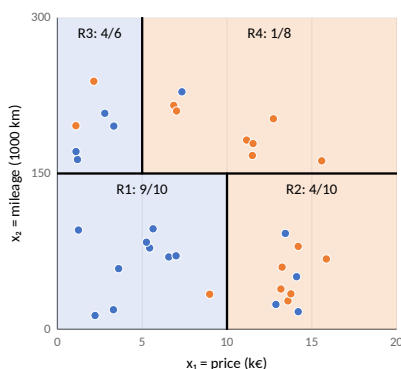
A tree predicts by asking a sequence of **yes/no questions**, each about **one feature**, until it reaches a **leaf** that holds the prediction.

Press → to follow one car down the tree.

CART (Classification And Regression Trees, Breiman et al. 1984) is the standard algorithm to **learn** the questions from data. It is what scikit-learn implements.

A small tree is just a set of if / else rules: easy to explain to anyone.

Decision Trees: a tree = boxes in feature space

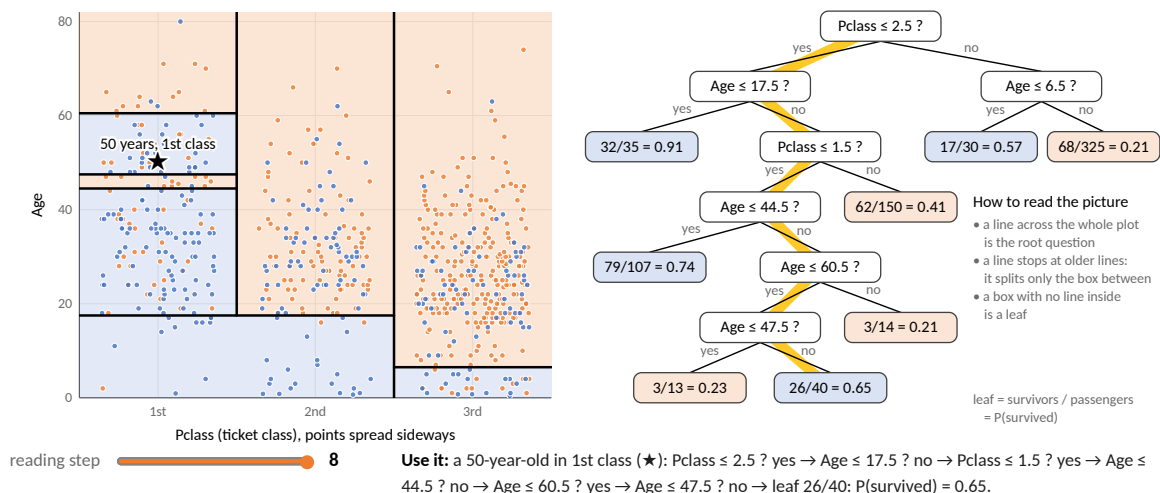


- **Root:** all the training data. **Internal node:** one question $x_j \leq s$ (feature j , threshold s). **Leaf:** no more questions.
- Each question cuts a box in two with a line **parallel to an axis**. The leaves are the final boxes $R_1, \dots, R_L$.

$$h(\mathbf{x}) = \sum_{\ell=1}^L c_{\ell} \mathbb{1}[\mathbf{x} \in R_{\ell}]$$

"Find the box that contains $\mathbf{x}$ and return its constant c_{ℓ} ." Classification: the class proportions of the box. Regression: the mean of its y .

Decision Trees: reading the tree off the boxes



Decision Trees: what a leaf predicts

A leaf predicts **one constant** c for all its examples. The best c depends on the loss:

loss on the leaf's examples	best constant c
squared error $\sum_i (y^{(i)} - c)^2$	the mean $\bar{y}$
absolute error $\sum_i y^{(i)} - c $	the median
0-1 loss (number of mistakes)	the majority class
log-loss	the class proportions $\hat{p}_k = N_k/N$

Examples. A regression leaf with $y = \{3, 5, 10\}$: mean 6, median 5. A classification leaf with 6 sold and 2 not sold: predicts "sold", with $\hat{p} = 6/8 = 0.75$. That $\hat{p}$ is what `predict_proba` returns.

Proof for the mean. Set the derivative with respect to c to zero:

$$\frac{d}{dc} \sum_{i=1}^N (y^{(i)} - c)^2 = \sum_{i=1}^N 2(y^{(i)} - c) \cdot (-1) = 0$$

chain rule: the derivative of $(y - c)^2$ with respect to c is $2(y - c) \cdot (-1)$

$$\sum_{i=1}^N y^{(i)} - Nc = 0 \Rightarrow c = \frac{1}{N} \sum_{i=1}^N y^{(i)} = \bar{y}$$

divide by -2 , and $\sum_{i=1}^N c = Nc$

Majority class. Predicting class k is wrong for a fraction $1 - p_k$ of the leaf: the fewest mistakes when p_k is the largest.

No gradient descent in trees: the **questions** are found by a greedy search, then each leaf simply takes its mean or its majority.

Decision Trees: which question is better?



Our running example: 16 used cars, 8 sold within a month and 8 not. Two candidate first questions.

Both send most sold cars to "yes". But B produces a **pure** child: 4 cars, none sold. Nothing more to ask there.

To compare questions we need a number, the **impurity** of a node: 0 if all its examples have the same class, largest when the classes are evenly mixed.

A question is good if its children are, on average, much **purier** than the parent. Next: entropy and Gini.

Decision Trees: entropy

With p_k = the fraction of class k in the node (K classes):

$$H = - \sum_{k=1}^K p_k \log_2 p_k = \sum_{k=1}^K p_k \cdot (-\log_2 p_k)$$

- **Pure** node: one $p_k = 1$ and $\log_2 1 = 0$, so $H = 0$ (by convention $0 \log_2 0 = 0$).
- **50/50** with two classes: $H = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 1 = 1$ bit, the maximum.
- Meaning: the average number of **bits** (yes/no answers) needed to tell the class of an example from the node.

The values of $-\log_2 p$ we need today:

p	1/2	1/4	3/4	1/3	2/3
$-\log_2 p$	1	2	0.415	1.585	0.585

The nodes of our example, written as (sold, not sold):

$$(8, 8): H = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 1 = 1$$

the root: $p = \frac{8}{16} = \frac{1}{2}$ for each class

$$(6, 2): H = \frac{3}{4} \cdot 0.415 + \frac{1}{4} \cdot 2 = 0.311 + 0.5 = 0.811$$

$p_{\text{sold}} = \frac{6}{8} = \frac{3}{4}, p_{\text{not}} = \frac{2}{8} = \frac{1}{4}$; the same for (2, 6)

$$(8, 4): H = \frac{2}{3} \cdot 0.585 + \frac{1}{3} \cdot 1.585 = 0.390 + 0.528 = 0.918$$

$p_{\text{sold}} = \frac{8}{12} = \frac{2}{3}, p_{\text{not}} = \frac{4}{12} = \frac{1}{3}$

$$(0, 4): H = 0 \quad \text{pure}$$

Decision Trees: information gain

The **information gain** of a question is the entropy before, minus the **size-weighted** entropy after:

$$IG = H(\text{parent}) - \sum_{\text{children } c} \frac{N_c}{N} H(c)$$

- N_c/N = the fraction of the parent's examples that go to child c .
- Weighting by size: a tiny pure child is not worth much.
- CART always asks yes/no questions: two children. (ID3, an older algorithm, splits a categorical feature into one child per value.)

The tree asks the question with the **largest gain**.

Question A (price): children (6, 2) and (2, 6), 8 cars each.

$$IG_A = 1 - \left(\frac{8}{16} \cdot 0.811 + \frac{8}{16} \cdot 0.811 \right) = 1 - 0.811 = 0.189$$

Question B (mileage): children (8, 4) with 12 cars and (0, 4) with 4.

$$IG_B = 1 - \left(\frac{12}{16} \cdot 0.918 + \frac{4}{16} \cdot 0 \right) = 1 - 0.689 = 0.311$$

$IG_B = 0.311 > IG_A = 0.189$: ask about the mileage first.

B wins even though its big child (8, 4) is more mixed than A's children (0.918 vs 0.811). The pure child is what pays off.

Decision Trees: Gini impurity

Draw two examples from the node at random, with replacement. **Gini** = the probability that their classes **differ**:

$$P(\text{same}) = \sum_{k=1}^K p_k \cdot p_k \Rightarrow \text{Gini} = 1 - \sum_{k=1}^K p_k^2$$

both draws are of class k with probability $p_k \cdot p_k$; add over the classes

$$\text{two classes: } 1 - p^2 - (1-p)^2 = 1 - p^2 - 1 + 2p - p^2 = 2p(1-p)$$

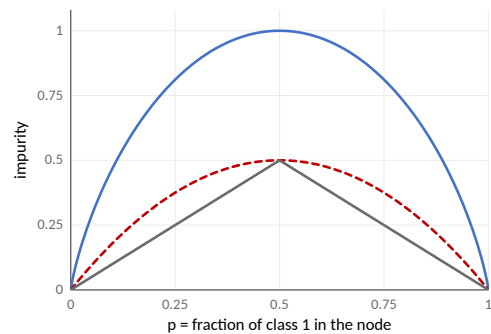
Our example. Root: $2 \cdot \frac{1}{2} \cdot \frac{1}{2} = 0.5$.

$$A: 2 \cdot \frac{3}{4} \cdot \frac{1}{4} = 0.375 \text{ (both children)} \Rightarrow \Delta = 0.5 - 0.375 = 0.125$$

$$B: (8, 4) \rightarrow 2 \cdot \frac{2}{3} \cdot \frac{1}{3} = 0.444, \quad (0, 4) \rightarrow 0$$

$$\Delta = 0.5 - \left(\frac{12}{16} \cdot 0.444 + \frac{4}{16} \cdot 0 \right) = 0.5 - 0.333 = 0.167$$

B wins again. Gini is the CART and scikit-learn default: no logarithm, and in practice almost the same trees as entropy.

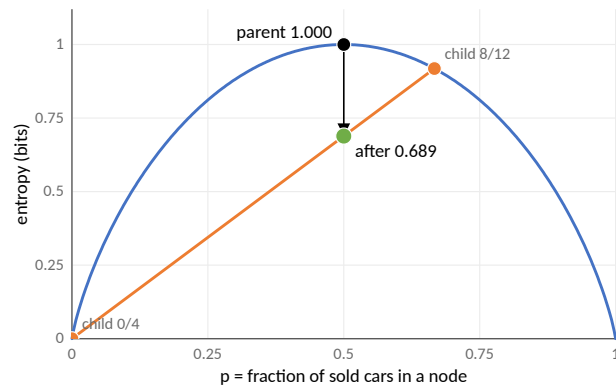


— entropy (max 1 bit) — Gini $2p(1-p)$ (max 0.5)

— misclassification $\min(p, 1-p)$

All three are 0 for a pure node and largest for a 50/50 node. The third one, the **misclassification error** $1 - \max_k p_k$, looks like the natural choice. Next slide: why trees don't use it.

Decision Trees: why not the misclassification error?



Misclassification Gini Entropy Question A Question B

parent	after (weighted)	gain
1.000	0.689	0.311

► Entropy, question B: gain 0.311 (A: 0.189). The curve rewards the pure child

Misclassification of a node = the fraction in the minority class, $1 - \max_k p_k$.

A: $\frac{1}{2} \cdot \frac{1}{4} + \frac{1}{2} \cdot \frac{1}{4} = 0.25$. B: $\frac{12}{16} \cdot \frac{1}{3} + \frac{4}{16} \cdot 0 = 0.25$. **A tie:** both questions leave exactly 4 cars in a minority (2 + 2, or 4 + 0).

The weighted impurity after a split is a point on the **chord** between the two children, because the parent's p is the weighted average of theirs.

Gini and **entropy** are curved (strictly concave): the chord lies below the curve, and the more extreme the children, the lower it gets. Pure children are rewarded.

The misclassification error is made of straight pieces, so many useful splits don't change it at all. It is still fine for **measuring** the final tree.

Decision Trees: searching the threshold

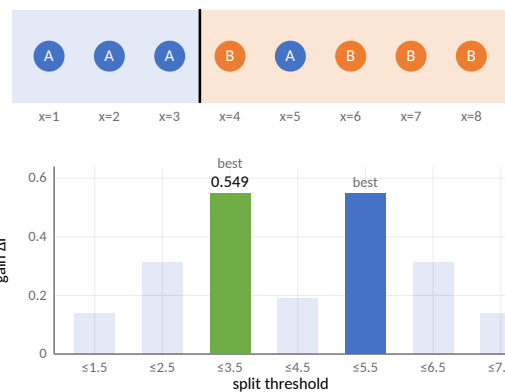
For a numeric feature the question is $x_j \leq s$. Which thresholds s do we try?

$$\Delta I = I(\text{parent}) - \left(\frac{N_L}{N} I(L) + \frac{N_R}{N} I(R) \right)$$

The same formula for any impurity I . With entropy, ΔI is the information gain.

Only **midpoints between consecutive sorted values** change the partition: 8 distinct values give only 7 candidate thresholds. Sort the feature once, and all of them are scored in one pass.

By hand, 8 points with root 4 A / 4 B (Gini 0.5), threshold 3.5: left (A,A,A) has Gini 0; right (B,A,B,B,B) has $1 - 0.2^2 - 0.8^2 = 0.32$. Weighted: $\frac{3}{8} \cdot 0 + \frac{5}{8} \cdot 0.32 = 0.20$, so the gain is $0.5 - 0.20 = 0.30$.



Threshold 3.5 Gini Entropy

root	left	right	weighted	gain ΔI
1.000	0.000	0.722	0.451	0.549

► Entropy instead of Gini: other numbers (0.549), the same two winners

Decision Trees: the CART algorithm

Start at the root with the whole training set, then at every node:

1. **Stop?** If the node is pure, too small or at `max_depth`, make it a **leaf**: mean, majority class or class proportions.
2. **Search:** for every feature j and every candidate threshold s , compute the gain $\Delta I(j, s)$.
3. **Split:** keep the best (j^*, s^*) . Examples with $x_{j^*} \leq s^*$ go left, the rest go right.
4. **Recurse:** repeat 1–3 on each child, independently.

Predict: drop x down the tree, one question per level, until it reaches a leaf.

```
grow(S, depth):
    if stop(S, depth): # pure, too small, max_depth
        return Leaf(value(S)) # mean / majority / proportions
    for each feature j, threshold s:
        score ΔI(j, s)
    (j*, s*) = argmax ΔI
    return Node(j*, s*,
        grow(S_left, depth + 1),
        grow(S_right, depth + 1))
```

Greedy: each split is the best one-step choice and is never revisited. Finding the optimal tree is NP-hard; the greedy tree is fast and works well. [A worked example](#).

Decision Trees: regression trees

Same algorithm. The leaf predicts the **mean**, and the impurity is the **variance** of y in the node:

$$I_V(S) = \frac{1}{N} \sum_{i \in S} (y^{(i)} - \bar{y}_S)^2 \Rightarrow N I_V(S) = \text{SSE}(S)$$

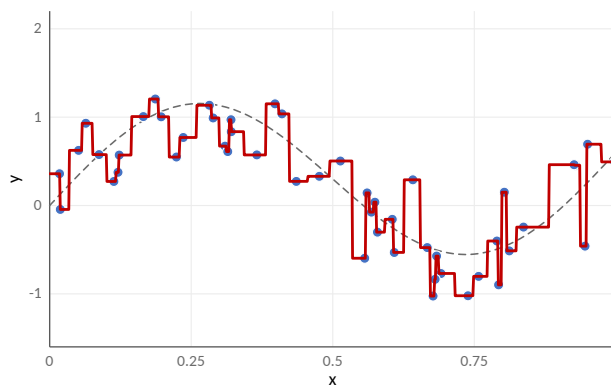
$\text{SSE}(S)$ = the squared error of predicting the mean on S .
Multiply the gain by N :

$$N \Delta I_V = \text{SSE}(\text{parent}) - [\text{SSE}(L) + \text{SSE}(R)]$$

So the best split is the one that **reduces the squared error the most**.

By hand: $y = \{1, 2, 6, 7\}$, mean 4: SSE
 $= 9 + 4 + 4 + 9 = 26$. Split into $\{1, 2\}$ and $\{6, 7\}$: means 1.5 and 6.5, SSE = $0.5 + 0.5 = 1$. The split removes 25 of the 26.

The prediction is a **step function**. Each extra level can double the number of steps.



► Depth 10: one leaf per training point. Train MSE 0, test MSE worse than depth 1

Decision Trees: depth and overfitting



CART (Gini) on 160 points. The true boundary is the dashed wave, and 12 % of the labels are flipped. Test accuracy on 1000 new points.

leaves	train acc.	test acc.
31	1.000	0.720

- Grown until every leaf is pure, a tree reaches **zero training error**: with at most $m - 1$ splits it can isolate each of the m training points.
- Low bias, **very high variance**.
- Boundaries are staircases of axis-aligned segments.
- Depth 2 predicts the same as depth 1 here: its splits change no majority, but they make depth 3 possible. A greedy tree can need splits that look useless on their own.

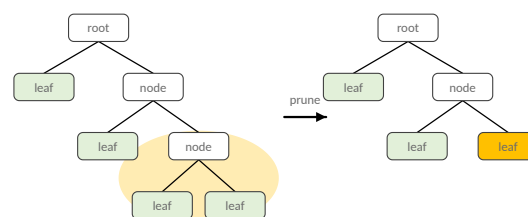
Decision Trees: regularization

1 • Stop early (pre-pruning). The scikit-learn hyperparameters:

- `max_depth`: at most this many questions on any path (typically 3 to 10).
- `min_samples_leaf`, `min_samples_split`: no tiny nodes.
- `min_impurity_decrease`: skip splits with a small gain.
- `max_leaf_nodes`.

Simple, but short-sighted: a split that looks useless may enable very good ones below it (XOR is the classic case).

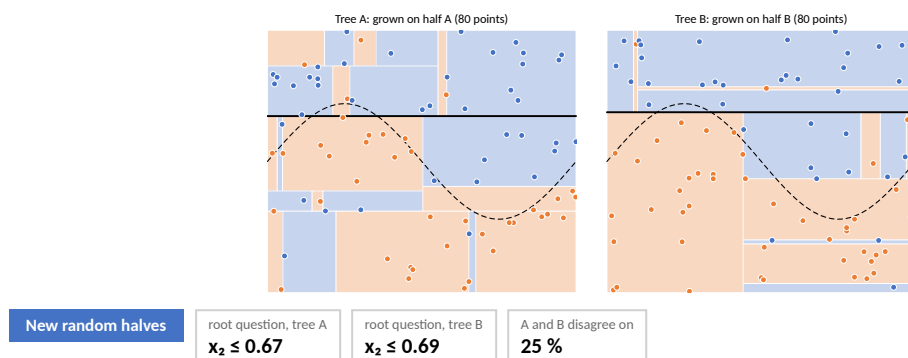
2 • Grow, then prune (post-pruning): let the tree grow, then collapse the branches that do not pay for themselves.



A branch is worth keeping only if it lowers the error enough to justify its extra leaves. How to decide, precisely (**cost-complexity pruning**), is in the [notes](#).

Decision Trees: unstable (high variance)

Split the same 160 points at random into two halves, and grow a full tree on each half. Same algorithm, same kind of data. Thick line: the root question; dashed: the true boundary.



► And another one: every sample of the data gives a quite different tree

Roughly a quarter of the plane changes class just because of which 80 points each tree saw. Small changes in the data → big changes in the model: that is **high variance**. The fix is to average many trees (next topic).

Decision Trees: pros and cons

Advantages

- Easy to read and to explain: a small tree is a set of if / else rules.
- **No feature scaling**: only the order of the values matters, so any increasing transformation of a feature gives the same tree.
- Numeric and (encoded) categorical features; non-linear effects and interactions are captured automatically.
- Fast to train, and very fast to predict: one question per level.
- Non-parametric: no assumption about the shape of the boundary.

Disadvantages

- **Unstable** (high variance): a small change in the data can change the top split, and with it the whole tree.
- Very prone to **overfitting**: with $m - 1$ splits it can isolate every training point.
- Axis-aligned and piecewise constant: a diagonal boundary needs a long staircase.
- Can't extrapolate: outside the training range it predicts the constant of the last leaf.
- Greedy: not the best possible tree.

The fix for the variance: average many trees (next topic).

LAB 4

Decision trees from scratch: how a split is chosen

Growing a tree repeats one decision: which question should this node ask? Every step by hand, with a picture. No tree class here: Lab 4b uses scikit-learn's.

- 1 What a split is, on one feature.
- 2 How mixed is a node? Gini and entropy, side by side.
- 3 Scoring a split: information gain, and why the children are weighted by their size.
- 4 Try every threshold, then every feature: the best question.
- 5 Regression: splits that reduce the MSE.
- 6 Recursion: do it again on each side; exercises.

[CO Open in Colab](#)

notebook Lab 4 - Decision Trees from scratch.ipynb
you need this chapter: entropy, Gini, information gain, the threshold search, regression trees
runs on CPU; every cell runs in seconds

Next: random forests (chapter 07), then Lab 4b with scikit-learn's trees.

Machine Learning

Máster en Innovación Tecnológica (MINT)

Next: Bagging, Random Forest and Boosting

imanolgranada@gmail.com

Machine Learning

Máster en Innovación Tecnológica (MINT)

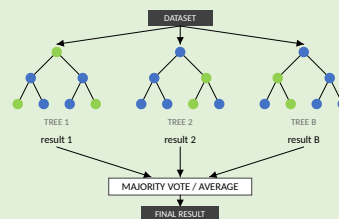
Bagging, Random Forest and Boosting

imanolgranada@gmail.com



1

Bagging, Random Forest & Boosting



2

Ensemble methods: the idea

Class exercise: how much does an Apple AirTag cost? Everyone writes a guess, **without talking** to anyone.

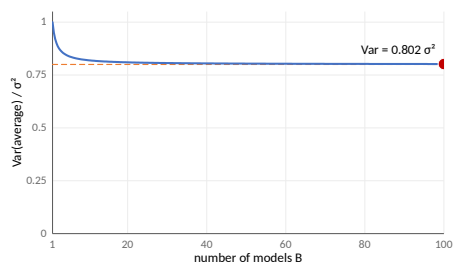
Each guess is noisy, but the **average** of the class is usually close to the real price (35 €).

Averaging B **independent** guesses, each with variance σ^2 , divides the variance by B :

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B h_b\right) = \frac{\sigma^2}{B}$$

If the guesses are **correlated** (people talk), with correlation ρ between any two:

$$\text{Var}(\bar{h}) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2$$



correlation ρ

0.80

models B

100

Var(average)
0.802 σ^2

floor $\rho \sigma^2$ (dashed)
0.80 σ^2

More models never hurt, but the floor $\rho \sigma^2$ stays. To go lower, make the models **less correlated**.

► $\rho = 0.8$: correlated models, averaging barely helps

Ensemble methods: why averaging works (proof)

B models at a fixed point $\mathbf{x}$. Each has variance $\text{Var}(h_b) = \sigma^2$, and any two have correlation ρ , i.e. covariance $\text{Cov}(h_b, h_{b'}) = \rho \sigma \cdot \sigma = \rho \sigma^2$.

1 • Variance of a sum = all the variances + the covariances of all pairs:

$$\text{Var}\left(\sum_{b=1}^B h_b\right) = \sum_b \text{Var}(h_b) + \sum_{b \neq b'} \text{Cov}(h_b, h_{b'})$$

2 • Count the terms, in plain words. Squaring a sum multiplies **every term by every term**: $(a + b + c)(a + b + c)$ gives $3 \times 3 = 9$ products. Three are a term with itself (a^2, b^2, c^2), six are two different terms (ab, ba, ac, ca, bc, cb). The variance of a sum works the same way, with covariances. Here $B = 3$ models, one cell for each "model $\times$ model":

	h_1	h_2	h_3
h_1	σ^2	$\rho \sigma^2$	$\rho \sigma^2$
h_2	$\rho \sigma^2$	σ^2	$\rho \sigma^2$
h_3	$\rho \sigma^2$	$\rho \sigma^2$	σ^2

Blue diagonal: a model with itself, its variance σ^2 . B cells.

Orange: two different models, their covariance $\rho \sigma^2$. The table has $B \cdot B$ cells, so $B \cdot B - B = B(B - 1)$ of them: $3 \cdot 2 = 6$.

The variance of the sum is the sum of the whole table: B blue cells and $B(B - 1)$ orange cells. For $B = 10$: 10 blue and $10 \cdot 9 = 90$ orange.

$$= B\sigma^2 + B(B - 1)\rho\sigma^2$$

3 • The average is the sum times $\frac{1}{B}$, and $\text{Var}(cX) = c^2 \text{Var}(X)$:

$$\text{Var}(\bar{h}) = \frac{B\sigma^2 + B(B - 1)\rho\sigma^2}{B^2} = \frac{\sigma^2}{B} + \frac{B - 1}{B} \rho\sigma^2$$

4 • Rearrange, using $\frac{B-1}{B} = 1 - \frac{1}{B}$:

$$= \frac{\sigma^2}{B} + \rho\sigma^2 - \frac{\rho\sigma^2}{B} = \rho\sigma^2 + \frac{1 - \rho}{B} \sigma^2 \quad \blacksquare$$

Numbers: $\sigma^2 = 1, \rho = 0.3$. One model: 1. $B = 10$: $0.3 + 0.07 = 0.37$. $B = 100$: 0.307 . $B \rightarrow \infty$: 0.3 .

- Averaging does not change the **bias** (the average of the models has the same expected value as one model), so average **low-bias** models: deep trees.
- As $B \rightarrow \infty$ the second term vanishes: more trees never overfit, the error just levels off.
- The floor $\rho \sigma^2$ only goes down if the models are **less alike**.

Ensemble methods: the bootstrap

Original training set, $m = 10$:



One bootstrap sample, drawn **with replacement** ($\times n$ = drawn n times, dashed = never drawn):



Draw a new sample

Draw 100 more

samples
103

left out, this one
3 of 10

left out, average
34.6 %

► 100 more: the left-out fraction settles near 35 %

We have **one** training set of m examples. To get "different" ones, draw m examples **with replacement**: a **bootstrap sample**. How likely is an example to be **left out**? Follow example #1.

One draw. We pick one of the m examples at random. It is #1 with probability $\frac{1}{m}$, so it is **not** #1 with probability $1 - \frac{1}{m}$. With $m = 10$: $1 - \frac{1}{10} = \frac{9}{10}$.

All the draws. The example goes back after each draw, so the m draws are independent, each with the same chance of missing #1. To be left out, #1 must be missed **every time**: multiply the m chances.

$$P(\text{left out}) = \underbrace{\left(1 - \frac{1}{m}\right) \cdots \left(1 - \frac{1}{m}\right)}_{m \text{ factors}} = \left(1 - \frac{1}{m}\right)^m$$

$m = 10$: $0.9 \times 0.9 \times \dots$ (ten times) $= 0.9^{10} = 0.349$, about 35 %.

m	10	100	1000
$(1 - 1/m)^m$	0.349	0.366	0.368

For larger m it settles at $e^{-1} \approx 0.368$ (proof in the notes). A bootstrap sample holds about **63 %** of the distinct examples (65 % for $m = 10$); the other ~37 % are its **out-of-bag** (OOB) examples.

Ensemble methods: out-of-bag (OOB) error

6 training examples, 5 trees. Each cell says how many times the example is in that tree's bootstrap sample. **0 = out-of-bag**: that tree never saw it.

example	tree 1	tree 2	tree 3	tree 4	tree 5	OOB trees
1	1	0	3	1	0	2, 5
2	1	0	1	2	2	2
3	0	3	2	2	0	1, 5
4	2	1	0	0	3	3, 4
5	0	1	0	1	1	1, 3
6	2	1	0	0	0	3, 4, 5

Each column adds up to 6: every tree is trained on 6 draws.

For every training example:

1. take only the trees that did **not** see it (its OOB trees),
2. average their predictions for it (example 4: trees 3 and 4),
3. compare with its true label.

The average error over all examples is the **OOB error**. Every prediction is made by trees that did not train on that example, so it estimates the **test error**, like cross-validation, but for free: no extra training and no held-out set.

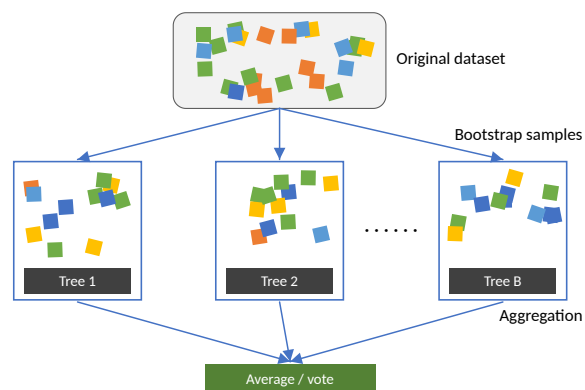
With many trees, each example is OOB for about 37 % of them. In scikit-learn: `RandomForestClassifier(oob_score=True)`, then read `oob_score_`.

Ensemble methods: bagging

Bootstrap aggregating (Breiman, 1996):

```
for b = 1, ..., B:
    draw a bootstrap sample D_b (m examples)
    fit a deep tree h_b on D_b      # no pruning
predict:
    regression:    average of the h_b(x)
    classification: majority vote
                    (or average the probabilities)
```

It helps any **unstable** model (small changes in the data → big changes in the model): deep trees are the classic case. A stable model such as linear regression gains almost nothing.



Ensemble methods: random forest

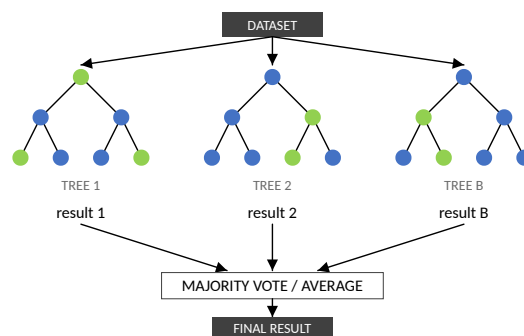
Random forest (Breiman, 2001) = bagging + one more source of randomness:

1. Draw B bootstrap samples of the training set.
2. On each one grow a **deep** tree, but at **every split** only a random subset of `max_features` features may be used (a new subset at each node).
3. Combine the B trees: majority vote or average of the probabilities (classification), average (regression).

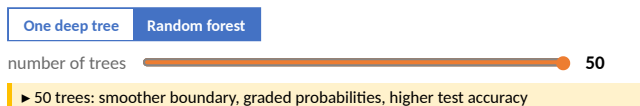
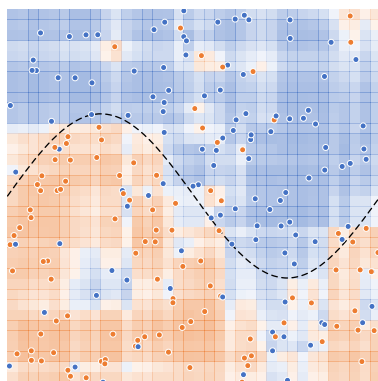
Why the random features? Bagged trees are still **correlated**: if one feature is very strong, almost every tree puts it at the top, the trees look alike, and ρ stays high.

Random features make the trees less alike: lower ρ , lower floor $\rho\sigma^2$. Each tree gets a little worse, but the average gets better.

Typical `max_features`, with n features: $\sqrt{n}$ for classification, $n/3$ for regression (scikit-learn's default there is all n). Using all n features is plain bagging.



Random forest: one tree vs a forest



220 noisy points (15 % of the labels flipped). Deep trees on bootstrap samples, 1 random feature (of 2) per split. Accuracy on 1000 new points:

one deep tree
0.627

forest of B trees
0.746

- The single tree carves tiny regions around individual noisy points.
- The forest averages those accidents away: a smoother boundary and graded probabilities (pale colour = unsure).
- The best possible accuracy here is 0.85: 15 % of the labels are noise.

Random forest: review

Pros

- Much less overfitting than a single tree; very good accuracy with little tuning.
- No feature scaling; handles non-linear effects and interactions.
- Robust to outliers in the inputs (trees only use the order of the values).
- Trees are independent: training runs in parallel. The OOB error comes for free.

Cons

- Less interpretable than one tree; bigger and slower to predict (B trees).
- Can't extrapolate beyond the training range.
- Less suited to very sparse, high-dimensional data (e.g. text), where linear models do well.

Hyperparameter	Effect
<code>n_estimators</code> (B)	more = lower variance, never overfits; stop when the OOB error is flat
<code>max_features</code>	lower = less correlated but weaker trees ($\sqrt{n}$ for classification)
<code>max_depth</code> , <code>min_samples_leaf</code>	complexity of each tree (usually fully grown)

Feature importance. MDI (`feature_importances_`): add up the impurity decreases of all the splits on each feature, averaged over the trees. Free, but computed on training data and biased towards features with many distinct values.

Permutation importance: shuffle one feature in the validation data and measure how much the score drops. More trustworthy.

LAB 4b

Decision trees and random forests

Real trees with scikit-learn: see why one tree overfits and wobbles, and how a forest fixes it.

- 1 How a tree splits the data, and how depth controls overfitting.
- 2 A single tree is unstable.
- 3 Bagging and random forests; the out-of-bag score and the number of trees.
- 4 Random-forest regression, and what happens outside the training range.
- 5 Feature importance.

[CO Open in Colab](#)

notebook `lab 4b - Decision Trees and Random Forests.ipynb`

you need decision trees (chapter 06, Lab 4), bootstrap, bagging, OOB and random forests (this chapter)

runs on CPU; every cell runs in seconds

Next: boosting, then Lab 6 (XGBoost).

Boosting: the idea

Bagging: many strong (deep) models trained **independently**, in parallel. The average lowers the **variance**.

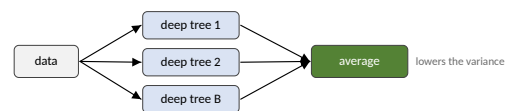
Boosting: many **weak** models (shallow trees) trained **one after another**. Each one focuses on what the ensemble so far gets wrong. It lowers the **bias**.

The result is an **additive model**: a weighted sum of M weak learners,

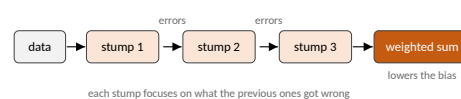
$$F_M(\mathbf{x}) = \sum_{t=1}^M \alpha_t h_t(\mathbf{x})$$

Two ways to "focus on the mistakes": **AdaBoost** re-weights the training examples; **gradient boosting** fits each new tree to the remaining errors (the residuals).

Bagging / random forest: independent models, in parallel

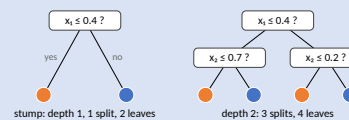


Boosting: weak models, one after another



each stump focuses on what the previous ones got wrong

Shallow tree = a tree with only a few levels of questions. The shallowest is a **stump**: depth 1, a **single split** (one question, two leaves).



AdaBoost usually uses stumps; gradient boosting, depth 2 to 6 (default 3 in scikit-learn, 6 in XGBoost).

AdaBoost: the algorithm, in words

AdaBoost combines **stumps**: trees with a single question. Every training point carries a **weight**, how much it matters. At the start all weights are equal.

Round after round:

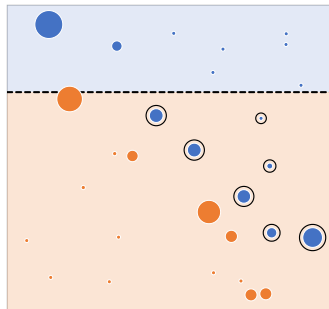
1. **Train a stump** on the weighted data. Heavy points are the ones it must get right.
2. **Count its mistakes**: add up the weights of the points it got wrong. This is its **weighted error** ϵ .
3. **Give it a vote** α . The fewer its mistakes, the bigger its say. A stump no better than a coin flip gets no say at all.
4. **Re-weight the points**. The ones it got wrong become heavier, the ones it got right lighter. The next stump is forced to look at the hard cases.

Final answer: all the stumps vote, and a stump's vote counts as much as its say: the best stumps decide.

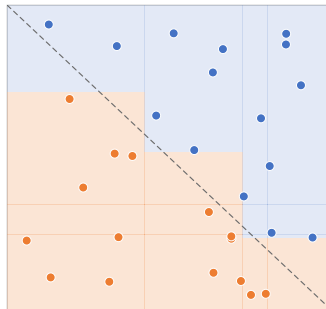
Next slide: the rounds one by one, with their numbers. The exact formulas for the vote and the new weights are in the notes.

AdaBoost in action

Round 20: stump 20 on the weighted data



The vote of stumps 1...20



Round 20. The points stump 19 got wrong are now heavier (bigger dots). Stump 20 cuts horizontally and gets **7** points wrong, holding $\epsilon = 0.285$ of the weight: vote $\alpha = 0.46$.

Right: no training errors left. Axis-aligned steps approximate the diagonal boundary (dashed).

new weights	7 mistakes	23 hits
each point	$\times 1.75$	$\times 0.7$
all together	29 % $\rightarrow$ 50 %	71 % $\rightarrow$ 50 %

Mistakes up, hits down: the mistakes always end up with half of the weight, so the next stump cannot ignore them.

round t  20

weighted error ϵ_t
0.285

vote α_t
0.46

stump t wrong on
7 of 30

errors of the vote
0

Gradient boosting: fit the residuals

Regression. Instead of re-weighting points, **each new tree learns what is still missing**: the error the current model still makes on every example.

1. Start with a boring prediction: the **average** of all the targets, the same for everybody.
2. For each example, compute the **residual**: real value minus current prediction. Positive: we predict too low. Negative: too high.
3. Train a **small tree** to predict those residuals from the features.
4. Add a **fraction** of the tree's answer to the current prediction (the fraction is the **learning rate** ν , for example $\frac{1}{2}$). Go back to 2.

Why only a fraction? Many small corrections generalize better than a few big ones.

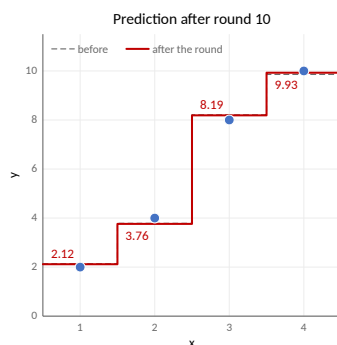
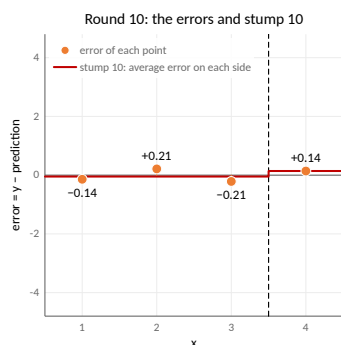
In symbols, with h_t the tree of round t :

$$F_0(\mathbf{x}) = \bar{y}, \quad F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \nu h_t(\mathbf{x})$$

After M rounds: $F_M(\mathbf{x}) = \bar{y} + \nu \sum_{t=1}^M h_t(\mathbf{x})$, the additive model of the boosting idea, with every weight equal to ν .

Next slide: 4 points, round by round.

Gradient boosting, round by round



Round 10. The errors before the round are $-0.14, +0.21, -0.21, +0.14$. The best stump splits at $x \leq 3.5$ and answers, on each side, the average error there: $(-0.14 + 0.21 - 0.21)/3 = -0.05$ on the left, $+0.14$ (one point) on the right.

Every prediction moves by $\frac{1}{2} \times$ the stump's answer: -0.02 on the left, $+0.07$ on the right. Squared error: **0.13 $\rightarrow$ 0.11**.

4 points, stumps, learning rate $\nu = \frac{1}{2}$. Each round removes part of the error that is left, never all of it: the errors shrink and the prediction creeps towards the points.

round t 10

stump t splits at
 $x \leq 3.5$

stump t says (left | right)
 -0.05 | $+0.14$

squared error: before $\rightarrow$ after
0.13 $\rightarrow$ 0.11

Gradient boosting: why "gradient"?

Remember **gradient descent**: to lower a loss, nudge what you can change a little in the direction where the loss goes down fastest.

Boosting does the same, but what it nudges is not a weight: it is the **prediction for every training example**. The loss says, for each example, in which direction its prediction should move.

Squared error: that direction is simply **real value – prediction**, the residual. So fitting the residuals **is** a gradient-descent step.

Why a tree? Nudging predictions directly only works on the training examples. A small tree learns the *pattern* of the nudges from the features, so the step also works for a **new x** . It is gradient descent on functions instead of on numbers.

Any loss works. Use the direction in which that loss goes down (the **pseudo-residual**) and fit the tree to it. Same recipe, another loss.

Classification (log-loss): the direction is **real label (0 or 1) – predicted probability**. The same "prediction minus target" as in logistic regression, with the sign flipped.

real label	predicted probability	direction
1	0.8	+0.2: push up a little
0	0.8	-0.8: push down hard

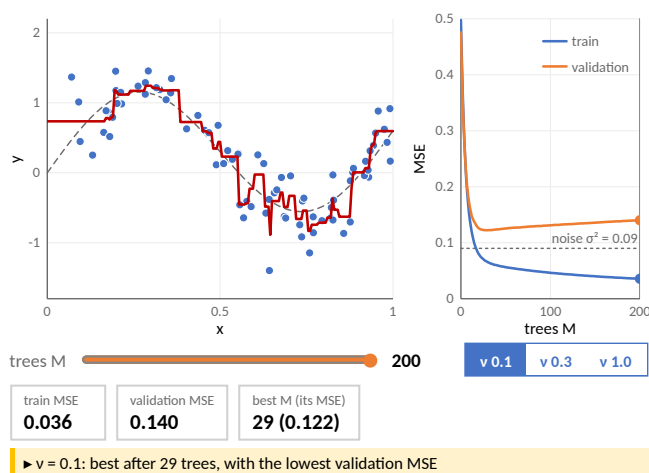
A confident mistake gets a big push, a good prediction a small one.

The starting guess is the best constant: the average of the targets (squared error) or the overall rate of positives (classification).

Gradient boosting in action

80 noisy points ($\sigma = 0.3$) around the dashed curve.
Trees of depth 2 with at least 5 points per leaf.

- **More trees**: the training error always falls; the validation error falls, then **rises**. Unlike a forest, boosting **does overfit**.
- Choose M by **early stopping**: stop when the validation error stops improving.
- **Smaller ν** : more trees are needed, but the validation error usually gets lower.
- No model beats the noise: the MSE on new data stays above $\sigma^2 = 0.09$ (dotted).



XGBoost

eXtreme Gradient Boosting: gradient boosting with **penalties on complex trees** and a very fast implementation.

- λ (`reg_lambda`) shrinks the leaf values (right). γ (`gamma`) is the minimum improvement a split must bring, or it is not made.
- It looks at the slope of the loss **and** at how curved it is (first and second derivatives): a smarter step, called a Newton step.
- Learning rate `eta`, row and column subsampling, early stopping, missing values, parallel split search.
- A top choice for **tabular** data, with LightGBM and CatBoost. For images, audio and text, deep learning wins.

The value of a leaf (squared loss). In plain gradient boosting a leaf answers the **average residual** of the examples that fall in it.

XGBoost penalizes big leaf values. The effect is easy to picture: **as if λ extra examples with residual 0 had fallen in the leaf**, and then we take the average:

$$\text{leaf value} = \frac{\text{sum of the residuals}}{\text{number of examples} + \lambda}$$

$\lambda = 0$: the plain average. $\lambda > 0$: pulled towards 0, much more for a leaf with few examples (little evidence) than for a big one.

Example: residuals $\{2, 4\}$ and $\lambda = 1$: $\frac{6}{2+1} = 2$ instead of 3. A leaf with 100 examples is barely touched: $\frac{100}{101} \approx 0.99$ of its average.

Summary: from one tree to boosting

Decision tree	Bagging	Random forest	Bagging / random forest		Boosting
a set of if / else rules	deep trees on bootstrap samples, averaged	bagging + random features at each split	trees	deep, independent	shallow, sequential
			mainly reduces	variance	bias
AdaBoost	Gradient boosting	XGBoost	combination	plain average / vote	weighted sum
stumps in sequence, re-weighting the mistakes	each tree fits the negative gradient of the loss	regularized, fast gradient boosting	more trees	never overfits, levels off	eventually overfits: early stopping
			tuning	little (good defaults)	ν , M , depth, subsampling

LAB 6

Gradient boosting and XGBoost

Boosting by hand on a toy curve, then XGBoost on three real datasets.

- 1 Gradient boosting from scratch: each tree fits the residuals of the previous ones.
- 2 XGBoost for regression (diabetes), binary (breast cancer) and multiclass (wine) problems.
- 3 Cross-validation and a randomized hyperparameter search.
- 4 Learning rate, number of trees and early stopping.
- 5 Feature importance.

Reference: [XGBoost documentation](#) · [Parameters](#) · [Python package](#)

 **Open in Colab**

notebook `Lab 6 - XGBoost and Gradient Boosting.ipynb`

you need gradient boosting and XGBoost (this chapter), cross-validation (chapter 01)

runs on CPU; every cell runs in seconds

Next: support vector machines (chapter 08), then Lab 5.

Machine Learning

Máster en Innovación Tecnológica (MINT)

Next: Support Vector Machines

imanolgranada@gmail.com

Machine Learning

Máster en Innovación Tecnológica (MINT)

Support Vector Machines

imanolgranada@gmail.com



1

**Support Vector
Machines**

$$\max \frac{2}{\|\mathbf{w}\|}$$



2

Support Vector Machine (SVM): examples

Facial expression classification

Happy, sad, surprised, angry: features from face images, one SVM per expression.

Breast cancer classification

Predicting whether a tumour is benign or malignant from several measurements.

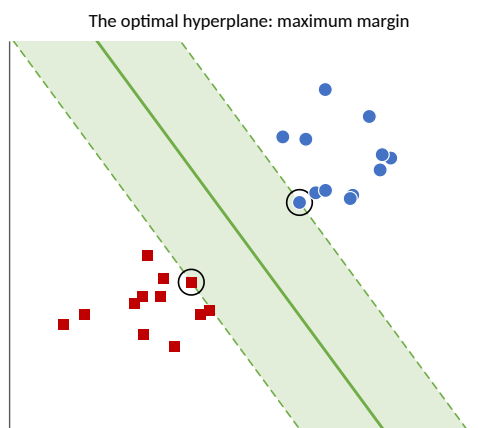
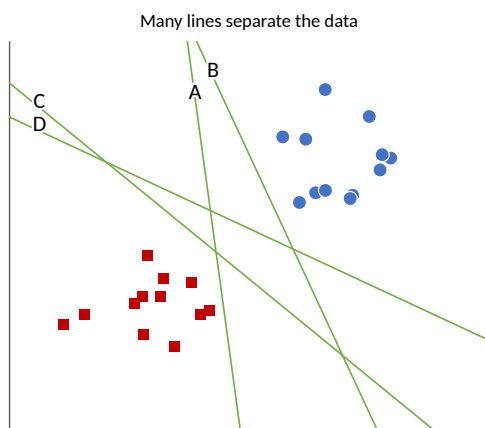
Clothing images

Fashion-MNIST: ankle boot, T-shirt, dress, pullover, sneaker, sandal...

A strong classifier for small and medium datasets, and the standard choice before deep learning took over images and text.

The plan. (1) Build the SVM from geometry: hyperplane, dot product, margin. (2) Solve it with Lagrange multipliers: the dual and the support vectors. (3) Kernels, for curved boundaries. (4) Soft margin: the parameter C , and how to use it in scikit-learn.

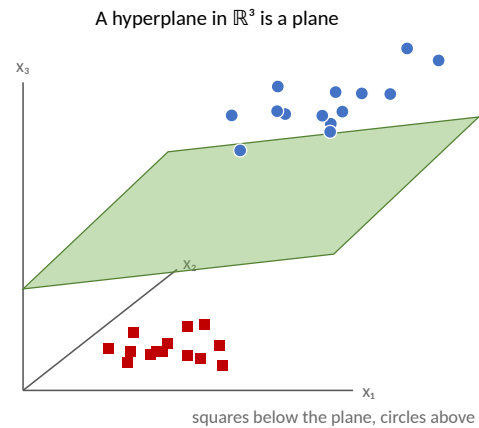
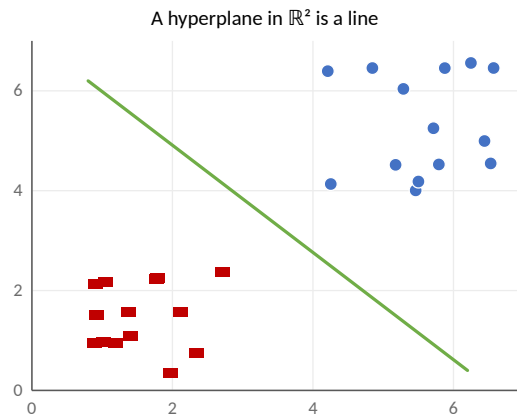
SVM: motivation



Many lines separate the data perfectly. **Which one would you pick?** A, B, C or D?

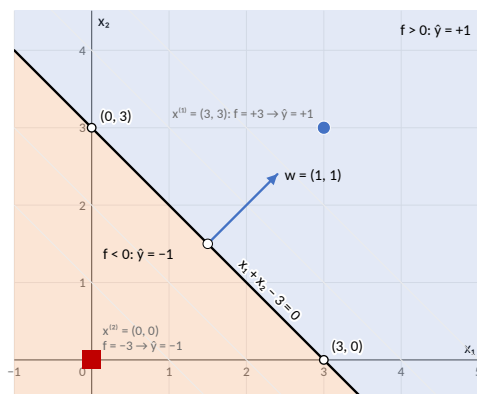
The SVM chooses the one that stays **as far as possible from the closest points** of both classes: the maximum margin.

SVM: hyperplanes



In $\mathbb{R}^n$ a hyperplane is an $(n - 1)$ -dimensional "flat" that cuts the space in two halves: a line in 2-D, a plane in 3-D, and so on.

SVM: the hyperplane and the decision rule



Input $\mathbf{x} \in \mathbb{R}^n$. Parameters: a weight vector $\mathbf{w} \in \mathbb{R}^n$ and a bias $b \in \mathbb{R}$. Labels are $y \in \{-1, +1\}$ (not 0/1: it makes the formulas symmetric).

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = w_1 x_1 + \dots + w_n x_n + b$$

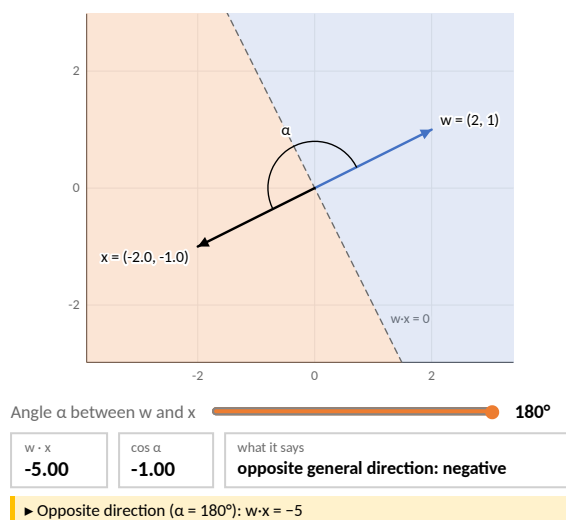
The **hyperplane** is the set of points with $f(\mathbf{x}) = 0$. In 2-D, $w_1 x_1 + w_2 x_2 + b = 0$: one equation, two unknowns, so one coordinate stays free and the solutions form a line.

$$\text{decision rule: } \hat{y} = \begin{cases} +1 & \text{if } f(\mathbf{x}) > 0 \\ -1 & \text{if } f(\mathbf{x}) < 0 \end{cases}$$

Example: $\mathbf{w} = (1, 1)$, $b = -3$, the line $x_1 + x_2 - 3 = 0$:

point	$f(\mathbf{x}) = x_1 + x_2 - 3$	prediction
$\mathbf{x}^{(1)} = (3, 3)$	+3	+1
$\mathbf{x}^{(2)} = (0, 0)$	-3	-1
(1.5, 1.5)	0	on the boundary

SVM: the dot product, what it means



The **dot product** of two vectors: multiply the matching coordinates and add.

$$\mathbf{w} \cdot \mathbf{x} = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

Example: $\mathbf{w} = (2, 1)$ and $\mathbf{x} = (1, 3)$ give $\mathbf{w} \cdot \mathbf{x} = 2 \cdot 1 + 1 \cdot 3 = 5$.

It is one number that says **how much two vectors point the same way**:

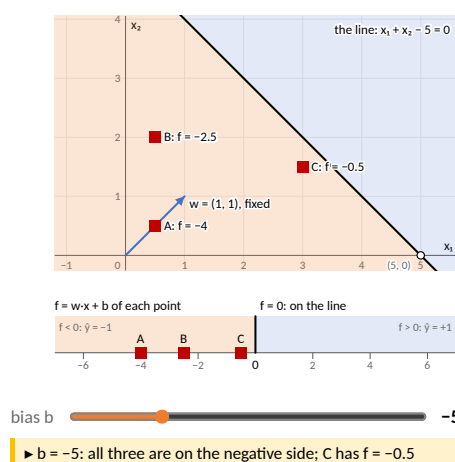
- **positive**: roughly the same direction. $(2, 1) \cdot (2, 1) = 5$.
- **zero**: perpendicular. $(2, 1) \cdot (-1, 2) = -2 + 2 = 0$.
- **negative**: roughly opposite directions. $(2, 1) \cdot (-2, -1) = -5$.

In one formula, with α the angle between the vectors and

$$\|\mathbf{w}\| = \sqrt{w_1^2 + \dots + w_n^2} \text{ the length of } \mathbf{w}:$$

$$\mathbf{w} \cdot \mathbf{x} = \|\mathbf{w}\| \|\mathbf{x}\| \cos \alpha. \text{ In the figure both lengths are } \sqrt{5}, \text{ so } \mathbf{w} \cdot \mathbf{x} = 5 \cos \alpha.$$

SVM: what b does



Keep $\mathbf{w} = (1, 1)$ fixed and move only b . Three points, $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$:

point	$w \cdot x$	$f = w \cdot x + b$	$\hat{y}$
A = (0.5, 0.5)	1	-4	-1
B = (0.5, 2)	2.5	-2.5	-1
C = (3, 1.5)	4.5	-0.5	-1

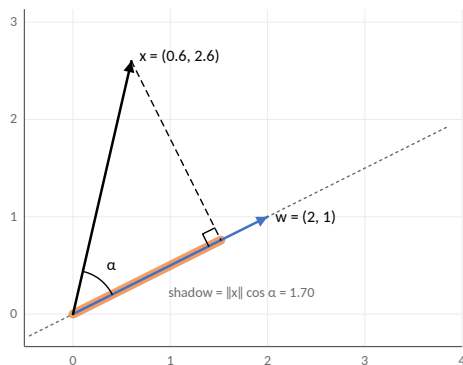
$\mathbf{w} \cdot \mathbf{x}$ does not depend on b : it is how far each point lies along $\mathbf{w}$ (previous slide). b adds **the same number** to all three, so on the number line under the plot they slide together.

A point changes class when its f crosses 0, that is, when the line passes over it: A at $b = -1$, B at $b = -2.5$, C at $b = -4.5$.

The line $x_1 + x_2 + b = 0$ cuts the axes at $(-b, 0)$ and $(0, -b)$: changing b moves it **parallel to itself**, it never turns.

$\mathbf{w}$ sets the direction of the boundary (perpendicular to $\mathbf{w}$); b sets where it sits.

SVM: the dot product is a shadow



Shine a light perpendicular to $\mathbf{w}$: the vector $\mathbf{x}$ casts a **shadow** on the line of $\mathbf{w}$ (orange).

Its length is $\|\mathbf{x}\| \cos \alpha$ (the side of the right triangle next to the angle).
Compare with $\mathbf{w} \cdot \mathbf{x} = \|\mathbf{w}\| \|\mathbf{x}\| \cos \alpha$:

$$\mathbf{w} \cdot \mathbf{x} = \|\mathbf{w}\| \times \text{shadow} \Rightarrow \text{shadow} = \frac{\mathbf{w} \cdot \mathbf{x}}{\|\mathbf{w}\|}$$

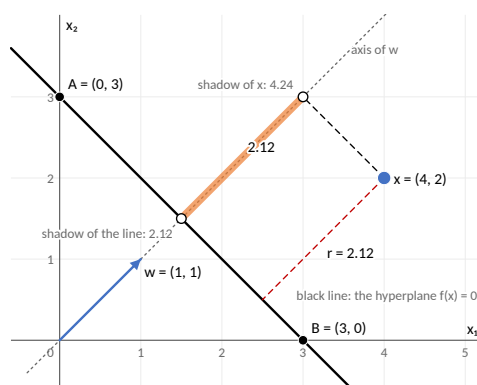
Check: $\mathbf{w} = (2, 1)$ and $\mathbf{x} = (0.6, 2.6)$. Then

$\mathbf{w} \cdot \mathbf{x} = 2 \cdot 0.6 + 1 \cdot 2.6 = 3.8$ and $\|\mathbf{w}\| = \sqrt{5} = 2.24$, so the shadow is $\frac{3.8}{2.24} = 1.70$, as in the figure.

If $\mathbf{x}$ points away from $\mathbf{w}$ the dot product is negative: the shadow falls on the **other side** of the origin.

Dividing by $\|\mathbf{w}\|$ turns the score $\mathbf{w} \cdot \mathbf{x}$ into a **length**. Next: the distance to a hyperplane.

SVM: the hyperplane is a wall of equal shadows



The hyperplane is where $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = 0$, that is, $\mathbf{w} \cdot \mathbf{x} = -b$. Dividing by $\|\mathbf{w}\|$, **every point of the hyperplane has the same shadow**:

$$\text{shadow}(\mathbf{x}) = \frac{\mathbf{w} \cdot \mathbf{x}}{\|\mathbf{w}\|} = \frac{-b}{\|\mathbf{w}\|}$$

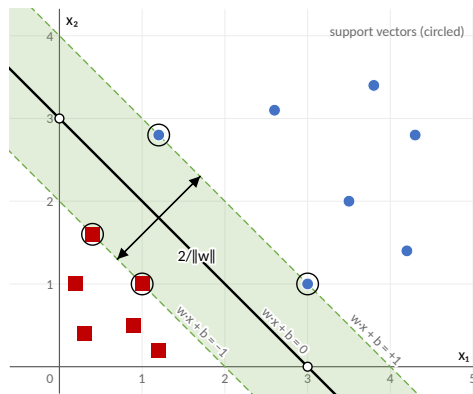
1 • Points A and B of the figure project to the same spot on the axis of $\mathbf{w}$. So the hyperplane is a straight wall **perpendicular to $\mathbf{w}$** .

2 • The distance from $\mathbf{x}$ to the wall is the difference of two shadows, the one of $\mathbf{x}$ and the one of the wall:

$$r(\mathbf{x}) = \frac{\mathbf{w} \cdot \mathbf{x}}{\|\mathbf{w}\|} - \frac{-b}{\|\mathbf{w}\|} = \frac{\mathbf{w} \cdot \mathbf{x} + b}{\|\mathbf{w}\|} = \frac{f(\mathbf{x})}{\|\mathbf{w}\|}$$

Check: $\mathbf{w} = (1, 1)$, $b = -3$, $\mathbf{x} = (4, 2)$. $f = 4 + 2 - 3 = 3$, so $r = \frac{3}{\sqrt{2}} = 2.12$. The closest point of the line is $(2.5, 0.5)$, at distance $\sqrt{1.5^2 + 1.5^2} = 2.12$ ✓. The sign of r tells the side.

SVM: fixing the scale



The length of $\mathbf{w}$ is free. $\mathbf{w} = (1, 1), b = -3$ and $\mathbf{w} = (2, 2), b = -6$ give the same line $x_1 + x_2 = 3$: only the values of f double. The distances $r = f / \|\mathbf{w}\|$ do not change.

We spend that freedom: choose the scale so that the **closest points** of each class have $f = +1$ and $f = -1$. Then every training point satisfies

$$\begin{cases} \mathbf{w} \cdot \mathbf{x}^{(i)} + b \geq +1 & \text{if } y^{(i)} = +1 \\ \mathbf{w} \cdot \mathbf{x}^{(i)} + b \leq -1 & \text{if } y^{(i)} = -1 \end{cases} \iff y^{(i)} (\mathbf{w} \cdot \mathbf{x}^{(i)} + b) \geq 1$$

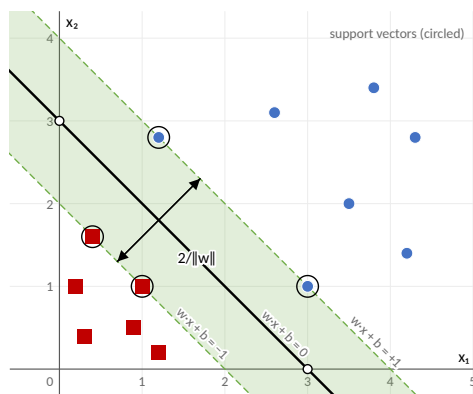
Multiplying by $y^{(i)} = \pm 1$ merges both cases into one line (for $y^{(i)} = -1$ it flips the $\leq$ into $\geq$).

Equality holds exactly for the closest points, the **support vectors** (circled):

$$y^{(i)} (\mathbf{w} \cdot \mathbf{x}^{(i)} + b) = 1$$

No training point lies strictly between the dashed lines $f = \pm 1$. That empty band is the **margin**.

SVM: measuring the margin



Take a support vector on each side: $\mathbf{x}_+$ with $f = +1$ and $\mathbf{x}_-$ with $f = -1$. The margin is the difference of their shadows on $\mathbf{w}$:

$$d = \frac{\mathbf{w} \cdot \mathbf{x}_+}{\|\mathbf{w}\|} - \frac{\mathbf{w} \cdot \mathbf{x}_-}{\|\mathbf{w}\|} = \frac{\mathbf{w} \cdot \mathbf{x}_+ - \mathbf{w} \cdot \mathbf{x}_-}{\|\mathbf{w}\|}$$

From the support-vector equations, $\mathbf{w} \cdot \mathbf{x}_+ + b = 1$ and $\mathbf{w} \cdot \mathbf{x}_- + b = -1$:

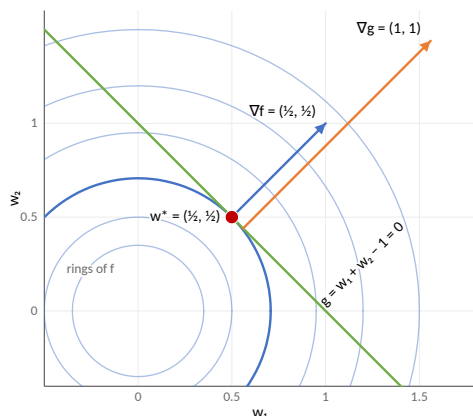
$$\begin{aligned} \mathbf{w} \cdot \mathbf{x}_+ &= 1 - b, & \mathbf{w} \cdot \mathbf{x}_- &= -1 - b \\ d &= \frac{(1 - b) - (-1 - b)}{\|\mathbf{w}\|} = \frac{1 - b + 1 + b}{\|\mathbf{w}\|} \end{aligned}$$

$$d = \frac{2}{\|\mathbf{w}\|}$$

Check on the figure: $\mathbf{w} = (1, 1), b = -3, \mathbf{x}_+ = (3, 1), \mathbf{x}_- = (1, 1)$:
 $d = \frac{4 - 2}{\sqrt{2}} = \sqrt{2} = \frac{2}{\sqrt{2}}$ ✓

b **cancels**. Once the scale is fixed, a **short $\mathbf{w}$** means a **wide margin**.

Review: Lagrange multipliers



Minimize $f(\mathbf{w})$ **subject to** a constraint $g(\mathbf{w}) = 0$. Example:
 $f = \frac{1}{2}(w_1^2 + w_2^2)$ and $g = w_1 + w_2 - 1$.

The picture. The blue rings join the points with the same f . The green line is where the constraint holds. The best allowed point is where a ring **just touches** the line, and there ∇f and ∇g point the same way: $\nabla f = \lambda \nabla g$.

The recipe. Build the **Lagrangian** with a multiplier λ , and set its derivatives to zero:

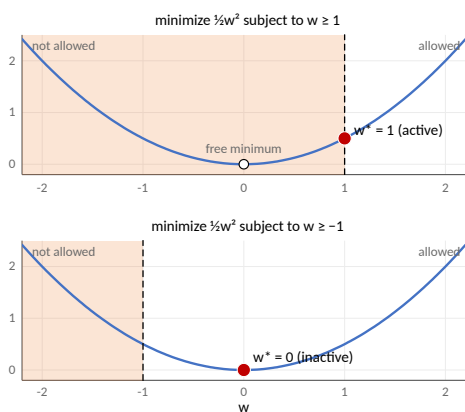
$$\mathcal{L}(\mathbf{w}, \lambda) = f(\mathbf{w}) - \lambda g(\mathbf{w})$$

The example. $\frac{\partial \mathcal{L}}{\partial w_1} = w_1 - \lambda = 0$ and $\frac{\partial \mathcal{L}}{\partial w_2} = w_2 - \lambda = 0$, so $w_1 = w_2 = \lambda$. The constraint $w_1 + w_2 = 1$ then gives $2\lambda = 1$:

$$\lambda = \frac{1}{2}, \quad \mathbf{w}^* = \left(\frac{1}{2}, \frac{1}{2}\right)$$

In the picture, $\nabla f = \mathbf{w}^* = (\frac{1}{2}, \frac{1}{2})$ and $\nabla g = (1, 1)$: $\nabla f = \frac{1}{2} \nabla g$ ✓.

Lagrange multipliers: an inequality, in pictures



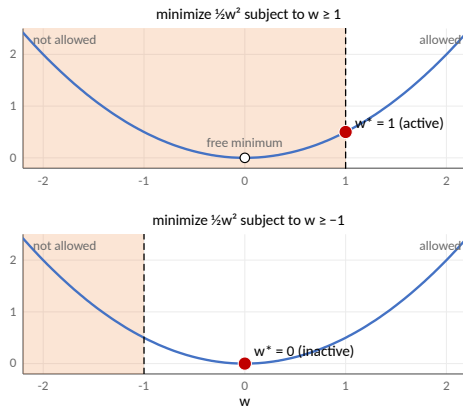
The SVM has **inequality** constraints (≥ 1), not equalities. One dimension first: minimize $\frac{1}{2}w^2$, a bowl with its bottom at $w = 0$.

Top: $w \geq 1$. The orange zone is forbidden, and it holds the bottom of the bowl. The best allowed point is the nearest edge, $w^* = 1$: the constraint is **active**, the solution is pressed against it.

Bottom: $w \geq -1$. The bottom of the bowl is allowed. $w^* = 0$, as if the constraint did not exist: **inactive**.

Each constraint $g(w) \geq 0$ gets a multiplier $\alpha \geq 0$. **Active:** the constraint pushes, $\alpha > 0$. **Inactive:** it does nothing, $\alpha = 0$. In both cases $\alpha \cdot g(w^*) = 0$. This is the rule that will pick the support vectors.

Lagrange multipliers: an inequality, the maths



Top problem: $\min_w \frac{1}{2}w^2$ subject to $g(w) = w - 1 \geq 0$.

1 • Lagrangian, with a multiplier $\alpha \geq 0$:

$$\mathcal{L}(w, \alpha) = \frac{1}{2}w^2 - \alpha(w - 1)$$

2 • Minimize over w (derivative to zero):

$$\frac{\partial \mathcal{L}}{\partial w} = w - \alpha = 0 \Rightarrow w = \alpha$$

3 • Substitute: the **dual**, a function of α only:

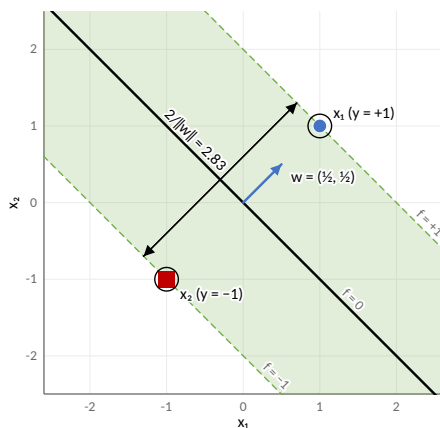
$$\frac{1}{2}\alpha^2 - \alpha(\alpha - 1) = \alpha - \frac{1}{2}\alpha^2$$

4 • Maximize over $\alpha \geq 0$: the derivative $1 - \alpha = 0$ gives $\alpha = 1$, so $w = \alpha = 1$ ✓. Active constraint, $\alpha > 0$.

Bottom problem ($w \geq -1$): the same steps give $w = \alpha$ and the dual $-\alpha - \frac{1}{2}\alpha^2$. It only decreases for $\alpha \geq 0$, so its maximum is at $\alpha = 0$ and $w = 0$ ✓. Inactive.

Both cases: $\alpha \cdot g(w^*) = 0$, **complementary slackness**.

SVM: two known support vectors, with the Lagrangian



1 • Lagrangian, one multiplier per point:

$$\mathcal{L} = \frac{1}{2}\|\mathbf{w}\|^2 - \alpha_1[y_1(\mathbf{w} \cdot \mathbf{x}_1 + b) - 1] - \alpha_2[y_2(\mathbf{w} \cdot \mathbf{x}_2 + b) - 1]$$

2 • $\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \alpha_1 y_1 \mathbf{x}_1 - \alpha_2 y_2 \mathbf{x}_2 = \mathbf{0}$. Since $y_1 \mathbf{x}_1 = y_2 \mathbf{x}_2 = (1, 1)$, we get

$$\mathbf{w} = \alpha_1 y_1 \mathbf{x}_1 + \alpha_2 y_2 \mathbf{x}_2 = (\alpha_1 + \alpha_2) (1, 1)$$

3 • $\frac{\partial \mathcal{L}}{\partial b} = -\alpha_1 y_1 - \alpha_2 y_2 = -\alpha_1 + \alpha_2 = 0$, so $\alpha_1 = \alpha_2 = \alpha$ and $\mathbf{w} = (2\alpha, 2\alpha)$.

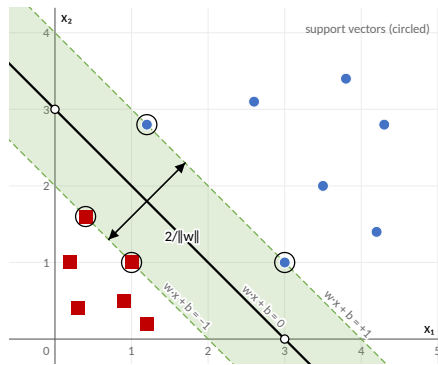
4 • Use the two equalities: $\mathbf{w} \cdot \mathbf{x}_1 + b = 4\alpha + b = 1$ and $\mathbf{w} \cdot \mathbf{x}_2 + b = -4\alpha + b = -1$. Adding them: $b = 0$, and then $\alpha = \frac{1}{4}$.

$$\mathbf{w} = \left(\frac{1}{2}, \frac{1}{2}\right), \quad b = 0, \quad \alpha_1 = \alpha_2 = \frac{1}{4}$$

Margin $\frac{2}{\|\mathbf{w}\|} = 2\sqrt{2} = 2.83$, the distance between the two points ✓. And $\mathbf{w} = \frac{1}{4}\mathbf{x}_1 - \frac{1}{4}\mathbf{x}_2$ is built from the two support vectors.

SVM: in general we do not know the support vectors

1 · Lagrangian 2 · minimize over $\mathbf{w}, b$ 3 · the dual 4 · solve for α 5 · $\mathbf{w}, b$, predict



With two points it was obvious. With 13 points, or 13,000, **which ones** end up on the margin? We cannot know before solving.

The fix: one multiplier $\alpha_i \geq 0$ **per point**, and let the maths decide. For a problem with constraints $g_i(\mathbf{w}) \geq 0$:

$$\mathcal{L}(\mathbf{w}, \alpha) = f(\mathbf{w}) - \sum_i \alpha_i g_i(\mathbf{w}), \quad \alpha_i \geq 0$$

Why it works. Fix $\mathbf{w}$ and make $\mathcal{L}$ as large as possible over the α_i :

- a constraint is violated ($g_i < 0$): then $-\alpha_i g_i$ grows without limit as α_i grows. An **infinite penalty**;
- all constraints hold: the best is $\alpha_i = 0$, and we get $f(\mathbf{w})$ back.

So minimizing over $\mathbf{w}$ the maximum over α is exactly the constrained problem. And at the optimum $\alpha_i g_i = 0$: **points off the margin get $\alpha_i = 0$** ; the others are the support vectors.

SVM: the Lagrangian

1 · Lagrangian 2 · minimize over $\mathbf{w}, b$ 3 · the dual 4 · solve for α 5 · $\mathbf{w}, b$, predict

For the SVM: $f = \frac{1}{2} \|\mathbf{w}\|^2$, and one constraint per training point, $g_i = y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \geq 0$.

Shorthand from now on: $\mathbf{x}_i, y_i$ is the i -th training example (so far $\mathbf{x}^{(i)}, y^{(i)}$), for $i = 1, \dots, m$.

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1]$$

The same expression as for the two points, with m terms instead of 2. The multipliers must satisfy $\alpha_i \geq 0$.

Expand it and group the terms:

$$\mathcal{L} = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \sum_i \alpha_i y_i \mathbf{w} \cdot \mathbf{x}_i - b \sum_i \alpha_i y_i + \sum_i \alpha_i$$

The plan, as in 1-D:

- minimize** over $\mathbf{w}$ and b : derivatives to zero;
- substitute** the result back: a function of α only, the **dual**;
- maximize** it over $\alpha_i \geq 0$.

For convex problems like this one, minimizing first and maximizing afterwards gives the same answer as the constrained problem.

SVM: minimizing over w and b

1 · Lagrangian

2 · minimize over w, b

3 · the dual

4 · solve for α 5 · w, b , predict

The expanded Lagrangian:

$$\mathcal{L} = \frac{1}{2} \mathbf{w} \cdot \mathbf{w} - \sum_i \alpha_i y_i \mathbf{w} \cdot \mathbf{x}_i - b \sum_i \alpha_i y_i + \sum_i \alpha_i$$

HINT

$\frac{\partial}{\partial \mathbf{w}} \left(\frac{1}{2} \mathbf{w} \cdot \mathbf{w} \right) = \mathbf{w}$ and $\frac{\partial}{\partial \mathbf{w}} (\mathbf{w} \cdot \mathbf{x}_i) = \mathbf{x}_i$, the vector versions of $\frac{d}{dw} \frac{1}{2} w^2 = w$ and $\frac{d}{dw} wx = x$.

With respect to w :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i = \mathbf{0}$$

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i$$

The optimal $\mathbf{w}$ is a **weighted sum of the training points**: the formula we found for the two points.

With respect to b (only the third term contains b):

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^m \alpha_i y_i = 0$$

$$\sum_{i=1}^m \alpha_i y_i = 0$$

The multipliers of the two classes balance: $\sum_{y_i=+1} \alpha_i = \sum_{y_i=-1} \alpha_i$.

SVM: the dual problem

1 · Lagrangian

2 · minimize over w, b

3 · the dual

4 · solve for α 5 · w, b , predict

Substitute both results into the Lagrangian. The b term is $b \cdot 0 = 0$, and the two $\mathbf{w}$ terms become the same double sum:

$$\begin{aligned} \frac{1}{2} \mathbf{w} \cdot \mathbf{w} &= \frac{1}{2} \left(\sum_i \alpha_i y_i \mathbf{x}_i \right) \cdot \left(\sum_j \alpha_j y_j \mathbf{x}_j \right) = \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \\ \sum_i \alpha_i y_i \mathbf{w} \cdot \mathbf{x}_i &= \sum_i \alpha_i y_i \left(\sum_j \alpha_j y_j \mathbf{x}_j \right) \cdot \mathbf{x}_i = \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \end{aligned}$$

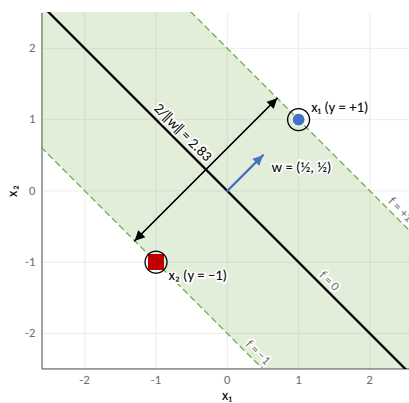
Half of the double sum, minus all of it, leaves **minus half**. The **dual problem**:

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \quad \text{s.t.} \quad \alpha_i \geq 0, \quad \sum_{i=1}^m \alpha_i y_i = 0$$

- A problem in m unknowns, one α_i per training point, with simple constraints. Solvers (SMO) handle it well.
- The data enter **only through the dot products** $\mathbf{x}_i \cdot \mathbf{x}_j$ (highlighted). More on this soon.

SVM: the same two points, through the dual

1 · Lagrangian 2 · minimize over $\mathbf{w}, b$ 3 · the dual **4 · solve for α** 5 · $\mathbf{w}, b$, predict



Same two points as before. The dual has one unknown per point, and we can solve it by hand.

1 • The constraint $\sum_i \alpha_i y_i = \alpha_1 - \alpha_2 = 0$ gives $\alpha_1 = \alpha_2 = \alpha$.

2 • Dot products: $\mathbf{x}_1 \cdot \mathbf{x}_1 = 2$, $\mathbf{x}_2 \cdot \mathbf{x}_2 = 2$, $\mathbf{x}_1 \cdot \mathbf{x}_2 = -2$. The double sum has 4 terms:

$$\sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j = \alpha^2 (2 + 2 + 2 \cdot (-1)(-2)) = 8\alpha^2$$

3 • Dual objective: $2\alpha - \frac{1}{2} \cdot 8\alpha^2 = 2\alpha - 4\alpha^2$. Its derivative $2 - 8\alpha = 0$ gives $\alpha = \frac{1}{4}$.

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i = \frac{1}{4}(1, 1) - \frac{1}{4}(-1, -1) = \left(\frac{1}{2}, \frac{1}{2}\right)$$

4 • b from a support vector: $b = y_1 - \mathbf{w} \cdot \mathbf{x}_1 = 1 - \left(\frac{1}{2} + \frac{1}{2}\right) = 0$.

Same answer as with the Lagrangian directly. Boundary $\frac{1}{2}x_1 + \frac{1}{2}x_2 = 0$, margin $\frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{1/2}} = 2\sqrt{2} = 2.83$, exactly the distance between the two points ✓. Both are support vectors ($\alpha > 0$).

SVM: support vectors and prediction

1 · Lagrangian 2 · minimize over $\mathbf{w}, b$ 3 · the dual 4 · solve for α **5 · $\mathbf{w}, b$, predict**

Complementary slackness (KKT), for every training point:

$$\alpha_i [y_i (\mathbf{w} \cdot \mathbf{x}_i + b) - 1] = 0$$

- A point outside the margin has a bracket > 0 , so $\alpha_i = 0$.
- If $\alpha_i > 0$ the bracket must be 0: the point is **on** the margin. These are the **support vectors** (a point on the margin can still have $\alpha_i = 0$).

So $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ is built from the support vectors alone. Delete any other point and nothing changes.

The bias, from any support vector s :

$$y_s (\mathbf{w} \cdot \mathbf{x}_s + b) = 1 \Rightarrow b = y_s - \mathbf{w} \cdot \mathbf{x}_s \text{ (because } 1/y_s = y_s \text{ when } y_s = \pm 1).$$

Prediction for a new $\mathbf{x}$: substitute $\mathbf{w}$ into $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$:

$$f(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b, \quad \hat{y} = \text{sign } f(\mathbf{x})$$

Again only dot products, now between the new point and the support vectors.

In the two-point example:

$$f(\mathbf{x}) = \frac{1}{4} \mathbf{x}_1 \cdot \mathbf{x} - \frac{1}{4} \mathbf{x}_2 \cdot \mathbf{x} + 0 = \frac{1}{2} x_1 + \frac{1}{2} x_2 \quad \checkmark$$

SVM: the data only enter through inner products

Training: the dual problem.

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0 \end{aligned}$$

Prediction for a new point $\mathbf{x}$:

$$f(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} + b$$

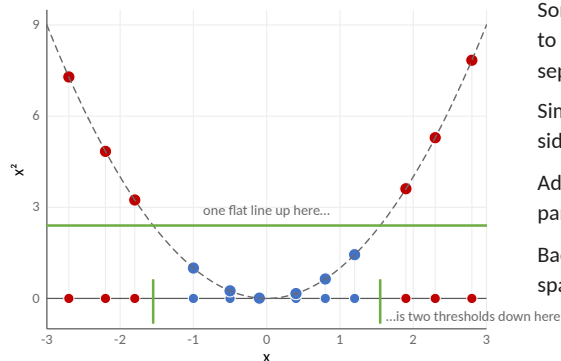
Look at where the data appear: only inside the highlighted **inner products** (dot products) of pairs of points.

Not the coordinates themselves, and not even how many features there are. Only the table of inner products between every pair of training points, the **Gram matrix**. For the two points of before:

	$\mathbf{x}_1$	$\mathbf{x}_2$
$\mathbf{x}_1$	$\mathbf{x}_1 \cdot \mathbf{x}_1 = 2$	$\mathbf{x}_1 \cdot \mathbf{x}_2 = -2$
$\mathbf{x}_2$	$\mathbf{x}_2 \cdot \mathbf{x}_1 = -2$	$\mathbf{x}_2 \cdot \mathbf{x}_2 = 2$

Give the algorithm that table and it can train and predict. If we replace the table by a different one, we get a different model, with **no change to the algorithm**. That is the door to **kernels**: the next topic.

SVM: kernels, the idea



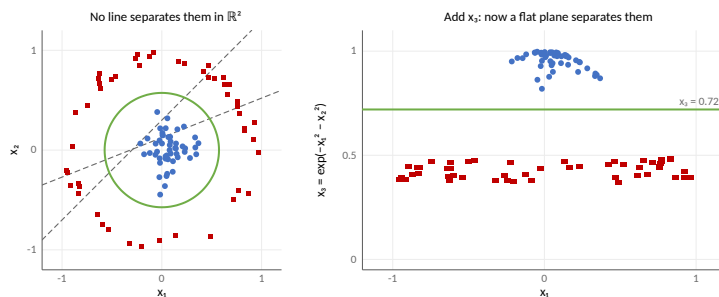
Some data can't be separated by any hyperplane. **Fix:** map every point to a richer feature space, $\mathbf{x} \mapsto \phi(\mathbf{x})$, where a hyperplane **does** separate them.

Simplest example, in 1-D: the blue class sits in the middle, red on both sides. No single threshold works.

Add the feature x^2 : $\phi(x) = (x, x^2)$. In that plane the points lie on a parabola, and a **horizontal line** separates them.

Back on the original axis, that line is **two** thresholds: a hyperplane in ϕ -space is a curved boundary in $\mathbf{x}$ -space.

SVM: kernels, a 2-D example



$$\phi(\mathbf{x}) = (x_1, x_2, e^{-(x_1^2 + x_2^2)}) \in \mathbb{R}^3$$

$$\phi(\mathbf{x}) \cdot \phi(\mathbf{z}) = x_1 z_1 + x_2 z_2 + e^{-(x_1^2 + x_2^2)} e^{-(z_1^2 + z_2^2)}$$

- The flat plane $x_3 = 0.72$ in $\mathbb{R}^3$ is, back in $\mathbb{R}^2$, the circle $x_1^2 + x_2^2 = -\ln 0.72 = 0.33$ (radius 0.57, green on the left).
- The dot product in $\mathbb{R}^3$ can be computed **directly from the 2-D vectors**. We never need to build $\phi(\mathbf{x})$ itself: that is the kernel trick.

SVM: the kernel trick

The dual and the prediction only use dot products $\mathbf{x}_i \cdot \mathbf{x}_j$.
Replace them by a **kernel**:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j)$$

We compute K , never $\phi(\mathbf{x})$: a linear SVM in the big space, at the cost of the small one.

Common kernels:

- Linear: $K(\mathbf{x}, \mathbf{z}) = \mathbf{x} \cdot \mathbf{z}$
- Polynomial: $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z} + c)^d$
- RBF (Gaussian): $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$, with $\gamma = \frac{1}{2\sigma^2}$. Its ϕ is infinite-dimensional.

Why the polynomial kernel works (2-D, $c = 1, d = 2$):

$$\begin{aligned} (\mathbf{x} \cdot \mathbf{z} + 1)^2 &= (x_1 z_1 + x_2 z_2 + 1)^2 \\ &= x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2 + 2x_1 z_1 + 2x_2 z_2 + 1 \\ &= \phi(\mathbf{x}) \cdot \phi(\mathbf{z}), \quad \phi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1 x_2) \end{aligned}$$

Check with numbers: $\mathbf{x} = (1, 2), \mathbf{z} = (3, 1)$. Kernel:

$$\begin{aligned} (3 + 2 + 1)^2 &= 36. \text{ Explicitly: } \phi(\mathbf{x}) = (1, \sqrt{2}, 2\sqrt{2}, 1, 4, 2\sqrt{2}), \\ \phi(\mathbf{z}) &= (1, 3\sqrt{2}, \sqrt{2}, 9, 1, 3\sqrt{2}), \text{ and} \\ \phi(\mathbf{x}) \cdot \phi(\mathbf{z}) &= 1 + 6 + 4 + 9 + 4 + 12 = 36 \checkmark \end{aligned}$$

For $n = 100$ features and degree 3, ϕ would have $\binom{103}{3} = 176,851$ coordinates. The kernel still costs one 100-D dot product.

SVM: the RBF kernel

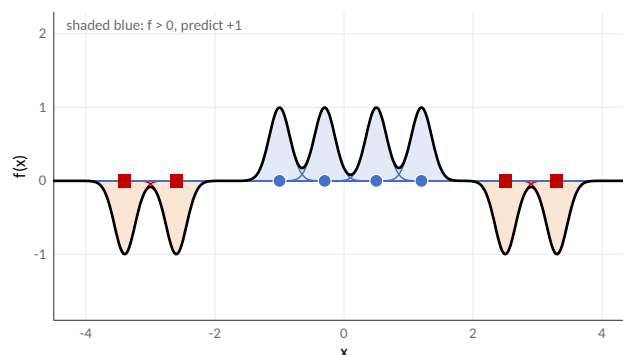
The dual, with kernels:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

with the same constraints $\alpha_i \geq 0$, $\sum_i \alpha_i y_i = 0$. The decision function for a new point:

$$f(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

RBF: $K(\mathbf{x}_i, \mathbf{x}) = e^{-\gamma \|\mathbf{x} - \mathbf{x}_i\|^2}$ is a bump centred on $\mathbf{x}_i$. So $f(\mathbf{x})$ is a **sum of Gaussian bumps**, one per support vector, up for +1, down for -1, with height α_i . γ sets their width.



γ (log scale) **20.0**

Here all $\alpha_i = 1$ and $b = 0$, to show the shape. Tune C and γ together by cross-validation on a log grid.

► Large γ : narrow bumps around each support vector: it memorizes the training set

SVM: which functions are kernels? (Mercer)

A function K is a valid kernel only if it really is a dot product $\phi(\mathbf{x}) \cdot \phi(\mathbf{z})$ in some feature space. **Mercer's theorem** gives a test on the **kernel (Gram) matrix** of any set of points $\mathbf{x}_1, \dots, \mathbf{x}_m$:

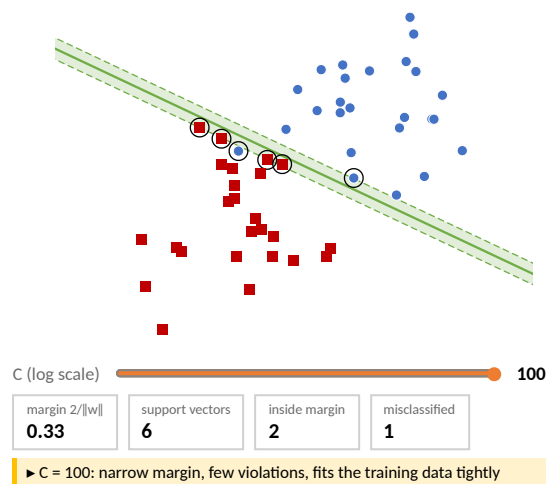
$$K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$$

K must be **positive semi-definite**: $\mathbf{v}^\top K \mathbf{v} \geq 0$ for every vector $\mathbf{v}$. This keeps the dual a concave maximization (a convex problem), with one global optimum.

Check for a real feature map: $\mathbf{v}^\top K \mathbf{v} = \sum_{i,j} v_i v_j \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j) = \left(\sum_i v_i \phi(\mathbf{x}_i) \right) \cdot \left(\sum_j v_j \phi(\mathbf{x}_j) \right) = \left\| \sum_i v_i \phi(\mathbf{x}_i) \right\|^2 \geq 0$.

Key takeaway: a valid kernel always produces a positive semi-definite kernel matrix. Linear, polynomial and RBF kernels all pass the test.

SVM: the soft margin



Real data overlap: no $\mathbf{w}$ satisfies every constraint. Allow each point a **slack** $\xi_i \geq 0$, and pay for it:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- $\xi_i = 0$: on the correct side, outside or on the margin. $0 < \xi_i \leq 1$: correct side, but inside the margin. $\xi_i > 1$: misclassified.
- C is the **regularization parameter**: margin width against violations.
 - Large C : violations are expensive $\rightarrow$ narrow margin, fits the training data tightly. $C \rightarrow \infty$ is the hard margin.
 - Small C : wide margin, many violations tolerated.

SVM: the soft margin in scikit-learn

The C of the previous slide is exactly the C hyperparameter of `SVC`. It minimizes the margin term plus C times the total violation:

$$\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i$$

```
from sklearn.svm import SVC
model = SVC(kernel="linear", C=1.0) # C = 1 is the default
model.fit(X_train, y_train)
```

	small C	large C
a violation costs	little	a lot
margin	wide	narrow
support vectors	many	few
risk	underfitting	overfitting

- Choose C on a **log grid** (0.01, 0.1, 1, 10, 100) with cross-validation. It is the slider of the previous slide: $\log_{10} C$ from -2 to 2.
- **Scale the features first** (`StandardScaler`): the margin is a distance. Put the scaler and the SVM in one `Pipeline`.
- With the RBF kernel, tune C and γ together (`gamma="scale"` is the default).
- As in `LogisticRegression(C=...)`: a big C means **weak** regularization.

```
pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
grid = {"svc__C": [0.1, 1, 10, 100],
        "svc__gamma": [0.01, 0.1, 1]}
search = GridSearchCV(pipe, grid, cv=5)
search.fit(X_train, y_train)
search.best_params_
```

SVM: review

Pros

- Kernels give flexible, non-linear boundaries while training stays a convex problem: one global optimum.
- Maximum margin: good results even with smaller datasets.
- Sparse: the model only keeps the support vectors.

Cons

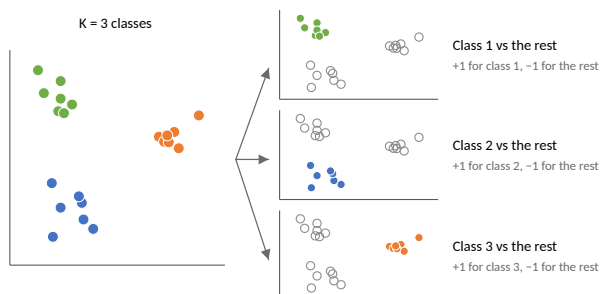
- Kernel training is slow on big data: roughly $O(m^2)$ to $O(m^3)$. Beyond $\sim 10^5$ examples, use a linear SVM.
- Prediction with kernels costs one K per support vector.
- Binary only: one-vs-rest or one-vs-one for more classes.
- No probabilities out of the box (Platt scaling fits a sigmoid on top).
- **Scale the features:** the margin is a Euclidean distance.
- Hyperparameters C and γ must be tuned by cross-validation.

	Logistic regression	Linear SVM
regularizer	L2 (usually)	L2 (always: it is the margin)
points that matter	all	support vectors
output	probability	score / class

Interesting reads:

- Support Vector Regression: [scikit-learn user guide, §1.4.2](#)

SVM: more than two classes



One-vs-rest: train K binary SVMs, one per class, with $y = +1$ for class k and -1 for the rest.

Winner takes all: predict the class with the highest score $f_k(\mathbf{x}) = \mathbf{w}_k \cdot \mathbf{x} + b_k$.

One-vs-one: one SVM for every pair of classes, $\frac{K(K-1)}{2}$ of them, each on less data; predict by majority vote.

scikit-learn: `LinearSVC` uses one-vs-rest; `SVC` uses one-vs-one.

LAB 5

Support vector machines

The maximum-margin classifier with scikit-learn, from the picture to a tuned model.

- 1 Why the maximum-margin boundary is a sensible choice.
- 2 Fit an SVM; find the support vectors and see that only they define the model.
- 3 The kernel trick for curved boundaries.
- 4 Soft margin: the role of C on overlapping classes.
- 5 Tune the kernel, C and γ with cross-validation, in a pipeline.

[CO Open in Colab](#)

notebook Lab 5 - Support Vector Machines.ipynb
you need this chapter: margin, support vectors, kernels, soft margin and C ; pipelines and cross-validation
runs on CPU; every cell runs in seconds

Next: a bonus lab on time series (next slide), then the neural-network part.



Universidad
de Navarra

35

BONUS · LAB 16

Linear regression for time series

Same model $\hat{y} = \theta^\top \mathbf{x}$, same least squares. New data: **days in order**. Today looks a lot like yesterday, so the examples are **not independent**.

The model does not change, the **method** does:

1. **Never shuffle**. Train on the past, test on the future.
2. **Features only from the past**: yesterday y_{t-1} , last week y_{t-7} . A 7-day mean that includes today gives $R^2 = 1.00$: arithmetic, not forecasting.
3. **Beat a naive forecast** ("same as last week"), not the mean. On the test year the lab's model halves its error (MAE 4.3 vs 8.5).
4. **Cross-validate forward** with `TimeSeriesSplit`.

Random split (shuffled)



test $R^2 = 0.89$: too good to be true, each test day sits between two training days (dashed box)

Time order



test $R^2 = 0.66$: the honest score

time

[CO Open in Colab](#)

notebook Lab 16 - Linear Regression for Time Series.ipynb
runs on CPU; the whole lab runs in seconds



Universidad
de Navarra

36

Machine Learning

Máster en Innovación Tecnológica (MINT)

Thank you!

imanolgranada@gmail.com



37